

Wandah Wibawanto

Game Edukasi RPG (Role Playing Game)



Software



Adobe Flash/Animate

GAME EDUKASI RPG

(ROLE PLAYING GAME)

Wandah Wibawanto

Penerbit LPPM UNNES

Game Edukasi RPG (*Role Playing Game*)

© All Rights Reserved

Hak cipta dilindungi undang-undang

Penulis:

Wandah Wibawanto, S.Sn. M.Ds.

Desain & Tata Letak:

Wandah Wibawanto, S.Sn. M.Ds.

Penerbit :

Penerbit LPPM UNNES

Gedung Prof. Retno Sriningsih Satmoko Lt 1

Universitas Negeri Semarang

Kampus Sekaran, Gunungpati, Semarang

e-mail: lp2m@mail.unnes.ac.id

ISBN : 978-623-7618-61-4

Nomor Pencatatan Hak Cipta : 000182048

Cetakan Pertama : 2020

Undang - Undang RI Nomor 19 Tahun 2002 Tentang Hak Cipta

Ketentuan Pidana

Pasal 72 (ayat 2)

Barang Siapa dengan sengaja menyiarkan, memamerkan, mengedarkan, atau menjual kepada umum suatu ciptaan atau barang hasil pelanggaran Hak Cipta atau hak terkait sebagaimana dimaksud pada ayat (1), dipidana dengan pidana penjara paling lama 5 (lima) tahun dan/atau denda paling banyak Rp. 500.000.000,00 (lima ratus juta rupiah).

KATA PENGANTAR

Alhamdulillah robbil'aalamin, buku ini selesai ditulis setelah sekian lama terbesit keinginan untuk menyusun tutorial tentang pengembangan game bertipe *Role Playing* (RPG). Rasa syukur yang terdalam saya sampaikan karena diberikan kekuatan dan kemudahan dalam penulisan buku pengembangan game ke-6 ini.

Game Digital (komputer, konsol dan *mobilephone*) merupakan salah satu media yang dapat digunakan untuk menyampaikan materi pembelajaran kepada pemainnya dengan sangat efektif dan efisien. Dalam beberapa tahun terakhir, game edukasi semakin banyak dikembangkan. Salah satu jenis game yang seringkali ingin dikembangkan oleh pemula maupun profesional adalah game bertipe *Role Playing* (RPG). Disisi lain game bertipe RPG memiliki sistem yang lebih kompleks jika dibandingkan dengan kategori lainnya.

Seiring perkembangan waktu, terdapat beberapa aplikasi atau *software* untuk mempermudah kita dalam mengembangkan game RPG seperti RPG Maker, Unity, Construct dan sebagainya. Saya sengaja memilih *software* Adobe Flash atau Adobe Animate, karena *software* ini mudah didapatkan dan merupakan bagian dari kurikulum multimedia baik di tingkat Sekolah Menengah sampai Perguruan Tinggi. Pengembangan game RPG dengan Adobe Flash/Animate tidak serumit yang dibayangkan oleh sebagian besar pembelajar.

Minimnya literatur yang membahas baik secara teori maupun secara praktik tentang proses pengembangan game edukasi bertipe *Role Playing* (RPG) menjadikan kuantitas dan kualitas game jenis ini yang masih minim. Sebagai respon dari permasalahan tersebut, melalui buku ini saya mencoba untuk membahas secara konseptual pengembangan game RPG dengan muatan edukasi. Pembahasan dalam bentuk teori dan praktik, dilengkapi dengan penjelasan di tiap-tiap program, dengan langkah-langkah yang paling mudah untuk diikuti oleh semua kalangan. Semoga buku ini membawa kebermanfaatan bagi perkembangan game edukasi, dan mohon maaf sebesar-besarnya apabila banyak kekurangan yang ada dalam buku ini.

Penulis,

DAFTAR ISI

BAB I PENDAHULUAN

1.1 Game sebagai Media Pembelajaran	1
1.2 Tujuan Buku.....	2
1.3 Kebutuhan <i>Software</i> dalam Pengembangan Game di Buku Ini.....	3
1.4 Material Penunjang yang Dibutuhkan	5
1.5 Tutorial Teknik Dasar Adobe Animate/Flash.....	5
1.6 Lisensi Buku.....	6

BAB II *ROLE PLAYING GAME* (RPG)

2.1 Definisi <i>Role Playing Game</i> (RPG)	7
2.2 Cerita dalam RPG	7
2.3 Karakter Pemain	8
2.4 Arena Permainan (<i>Map</i>).....	9
2.5 Misi (<i>Quest</i>).....	10
2.6 Karakter Pendukung / <i>Non Playable Character</i> (NPC).....	11
2.7 Mode Pertempuran (<i>Battle System</i>)	11
2.7 RPG sebagai Game Edukasi.....	11

BAB III ADOBE ANIMATE / FLASH

3.1 Aplikasi Adobe Animate / Flash	13
3.2 Menuliskan Kode <i>Actionscript</i>	19
3.3 Dasar Pemrograman <i>Actionscript 3</i>	20
3.4 Sistem Koordinat Layar.....	25
3.5 Menambahkan Objek ke Layar dengan Kode	26
3.6 Membuat Gerakan Objek	31
3.7 Membuat Gerakan Objek dengan <i>Input Keyboard</i>	34

BAB IV SISTEM TILING DALAM GAME RPG

4.1 Sistem Tiling/Pengulangan	37
4.2 Mengenal <i>Array</i>	39
4.3 Pemrograman Dasar Sistem <i>Tiling</i>	40
4.4 Mengatur Gerakan Karakter dalam Sistem <i>Tiling</i>	45
4.5 Menggambar Peta dengan Menggunakan <i>Bitmap Data</i>	48

BAB V PENGEMBANGAN PETA / ARENA PERMAINAN

5.1 Aplikasi Editor untuk Mengembangkan Peta / Arena Permainan	55
5.2 Map Editor RPG berbasis <i>Bitmap Data</i>	56
5.3 Membuat Peta / Arena Permainan dengan Map Editor.....	58
5.4 Menguji Coba Peta / Arena Permainan.....	62
5.5 Menambahkan Pintu dan Item ke dalam Arena Permainan	64

BAB VI KARAKTER DALAM GAME RPG

6.1 Pembentukan Karakter dalam Game RPG	67
6.2 Karakter dalam Game RPG sebagai Agen Pedagogis.....	71
6.3 Menambahkan Karakter ke Arena Permainan.....	73

BAB VII PERPINDAHAN AREA DALAM RPG

7.1 Desain Arena Permainan	77
7.2 Mengembangkan Peta Permainan.....	80
7.3 Menguji Coba Perpindahan Area Permainan	82
7.4 Pengembangan peta lebih lanjut.....	83

BAB VIII GUI DAN SISTEM DIALOG

8.1 <i>Graphical User Interface</i> (GUI) dalam game RPG.....	87
8.2 Membuat Panel GUI.....	90
8.3 Membuat Sistem Dialog dalam Permainan.....	93

BAB IX NON PLAYABLE CHARACTER (NPC)

9.1 Konsep NPC atau Pemain Pendukung dalam RPG.....	97
9.2 Mengembangkan Grafis NPC (<i>npc sprites</i>).....	98
9.3 Menambahkan <i>Ordinary</i> NPC pada Arena Permainan	100
9.4 Menambahkan <i>Services</i> NPC pada Arena Permainan.....	103
9.5 Menambahkan <i>Services</i> NPC bertipe <i>Trading</i>	105
9.6 Menambahkan <i>Services</i> NPC bertipe <i>Trading</i> untuk item	108
9.7 Menambahkan data item	110

BAB X MUSUH DAN BATTLE SYSTEM

10.1 Konsep <i>Battle System</i> dalam game RPG	113
10.2 Mendesain karakter musuh (<i>enemy sprites</i>) berikut kemampuannya	115
10.3 Mengimplementasikan Sistem Kuis pada <i>Battle System</i>	119
10.4 Menyusun <i>database</i> Soal untuk Sistem Pertempuran	121

BAB XI HALAMAN PEMBUKA GAME (*TITTLE SCREEN*)

11.1 Mendesain Halaman Pembuka Permainan	125
11.2 Menambahkan Halaman Pembuka dan halaman <i>Game Over</i>	126

BAB XII SUARA DALAM GAME RPG

12.1 Konsep Suara dalam Game.....	129
12.2 Mengatur File Suara untuk Game RPG	130
12.3 Menambahkan Suara ke dalam Game.....	131

BAB XIII *PUBLISHING*

13.1 Tahap Terakhir dalam Pengembangan Game.....	135
13.2 <i>Publishing</i> untuk PC/Desktop.....	136
13.3 <i>Publishing</i> untuk Android	136

BAB XIV PENUTUP	143
------------------------------	------------

DAFTAR ISI

Gambar 1.1 Hasil dari tutorial di buku ini.....	2
Gambar 1.2 <i>Software</i> Adobe Animate CC 2019.....	3
Gambar 1.3 Tampilan <i>software</i> Flash Develop	4
Gambar 2.1 Karakter Cloud Strife dan karakter Link	9
Gambar 2.2 NPC dalam game RPG	11
Gambar 2.3 <i>Realtime Battle system</i> dan <i>Turn Based Battle System</i>	11
Gambar 2.4 Game RPG simulasi pertanian	12
Gambar 3.1 Ruang Kerja Adobe Animate <i>Classic Mode</i>	13
Gambar 3.2 Mengganti mode ruang kerja / antar muka Adobe Animate.....	14
Gambar 3.3 Main menu dengan opsi File aktif.....	14
Gambar 3.4 <i>Timeline</i> pada Adobe Animate/Flash.....	15
Gambar 3.5 <i>Tools</i> pada Adobe Animate.....	15
Gambar 3.6 <i>Library</i>	17
Gambar 3.7 Panel <i>properties</i>	18
Gambar 3.8 Panel <i>action</i>	18
Gambar 3.9 Membuat File baru bertipe <i>Actionscript 3</i>	19
Gambar 3.10 Menuliskan kode di panel <i>action</i>	19
Gambar 3.11 Kata <i>Hello World</i> pada panel <i>output</i>	19
Gambar 3.12 Sistem kordinat layar	26
Gambar 3.13 Pengaturan awal dokumen aplikasi	27
Gambar 3.14 Mengimport gambar ke <i>Stage</i>	27
Gambar 3.15 Tahapan membuat <i>Movieclip</i>	28
Gambar 3.16 Pengaturan <i>movieclip</i> ayamMC.....	28
Gambar 3.17 <i>Movieclip</i> ayamMC pada panel <i>Library</i>	29
Gambar 3.18 Tampilan kode pada panel <i>action</i>	29
Gambar 3.19 Hasil tutorial	29
Gambar 3.20 Hasil tutorial 3.....	32
Gambar 3.21 Kode <i>keyboard</i> dan gerakan ayam.....	35
Gambar 3.22 Kode <i>keyboard</i>	36
Gambar 4.1 Game Super Mario Bros	37
Gambar 4.2 <i>Tileset</i> dan sistim <i>tiling</i> pada game Super Mario.....	38
Gambar 4.3 <i>Tilling</i> pada Game The Legend of Zelda.....	38
Gambar 4.4 Game Tactics Ogre.....	39
Gambar 4.5 Astro Farmer.....	39
Gambar 4.6 Membuat <i>tile</i> dengan <i>rectangle tool</i>	40

Gambar 4.7 Membuat <i>movieclip tiles</i>	41
Gambar 4.8 Mode edit <i>movieclip tiles</i>	41
Gambar 4.9 Menambahkan <i>keyframe</i> pada <i>movieclip tiles</i>	41
Gambar 4.10 Membuat <i>movieclip char</i>	42
Gambar 4.11 Hasil sistem <i>tile</i> dengan data <i>array</i>	43
Gambar 4.12 Visualisasi data <i>array</i> dalam sistem <i>tiling</i>	45
Gambar 4.13 Sistem deteksi gerakan dalam sistem <i>tiling</i>	48
Gambar 4.14 Game Sim Taxi menggunakan teknik <i>frame by frame tiling</i>	49
Gambar 4.15 Proyeksi <i>tileset</i> dengan data <i>array</i>	49
Gambar 4.16 Mengimport <i>tileset</i> ke <i>stage</i>	50
Gambar 4.17 Pengaturan properties <i>bitmap</i>	50
Gambar 4.18 Mengatur panel <i>Actionscript</i> untuk <i>symbol bitmap</i>	50
Gambar 4.19 Struktur data yang dihasilkan oleh editor	51
Gambar 4.20 Hasil sistem <i>tiling</i> dengan metode <i>bitmapData</i>	53
Gambar 5.1 Game Sim Taxi dengan pengembangan peta secara manual	55
Gambar 5.2 Map Editor untuk pengembangan game berbasis <i>tile</i>	56
Gambar 5.3 <i>Interface</i> map editor FlashRPG	57
Gambar 5.4 Membuat file baru	58
Gambar 5.5 Mode <i>click</i> dan <i>brush</i> pada <i>tile</i> editor	58
Gambar 5.6 Konsep 9 <i>grid</i> dalam membentuk arena permainan	59
Gambar 5.7 Mode layer 1 untuk <i>background</i>	59
Gambar 5.8 Mode layer 2 untuk objek di atas <i>background</i>	60
Gambar 5.9 Gambar hasil pengeditan <i>tile</i>	60
Gambar 5.10 Mode edit <i>walk layer</i>	60
Gambar 5.11 Menambahkan <i>tile mode unwalkable/wall</i>	61
Gambar 5.12 Menyimpan file arena permainan	61
Gambar 5.13 Mengimport <i>tileset</i> ke <i>stage</i>	62
Gambar 5.14 Pengaturan <i>properties bitmap</i>	62
Gambar 5.16 Mengatur panel <i>Actionscript</i> untuk <i>symbol bitmap</i>	63
Gambar 5.17 Hasil ujicoba arena permainan	63
Gambar 5.18 menambahkan fitur pintu pada arena permainan	64
Gambar 5.19 Hasil dari penambahan door id	64
Gambar 5.20 Menambahkan item ke arena permainan	65
Gambar 5.21 Contoh peletakan item id pada arena permainan	65
Gambar 6.1 Karakter Astronot Petani pada game Astro Farmer	67
Gambar 6.2 Character Sprites Link dalam Game Zelda	68
Gambar 6.3 Karakter dalam Ace Gangster	68
Gambar 6.4 Pemberian misi dan <i>reward</i> pada Astro Farmer	69

Gambar 6.5 Referensi gambar anak pramuka	70
Gambar 6.6 Sketsa kasar karakter	70
Gambar 6.7 Sketsa final karakter	70
Gambar 6.8 Hasil digitalisasi dan perwarnaan karakter	70
Gambar 6.9 <i>Sprite</i> Karakter dengan animasi berjalan	71
Gambar 6.10 Contoh agen pedagogis dalam game Astro Farmer	72
Gambar 6.11 Agen pedagogis dalam game My New School	73
Gambar 6.12 <i>Sprites</i> karakter yang akan ditambahkan	73
Gambar 6.13 Mengimport karakter <i>sprites</i> ke <i>library</i>	74
Gambar 6.14 Menambahkan <i>linkage</i> pada gambar	74
Gambar 6.15 Hasil penambahan karakter	75
Gambar 7.1 Perencanaan peta dengan <i>door_id</i> dan <i>item_id</i>	77
Gambar 7.2 Contoh ukuran peta Game Zelda dan GTA Vice City	78
Gambar 7.3 Contoh peta yang memfasilitasi perkembangan karakter	79
Gambar 7.4 Peletakan NPC pada game Shining Force 2	79
Gambar 7.6 Peletakan item pada game Final Fantasy 2	80
Gambar 7.7 Perencanaan peta permainan untuk <i>door_id</i> 3	81
Gambar 7.8 Membuat peta baru dengan Map ID 3	81
Gambar 7.9 Pengembangan interior inn (map-3) pada map editor	82
Gambar 7.10 Hasil tutorial 10	83
Gambar 7.11 Pengembangan peta map-4	83
Gambar 7.12 Desain peta untuk map-5 dan map-6	84
Gambar 7.13 <i>Open Area</i> pada map-7	84
Gambar 8.1 <i>Tittle Screen</i> game Baldur's Gate	87
Gambar 8.2 Seting awal game Pokemon Emerald	87
Gambar 8.3 Panel <i>heroes properties</i> game Shining Force	88
Gambar 8.4 Panel jual beli game Secrets of Grindea	88
Gambar 8.5 Panel dialog dalam game FF7	88
Gambar 8.6 GUI set yang digunakan dalam tutorial	89
Gambar 8.7 File <i>UI_set</i> dan <i>Face_set</i>	90
Gambar 8.8 Mengimport file <i>UI Set</i>	90
Gambar 8.9 Hasil tutorial pengaturan GUI	91
Gambar 8.10 Hasil metode <i>draw_from</i>	92
Gambar 8.11 Membuat file bertipe <i>actionsript</i>	93
Gambar 8.12 Hasil sistem dialog	94
Gambar 9.1 Karakter <i>villain</i> dalam game FF7	97
Gambar 9.2 NPC pada game Golden Sun:Dark Down	97
Gambar 9.3 NPC Quest dalam game RDR2	98

Gambar 9.4 <i>Ordinary</i> NPC dalam Game Pokemon Omega Ruby	98
Gambar 9.5 Contoh NPC sprites yang disesuaikan dengan <i>avatar</i> NPC	99
Gambar 9.6 Posisi grafis pada <i>tilegrid</i>	99
Gambar 9.7 Mengedit map-2 dengan map editor	100
Gambar 9.8 Mengatur penambahan NPC	100
Gambar 9.9 Posisi peletakan NPC pada arena permainan	101
Gambar 9.10 Mengimport NPC <i>sprites</i>	101
Gambar 9.11 Menambahkan <i>linkage</i> pada NPC <i>Sprites</i>	101
Gambar 9.12 Membuat file <i>actionscript</i> baru	102
Gambar 9.13 Hasil penambahan NPC.....	103
Gambar 9.14 Penempatan NPC pada map-3	103
Gambar 9.15 <i>Sprites</i> NPC 2 dan 3	104
Gambar 9.16 Peletakan NPC 4 dan 5 pada map-4.....	105
Gambar 9.17 <i>Sprites</i> NPC 4 dan 5	106
Gambar 9.18 gambar untuk item set	106
Gambar 9.19 Hasil tutorial NPC <i>trading</i>	107
Gambar 9.20 Peletakan NPC 6 pada map-5.....	108
Gambar 9.21 <i>Sprites</i> NPC 6	108
Gambar 9.22 Hasil tutorial NPC <i>trading</i>	110
Gambar 9.23 Hasil tutorial peletakan item.....	112
Gambar 10.1 Final Fantasy dengan kemunculan musuh secara acak	114
Gambar 10.2 Sistem pertempuran dalam game Kingdom Hearts.....	114
Gambar 10.3 Sistem <i>turnbased</i> pada game Pokemon	115
Gambar 10.4 <i>Sprites</i> untuk NPC musuh	116
Gambar 10.5 Menyesuaikan atribut karakter musuh 1.....	116
Gambar 10.6 Membuat file <i>actionscript</i>	117
Gambar 10.7 Mengimport semua <i>sprites</i> musuh ke <i>library</i>	118
Gambar 10.8 <i>Linkage</i> pada <i>sprites</i> musuh.....	118
Gambar 10.9 Hasil tutorial penambahan musuh	119
Gambar 10.10 Referensi sistem kuis	119
Gambar 10.11 <i>Mockup</i> sistem pertempuran.....	120
Gambar 10.12 <i>Background</i> untuk sistem pertempuran.....	120
Gambar 10.6 Membuat file <i>actionscript</i>	121
Gambar 10.13 Hasil penambahan sistem pertempuran	122
Gambar 11.1 Halaman pembuka game dengan ilustrasi dan judul	123
Gambar 11.2 Halaman pembuka game dengan judul permainan	123
Gambar 11.3 Desain halaman pembuka permainan	124
Gambar 11.4 Ilustrasi halaman pembuka	124

Gambar 11.5 Ilustrasi <i>game over</i>	125
Gambar 11.6 Hasil penambahan halaman pembuka dan halaman <i>game over</i>	126
Gambar 12.1 File suara untuk game	129
Gambar 12.2 Membuat file <i>actionsript</i>	129
Gambar 12.3 Pengaturan kompresi suara	130
Gambar 12.4 Komparasi ukuran file.....	131
Gambar 13.1 Mengatur <i>publish settings</i> untuk PC/Desktop.....	134
Gambar 13.2 Hasil <i>publishing</i> berupa file EXE (<i>Application</i>)	134
Gambar 13.3 <i>Virtual Keyboard</i>	135
Gambar 13.4 Mengubah ukuran <i>stage</i>	136
Gambar 13.5 Halaman pembuka dengan resolusi 1200 x 600 <i>pixel</i>	136
Gambar 13.6 Mengupdate gambar <i>cover_bg</i>	136
Gambar 13.7 Mengatur target untuk <i>AIR for Android</i>	137
Gambar 13.8 Pengaturan tab <i>general Android Settings</i>	138
Gambar 13.9 Proses pembuatan sertifikat	138
Gambar 13.10 Pengaturan icon aplikasi	139
Gambar 13.11 Pengaturan tab <i>permissions</i> dan <i>Language</i>	139
Gambar 13.12 Proses <i>Publishing</i>	139
Gambar 13.13 Hasil game pada gawai <i>Android</i>	140

BAB I

PENDAHULUAN

Pemanfaatan game sebagai media belajar semakin banyak dilakukan oleh para pendidik baik di dalam maupun di luar negeri. Game sebagai media yang bersifat adiktif dan repetitif mampu dimanfaatkan untuk menyampaikan materi pelajar secara menarik kepada pemainnya.

PENDAHULUAN

1.1 Game sebagai Media Pembelajaran

Game sebagai media pembelajaran bukanlah sebuah konsep baru, konsep ini muncul seiring dengan munculnya berbagai macam game (permainan), baik itu game tradisional maupun game digital. Akibatnya secara spesifik muncul istilah game edukasi, yaitu game yang secara khusus memiliki muatan pembelajaran dan ditujukan untuk meningkatkan kemampuan pemainnya dalam mempelajari suatu materi. Game mampu memancing minat belajar pemainnya, sehingga menghasilkan pengalaman baru seperti perasaan senang yang pada akhirnya materi yang ingin disampaikan dapat diterima dengan mudah oleh pemain game.

Berbagai penelitian telah membuktikan efektifitas game dalam mempengaruhi pemainnya. Muatan atau konten yang berada di dalam game, dengan mudah dipahami oleh pemain hal ini dikarenakan interaktifitas yang ada dalam game, sekaligus imersi yang diberikan oleh game menjadikan pemain dalam kondisi yang paling rileks dan terbuka dalam menerima materi. Hal inilah yang dimanfaatkan oleh beberapa pengembang game untuk mempengaruhi pemain, dalam artian positif maupun negatif. Kecepatan game dalam mempengaruhi pemain ini sangat terkait dengan perkembangan siswa didik di era teknologi seperti ini.

Generasi “*gamer*”, mungkin inilah yang boleh saya sebut bagi siswa di era ini. Sebagian besar siswa pasti pernah memainkan game, baik melalui konsol seperti *Playstation*, melalui komputer, atau melalui gawai yang hampir semua memilikinya yaitu *mobilephone*. Generasi ini memiliki gaya kognitif yang ditandai dengan kemampuan *multitasking* sambil belajar, rentang perhatian yang pendek selama pembelajaran, dan pendekatan eksplorasi dalam belajar. Mereka lebih suka belajar melalui eksperimen daripada dengan instruksi langsung, mereka bergerak dengan mudah dan cepat dari satu informasi atau aktivitas ke yang lainnya, jika informasi tadi dinilai tidak menarik lagi bagi mereka. Mereka merespons dengan cepat sebuah pertanyaan dan menuntut respons cepat sebagai imbalan. Mereka mengharapkan interaktivitas pembelajaran, interaksi, visualisasi yang aktif dan interaktif, kinestesis, dan memiliki kedekatan dengan diri mereka. Semua itu dapat mereka dapatkan melalui sebuah game.

Game edukasi pada akhirnya harus memiliki umpan balik langsung, interaksi, partisipasi aktif, kontrol pemain atas materi pembelajaran, latihan berulang, tantangan, motivasi, dialog antara pemain, dan kerjasama tim. Hal inilah yang akan

dicoba untuk diaplikasikan melalui tutorial-tutorial dalam buku ini. Hasil akhir berupa game bertipe RPG yang memenuhi kriteria-kriteria game edukatif tersebut.

1.2 Tujuan Buku Ini

Buku ini secara spesifik akan menyampaikan materi tentang pengembangan game RPG (*Role Playing*) yang memiliki muatan edukasi. Pembaca akan diajak untuk mempelajari teknik pengembangan game secara bertahap, disertai dengan penjelasan-penjelasan tentang langkah-langkah yang dilakukan. Pada tahapan terakhir akan dihasilkan sebuah game RPG edukatif yang dapat dijalankan melalui komputer (PC), dan melalui gawai *mobilephone* berbasis Android.



Gambar 1.1 Hasil dari tutorial di buku ini

Selanjutnya buku ini ditujukan untuk beberapa *audiens* di antaranya :

1. Pengembang game edukasi

Buku ini sesuai untuk pengembang game edukasi, karena di dalamnya dibahas secara rinci teknis pengembangan game mulai dari sistem dasar (*game engine*) sampai tahapan *publishing*.

2. Pengembang proyek studi game edukasi

Pada satu dekade terakhir banyak siswa/mahasiswa yang mengembangkan proyek studi/tugas akhir berupa game edukasi. Buku ini diharapkan dapat menjadi sebuah referensi dan menjadi salah satu petunjuk pembuatan game dari awal sampai akhir.

3. Profesional

Bagi pengembang game profesional, buku ini dapat dijadikan sebagai referensi pembandingan antara satu teknik dengan teknik lain. Saya sangat berharap, pengembang game profesional memberikan masukan terhadap proses pengembangan game yang dijelaskan di dalam buku ini.

4. Awam yang baru belajar pengembangan game

Bagi orang awam yang baru belajar pengembangan game, buku ini juga sesuai karena kode utama untuk membangun game tidak dijelaskan secara panjang

lebar di buku ini, melainkan dilampirkan secara terpisah. Pembaca awam hanya butuh menuliskan beberapa baris kode untuk membangun game (kurang lebih hanya 25 baris kode untuk membentuk game secara utuh).

1.3 Kebutuhan *Software* dalam Pengembangan Game di Buku Ini

Buku ini menjelaskan teknik pengembangan game menggunakan aplikasi utama Adobe Animate. Secara spesifik beberapa *software* yang digunakan dalam buku ini adalah :

1. Adobe Animate / Adobe Flash.

Adobe Animate merupakan sebuah *software* multimedia yang diproduksi oleh Adobe untuk mengembangkan produk animasi, website, video interaktif dan media audio visual lainnya. Adobe Animate merupakan versi baru dari Adobe Flash dan Macromedia Flash.

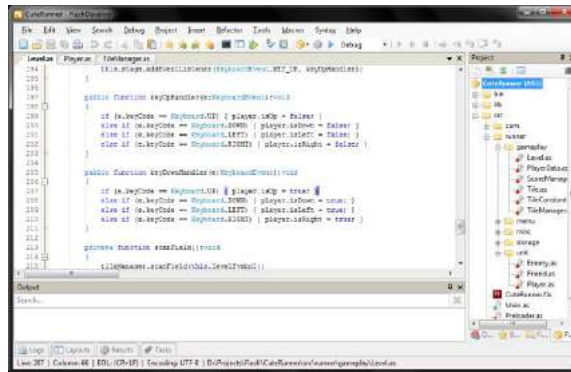


Gambar 1.2 *Software* Adobe Animate CC 2019

Adobe Animate memiliki dukungan bahasa pemrograman *actionscript* 3.0 yang digunakan untuk membentuk interaktivitas, sehingga dapat kita manfaatkan untuk membuat sebuah permainan digital. Versi yang mendukung kode *actionscript* 3.0 adalah Adobe Flash CS3 sampai dengan Adobe Animate CC 2019 (versi terakhir saat buku ini ditulis), sehingga buku ini dapat diaplikasikan pada *software* tersebut.

Apabila anda tidak memiliki versi terbaru dari Adobe Animate, anda tetap dapat menggunakan aplikasi dengan versi yang lebih lama yaitu Adobe Flash CS3, CS4, CS5, Adobe Animate CC, CC 2014, CC 2015, CC 2017 dan CC 2018.

Apabila anda kesulitan dengan lisensi *software* dan menginginkan tutorial dapat diterapkan pada aplikasi gratis (*freeware*), anda dapat menggunakan *software* Flash Develop dengan beberapa penyesuaian kode.



Gambar 1.3 Tampilan *software* Flash Develop

Flash Tips

Tutorial dalam buku ini ditulis untuk diterapkan pada *software* Adobe Animate/Flash. Sedangkan untuk pengembangan menggunakan Flash Develop anda membutuhkan sedikit penyesuaian kode. Teknik pengembangan game dengan Flash Develop dapat anda baca pada buku saya “Memprogram Flash Game 3D itu Mudah”, penerbit Andi 2013.

2. Photoshop

Software Photoshop digunakan untuk memproduksi dan mengedit grafik yang akan digunakan di dalam game. Photoshop merupakan *software* grafis yang secara umum mudah didapatkan, namun demikian anda juga dapat menggunakan *software* grafis lainnya seperti GIMP, Microsoft Paint dan *software* grafis lainnya untuk menghasilkan gambar yang anda butuhkan.

3. Notepad++

Notepad++ pada dasarnya hanya sebuah editor teks sederhana seperti halnya Notepad bawaan Windows. Aplikasi ini cukup ringan, gratis dan efektif digunakan untuk mengedit kode. Adobe Animate/Flash sebenarnya juga sudah dilengkapi dengan *code editor*, namun dalam praktik pengembangan selama lebih dari 15 tahun saya mendapati Notepad++ sangat bermanfaat untuk sekadar menulis kode, atau menyimpan baris kode secara sementara.

4. Audio Editor

Audio editor yang dimaksud disini adalah *software* untuk menghasilkan atau mengedit suara seperti Adobe Audition, Soundpad, Fruity loop, dan sebagainya. Pada tahapan tertentu dalam pengembangan game, dibutuhkan beberapa file audio (lihat BAB 12).

1.4 Material Penunjang yang Dibutuhkan

Tutorial di dalam buku ini membutuhkan beberapa material penunjang sebagai berikut :

1. Kode **flashRPG.as**

Merupakan kumpulan kode yang saya tulis untuk mempermudah proses pengembangan game bertipe RPG. Dalam pengembangan game yang ada pada tutorial 8-20, file flashRPG.as akan dibuka secara eksternal menggunakan perintah **include**. File flashRPG.as dapat anda peroleh melalui situs www.wandah.org/game-edukasi-RPG yang selanjutnya anda dapat memodifikasinya sesuai dengan kebutuhan.

2. Aset visual

Aset visual berupa grafik-grafik yang dibutuhkan untuk pengembangan game, meliputi :

- a. Tileset
- b. Karakter pemain
- c. Karakter NPC (*Non Playable Character*) dan musuh
- d. *Graphic User Interface* (GUI)
- e. dan grafik lainnya

Aset visual tersebut diperoleh dari situs penyedia grafik game, yang bersifat *free license* (boleh digunakan dengan mencantumkan kredit pembuat). Aset visual pada tutorial di buku ini diperoleh dari beberapa situs, yaitu :

- a. www.opengameart.org
- b. <https://pipoya.itch.io/>
- c. <https://forums.rpgmakerweb.com/index.php>
- d. dan sebagainya

3. Aset Audio

Aset audio berupa *background music* (BGM) dan *sound effect* (SFX) diperoleh dari situs www.freesound.org

1.5 Tutorial Teknik Dasar Adobe Animate/Flash

Dalam membangun sebuah media interaktif maupun game, anda butuh mengenal teknik dasar *software* Adobe Animate/Flash. Beberapa tulisan terdahulu saya yang terkait dengan pengembangan game/aplikasi berisi tentang teknik dasar Adobe Animate/Flash. Buku tersebut antara lain :

1. Membuat Game dengan Macromedia Flash (2003)
Buku pertama yang saya tulis menggunakan bahasa pemrograman *actionscript 1.0* merupakan buku dasar teknik pembuatan game sederhana, di dalamnya terdapat logika sederhana dalam pengembangan game.
2. Dasar-Dasar Pemrograman Flash Game (2005)
Buku ini menjelaskan secara teknis dan konseptual pengembangan game menggunakan Adobe Flash. Bagi pemula yang ingin mengembangkan game bertipe *arcade* dapat memanfaatkan buku ini.
3. Membuat Game Flash 3D itu Mudah (2013)
Buku ini secara teknis menjelaskan proses pengembangan game flash dengan grafis 3 Dimensi. Pengembangan game menggunakan kode *actionscript 3.0* dan memanfaatkan aplikasi gratis Flash Develop.
4. Desain dan Pemrograman Multimedia Pembelajaran Interaktif (2017)
Buku ini secara spesifik menjelaskan konsep dan teknis pengembangan multimedia pembelajaran interaktif. Di dalamnya terdapat teknik dasar penggunaan aplikasi Adobe Flash/Animate untuk animasi dasar, desain dan membangun interaktifitas.
5. Membuat Berbagai-bagai Game Android dengan Adobe Animate (2019)
Buku ini menjelaskan proses pembuatan game edukasi untuk platform Android. Terdapat beberapa pembahasan pengembangan game edukasi di antaranya game memilah sampah, game puzzle, kuis, game peta buta, dan game ular tangga.

Buku ini merupakan buku ke-6, dan secara spesifik lebih memfokuskan pada pengembangan game edukasi bertipe *Role Playing Game* (RPG).

1.6 Lisensi Buku

Buku ini diterbitkan secara online dalam bentuk *e-book* dan diedarkan secara gratis, dengan beberapa ketentuan sebagai berikut :

1. Pembaca diperbolehkan menyebarkan dan memperbanyak buku tanpa mengubah, menambah atau mengurangi konten buku.
2. Pembaca diperbolehkan mencetak buku untuk keperluan pribadi, dan tidak diperbolehkan mencetak untuk diperjual belikan (dilarang mengkomersialkan buku ini dalam bentuk apapun).
3. Penambahan atribut nama penulis (*credit*) diperlukan pada setiap aplikasi yang dihasilkan.
4. Pembaca diperbolehkan menggunakan dan memodifikasi file tutorial untuk keperluan apapun, termasuk keperluan komersial.



BAB II

ROLE PLAYING GAME (RPG)

Role Playing Game merupakan salah satu genre permainan dimana para pemainnya memerankan tokoh tertentu, dihadapkan pada permasalahan tertentu dalam membentuk suatu cerita.

ROLE PLAYING GAME (RPG)

2.1 Definisi Role Playing Game (RPG)

Sesuai dengan struktur kata yaitu *role playing* atau bermain peran, maka definisi sederhana dari RPG adalah permainan di mana pemain menjalankan peran karakter dalam sebuah cerita fiksi. Pemain bertugas untuk memerankan peran-peran yang ditentukan oleh narasi permainan, serta dituntut untuk melalui proses pengambilan keputusan terstruktur terkait pengembangan karakter. Peran yang dilakukan oleh pemain ditentukan oleh sistem aturan yang didefinisikan secara spesifik oleh pengembang permainan.

Pada dasarnya, terdapat beberapa bentuk permainan peran (*Role Play*), seperti *Tabletop RPG* (TRPG), *Live Action RPG* (LARP), *Massive Multi Player Online RPG* (MMO RPG), dan *Single player RPG* (RPG) dan beberapa klasifikasi lainnya. Di dalam buku ini secara spesifik hanya membahas tentang pengembangan *Single Player RPG*. Dalam *single player RPG* atau disebut sebagai RPG saja, secara umum pemain akan dihadapkan pada permasalahan tertentu dan diberikan misi (*quest*) untuk menyelesaikan permasalahan tersebut. Dalam perjalanan menyelesaikan misi tersebut perkembangan karakter seperti kekuatan (*power*), level, pengalaman (*experience*) dan kemampuan (*abilities*) merupakan kunci utama dalam permainan RPG. Misi (*quest*) yang diberikanpun juga sangat beragam dapat berupa taktik pertarungan, eksplorasi dan pencarian sesuatu serta penyelesaian teka-teki.

Sebuah game RPG harus memiliki elemen-elemen dasar, elemen tersebut antara lain : cerita, karakter dan perkembangannya, arena permainan, misi atau *quest*, *Non Playable Character* (NPC), dan mode pertempuran (*Battle System*). Penjelasan masing-masing elemen tersebut dapat diperjelas pada masing-masing bab.

2.2 Cerita dalam RPG

Dalam permainan peran, seorang pemain harus tahu sedang memerankan apa dalam cerita apa. Maka dari itu cerita dalam RPG merupakan elemen pertama yang melatarbelakangi seluruh kejadian yang ada di dalam permainan. Struktur cerita yang akan membawa pemain untuk mengembangkan karakter, menemukan masalah, dan menyelesaikan masalah (memenangkan permainan).

Dalam perkembangannya cerita dalam RPG dibagi menjadi 2 jenis yaitu cerita yang linear atau lurus dan cerita bercabang. Dalam kategori cerita linear, peraturan game akan mengarahkan karakter untuk menjalani cerita tertentu tahapan demi tahapan.

Game RPG klasik seperti Zelda, Shining Force, Final Fantasy generasi awal, menggunakan cerita linear dengan satu tugas utama. Dalam perkembangan selanjutnya cerita menjadi lebih berkembang dan menjadi dapat bercabang untuk menghasilkan akhir game yang berbeda-beda. Dalam kategori ini pemain mendapatkan kebebasan atau opsi dalam menentukan jalannya cerita. Dalam RPG dengan kategori cerita bercabang, tindakan pemain selama permainan, seperti interaksi ke NPC, menjalankan sebuah *quest*, sampai dengan dialog-dialog kunci, dapat memberikan konsekuensi pada kelanjutan cerita.

Memasukkan elemen edukasi ke dalam sebuah RPG, dapat dilakukan melalui pendekatan cerita. Menyusun cerita dalam game edukatif RPG menjadi tantangan tersendiri, komposisi antara cerita fiktif yang dibangun dengan materi edukasi harus seimbang, sehingga pemain tidak merasa digurui selama permainan. Sebagai contoh dalam game yang dikembangkan di buku ini, disusun cerita yang mengandung elemen edukasi sebagai berikut :

“Budi seorang siswa tertidur di kelas dan tiba-tiba terbangun di sebuah desa Ketpi (kebalikan dari IPTEK). Dia harus bertanya ke orang sekitar di desa tentang keberadaannya, dan mendapati bahwa dia harus mencapai level 5 untuk dapat mengaktifkan portal menuju ke tempat asalnya. Untuk mencapai level 5 dia harus bertempur melawan musuh di hutan, dimana musuh akan memberikan beberapa pertanyaan. Jika karakter menjawab benar, maka karakter akan menyerang musuh dan sebaliknya. Pertempuran akan berakhir jika salah satu kehabisan energi, dan setiap mengalahkan musuh, karakter pemain akan mendapatkan pengalaman untuk menaikkan levelnya”.

Pada konsep cerita di atas, terdapat elemen fiktif yaitu pindahnya karakter ke desa fiktif, konsep melawan monster dan konsep portal untuk menyelesaikan permainan. Namun juga terdapat elemen edukasi yaitu pertanyaan yang diberikan musuh saat pertempuran, pemain secara langsung “dipaksa” untuk belajar menjawab dengan benar dan berulang-ulang untuk dapat mengalahkan musuh dan menaikkan level.

2.3 Karakter Pemain

Elemen kedua adalah karakter pemain, yaitu karakter utama yang dimainkan oleh pemain. Dalam sebuah game RPG terdapat 2 kategori karakter pemain, yaitu karakter pemain tunggal dimana pemain hanya menjalankan satu pemain saja dari awal sampai akhir permainan. Kategori kedua adalah multi karakter, dimana pemain mengendalikan seorang karakter utama dan beberapa karakter lainnya.

Karakter dalam game memiliki atribut khusus yang dapat berkembang seiring permainan berjalan, atribut tersebut secara umum antara lain adalah :

1. Energi pemain (HP)
2. *Level* pemain, yang sangat terikat dengan atribut pengalaman dan akan naik ketika pengalaman mencapai titik tertentu. Seiring dengan naiknya *level*, maka atribut pemain lainnya akan naik. Sebagai contoh ketika *level* 1 HP maksimal pemain bernilai 10, dan setelah *level* pemain naik menjadi level 2, HP maksimal naik menjadi 15 poin.
3. Pengalaman (EXP), yang didapatkan ketika pemain menyelesaikan permasalahan tertentu, seperti memenangkan perang atau mendapatkan item khusus.
4. Kemampuan (*abilities*), yaitu statistik yang dimiliki oleh pemain, pada umumnya terdiri dari kemampuan menyerang (*attack*), kemampuan bertahan (*defend*), kecepatan (*speed*), skill sihir (*magic*) dan sebagainya.

Desain karakter pemain juga meliputi tampilan visual karakter, wajah (*avatar*) karakter, dan properti yang melekat pada pemain seperti pakaian dan aksesoris. Sering kali karakter sebuah game RPG menjadi icon yang selalu diingat karena tampilan visualnya. Sebagai contoh karakter Cloud Strife pada game Final Fantasy 7, sangat populer dengan rambut berwarna kuning dan pedang besarnya, sedangkan karakter Link pada game Zelda sangat iconic dengan pakaian warna hijau dan topi kerucut.



Gambar 2.1 Karakter Cloud Strife pada Game Final Fantasy 7 (*Copyright* Square Enix, 1997) dan karakter Link pada game Zelda: the Minish Cap (*Copyright* Capcom, 2004)

2.4 Arena Permainan (*Map*)

Game RPG identik dengan eksplorasi arena permainan atau seringkali disebut dengan *map*. Arena permainan didesain sedemikian rupa mengikuti tema cerita dan perkembangan karakter, sekaligus peletakan beberapa rahasia seperti pintu, ruang rahasia, item-item tertentu.

Mendesain arena permainan dapat dilakukan melalui berbagai macam teknik, mulai dari teknik *art base* yaitu menggambar arena secara menyeluruh menggunakan aplikasi grafis, atau menggunakan teknik *tiling* yaitu menggambar arena menggunakan potongan-potongan kecil gambar yang disebut sebagai *tile*. Dalam tutorial di buku ini digunakan penggabungan antara kedua teknik tersebut, sebuah gambar arena besar bertipe *art base* disusun atas *tile-tile* kecil melalui metode *tiling*. Proses ini nantinya dijelaskan secara spesifik pada bab 5.

Adapun tujuan dari sebuah desain arena permainan adalah sebagai berikut :

1. Sebagai ilustrasi utama permainan
Tujuan utama dari sebuah peta permainan dalam RPG adalah sebagai ilustrasi dimana kejadian tersebut berada. Dengan menampilkan sebuah tempat tertentu, maka pemain akan menyadari posisi tempat dia berada, waktu kejadian, elemen cerita apa yang sedang dialami.
2. Sebagai bantuan strategis bagi pemain
Dari sebuah peta permainan, karakter pemain dapat merencanakan strategi apa yang akan dilakukan untuk memenangkan permainan. Sebagai contoh, di awal permainan bisa jadi seorang karakter tidak memiliki uang untuk membeli perlengkapan, dan tanpa perlengkapan akan sulit untuk memasuki arena pertempuran. Sehingga strategi umum yang dilakukan pemain tersebut adalah mengeksplorasi arena permainan untuk mendapatkan item-item yang tersebar atau tersembunyi. Pemain juga dapat memutuskan pergi kemana terlebih dahulu untuk menyelesaikan sebuah misi atau *quest* dalam game.
3. Pengatur jalannya cerita
Sebuah arena permainan acapkali menjadi pengatur jalannya cerita. Beberapa arena tidak bisa diakses di awal permainan, dan baru akan terbuka pada saat karakter mencapai jalan cerita tertentu. Tujuan seperti ini hampir ditemui di semua game RPG populer.

2.5 Misi (*Quest*)

Sebuah game selalu memiliki tujuan tertentu, dimana ketika pemain mencapai tujuan maka permainan akan berakhir. Dalam game RPG misi atau *quest* merupakan tujuan dari permainan. Misi dalam game RPG dapat bersifat tunggal (satu misi utama), namun banyak juga game RPG yang memiliki sub misi (misi tambahan). Dalam game Zelda misalnya, misi utama yang diberikan kepada karakter Link adalah menyelamatkan putri Zelda dan kerajaan yang dikuasai oleh musuh, namun seiring jalannya permainan terdapat misi-misi kecil untuk diselesaikan dalam rangka membentuk plot cerita atau meningkatkan atribut pemain.

2.6 Karakter Pendukung / *Non Playable Character (NPC)*

Dalam game RPG karakter NPC selalu dibutuhkan sebagai penyusun cerita melalui dialog antara pemain dengan karakter NPC, membantu karakter dalam urusan logistik, memberi informasi tertentu atau sekadar pemanis tampilan dalam game. Secara spesifik pengembangan NPC akan dibahas pada bab 9.



Gambar 2.2 NPC dalam game RPG

2.7 Mode Pertempuran (*Battle System*)

Salah satu aspek permainan peran yang paling menonjol dan paling menarik dalam game RPG pada umumnya adalah *Battle System* (mode pertempuran). Pada mode pertempuran karakter pemain akan bertemu dengan musuh (*villain*) dan melakukan pertempuran dengan model yang ditetapkan dalam game. Pertempuran dapat berlangsung secara *realtime* atau secara bergantian (*turn based*). Secara spesifik mode pertempuran dan teknik pembuatannya akan dibahas pada bab 10.



Gambar 2.3 *Realtime Battle system* pada game Zelda (Copyright Capcom, 2004) dan *Turn Based Battle System* pada game Golden Sun (Copyright Camelot Software Planning, 2001).

2.7 RPG sebagai Game Edukasi

Role Playing bersifat kooperatif, improvisasi, terstruktur, dan bentuk bebas dari sebuah "cerita interaktif" yang terjadi dalam imajinasi pemainnya. Ketika bermain sebuah game, pemain pada umumnya dalam keadaan rileks dan senang sehingga

mudah menerima konten apapun yang disampaikan melalui game. Sebagai contoh dalam game RPG secara umum terdapat dialog yang membentuk cerita, hal ini secara ilmiah terbukti mampu mendorong dan meningkatkan keterampilan membaca pemainnya. Selain itu dalam bermain RPG sering kali pemain mempelajari bidang pengetahuan tertentu seperti sejarah, bahasa, politik, demografi, pengerjaan, geografi, sains, sastra, dan beberapa bidang lain. Pada game RPG populer Assassin Creed series (Copyright Ubisoft, 2007 – 2019) misalnya, pemain dibawa ke masa renaissance atau masa pemerintahan firaun di Mesir dan sejarah lainnya, dimana pemain bisa mempelajari sejarah secara tersirat, pemain dapat memahami beberapa bidang pelajaran seperti geografi, astronomi dan navigasi kelautan saat bermain game.

Game RPG Simulasi seperti Astro Farmer (Copyright Froyogames, 2013) dan Harvest Moon (Copyright Amccus, Natsume, 1996) mengajarkan pemain tentang pertanian organik, teknik menanam, cuaca dan pengaruhnya terhadap tanaman, serta teknik peternakan. Materi edukasi tersebut dijadikan sebagai bagian dari *gameplay*, sehingga pemain tidak merasakan dirinya sedang belajar secara konvensional. Pemain akan merasakan serunya bermain, tanpa sadar mereka mempelajari banyak hal baru yang bersifat konseptual maupun praktis.



Gambar 2.4 Game RPG simulasi pertanian Astro Farmer (Copyright Froyogames, 2013) dan Harvest Moon (Copyright Amccus, Natsume, 1996)

Beberapa game RPG yang secara khusus dirancang sebagai game edukasi pada satu dekade terakhir menjadi lebih banyak lagi. Berbagai *platform* permainan seperti PC (komputer), konsol (PS, Nintendo DS), gawai (*Android, iOS*) sampai dengan media baru seperti *Virtual Reality* dan *Mixed Reality* telah dimanfaatkan sebagai media untuk mengembangkan game edukasi bertipe RPG. Berbagai genre RPG juga tersedia untuk berbagai tingkatan usia seperti genre simulasi bisnis, genre politik, simulasi latihan (*training simulation*), genre fantasi, lingkungan dan sosial. Salah satu contoh list game RPG edukasi dapat anda temukan pada link berikut :

<https://www.onlinecolleges.net/50-great-sites-for-serious-educational-games/>

BAB III

ADOBE ANIMATE

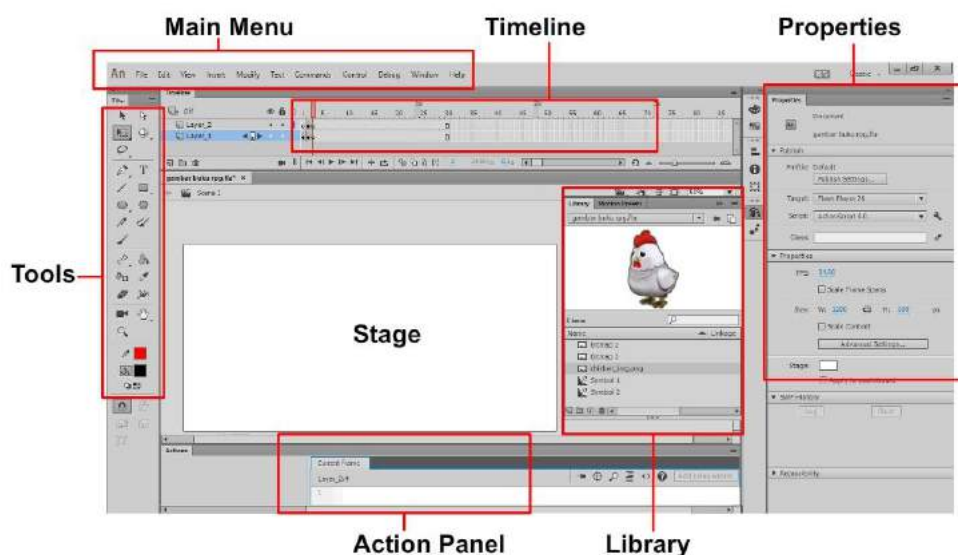
Pengenalan dasar aplikasi Adobe Animate / Flash yang meliputi ruang kerja dan dasar-dasar pemrograman *Actionscript 3.0*

ADOBE ANIMATE / FLASH

3.1 Aplikasi Adobe Animate / Flash

Adobe Animate merupakan program animasi multimedia yang dikembangkan oleh Adobe Systems. Adobe Animate pada dasarnya merupakan pengembangan dari aplikasi Adobe Flash Professional, Macromedia Flash, dan FutureSplash Animator. Adobe Animate memiliki fungsi yang sama dengan Adobe Flash dengan beberapa penambahan fitur. Aplikasi ini dapat digunakan untuk merancang grafik dan animasi vektor, dan mempublikasikannya untuk animasi, video online, situs web, aplikasi web, aplikasi internet, dan *videogames* dengan luaran file bertipe SWF, EXE, APK (Android) dan iOS. Adobe Animate pada dasarnya sama dengan Adobe Flash atau Macromedia Flash, sehingga sebagian besar tutorial yang ada di dalam buku ini dapat dilakukan dengan menggunakan beberapa aplikasi tersebut.

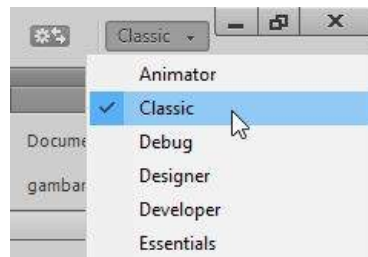
Untuk mengenal Adobe Animate lebih dekat, perlu diketahui ruang kerja dan perangkat dasar yang sering digunakan dalam mengembangkan aplikasi / game. Tutorial di dalam buku ini dibuat dengan menggunakan aplikasi Adobe Animate CC, namun dapat dibuat juga menggunakan aplikasi yang lebih baru maupun aplikasi yang lebih lama seperti Adobe Flash CS. Selanjutnya untuk mempermudah proses membaca, pada buku ini digunakan ruang kerja mode terang (*light mode*).



Gambar 3.1 Ruang Kerja Adobe Animate *Classic Mode*

Secara *default*, ruang kerja Adobe Animate menggunakan mode *essentials*. Pada buku ini digunakan mode *classic* untuk mempermudah penjelasan, terutama bagi yang terbiasa dengan lingkungan kerja Adobe Flash. Untuk merubah ruang kerja ke mode *classic*, tekan opsi

pada kanan atas aplikasi, kemudian pilih mode *classic* dan akan didapati tampilan seperti pada gambar di atas.

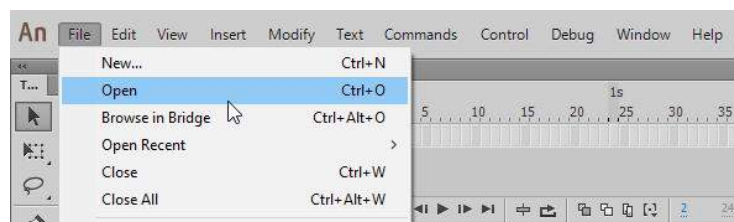


Gambar 3.2 Mengganti mode ruang kerja / antar muka Adobe Animate

Dalam mode **Classic** lingkungan kerja yang ada sebagian besar mirip dengan aplikasi pendahulunya yaitu Macromedia Flash dan Adobe Flash. Lingkungan kerja secara utama dapat dijelaskan sebagai berikut :

a. Main menu

Main menu dalam Adobe Animate memiliki fungsi utama seperti pengaturan file, pengaturan *symbol*, dan fitur untuk menjalankan aplikasi. Dalam buku ini fitur yang paling sering digunakan melalui main menu adalah menu **File**. Menu File digunakan untuk membuat file baru, menyimpan file dan mengimport objek yang berasal dari luar aplikasi seperti gambar dan suara..



Gambar 3.3 Main menu dengan opsi File aktif

b. Timeline

Timeline berisi sebuah kotak-kotak disebut dengan *frame* yang masing-masing kotak mewakili sebuah waktu tertentu. Adobe Animate pada dasarnya adalah sebuah aplikasi untuk membuat animasi, sehingga istilah *frame* bisa diibaratkan sebagai lembaran-lembaran kertas yang menyusun sebuah animasi. Dalam *Timeline* terdapat beberapa istilah terkait dengan animasi sebagai berikut :

1. Keyframe

Keyframe atau frame kunci adalah sebuah frame yang di dalamnya terdapat sebuah objek dan menjadi awal atau akhir sebuah transisi (digambarkan dalam bentuk titik hitam). Dalam Adobe Animate sebuah objek hanya dapat diletakkan pada *timeline* yang memiliki *Keyframe*.

Untuk membuat *Keyframe* pada *timeline* cukup dengan menyeleksi frame kemudian menekan tombol F6 atau klik kanan *frame* dan memilih opsi *Insert KeyFrame*.

2. Blank Keyframe

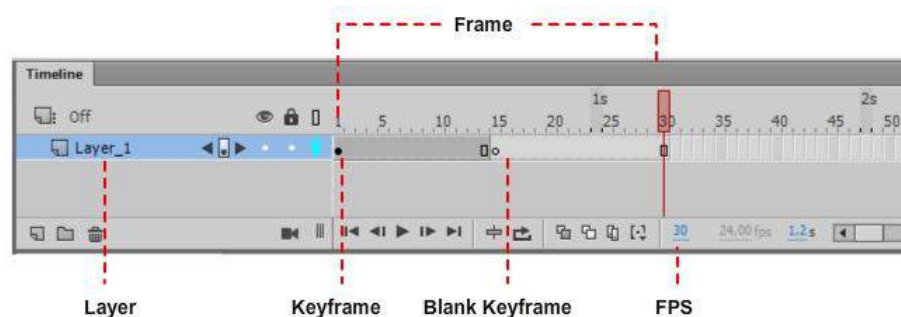
Blank Keyframe adalah *keyframe* yang di dalamnya tidak ada objek sama sekali (digambarkan dalam bentuk titik kosong). Dalam beberapa kasus *Blank Keyframe* dibutuhkan seperti untuk membuat jeda waktu dalam animasi tanpa adanya objek dan untuk mempermudah peletakan kode (*action*).

Untuk membuat *Blank Keyframe* pada *timeline* cukup dengan menyeleksi frame kemudian menekan tombol F7 atau klik kanan *frame* dan memilih opsi *Insert Blank KeyFrame*.

3. Frame

Frame adalah durasi waktu yang ingin didapatkan dalam animasi, sebagai contoh jika ingin mendapatkan animasi berdurasi 5 detik dengan kecepatan 30 FPS (*Frame Per Second*), maka dibutuhkan frame sepanjang 150 *frame*.

Untuk membuat *frame* pada *timeline* cukup dengan menyeleksi *frame* kemudian menekan tombol F5 atau klik kanan *frame* dan memilih opsi *Insert Frame*.



Gambar 3.4 Timeline pada Adobe Animate/Flash

c. Tools

Tools berisi peralatan standar untuk membuat dan mengelola objek. Secara *default* dari kiri ke kanan, tool dalam Adobe Animate dapat dijelaskan sebagai berikut :



Gambar 3.5 Tools pada Adobe Animate

1. Selection Tool

Digunakan untuk menyeleksi objek yang berada di *Stage*

2. *Subselection Tool*

Digunakan untuk menyeleksi *point* pembentuk kurva. Sebagai contoh apabila kita membuat sebuah kotak, maka dengan *Subselection Tool* kita dapat mengubah posisi masing-masing ujung kotak setelah mengaktifkan *Subselection Tool*.

3. *Free Transform Tool*

Digunakan untuk mengubah ukuran dan rotasi sebuah objek di *Stage*.

4. *3D Rotation Tool*

Digunakan untuk merotasi sebuah *Symbol* secara 3 dimensional (sumbu XYZ). Perlu diperhatikan bahwa fitur rotasi 3D hanya dapat digunakan pada objek yang telah diubah menjadi *Symbol*.

5. *Lasso Tool*

Digunakan untuk menyeleksi objek dalam mode laso (drag).

6. *Pen Tool*

Digunakan untuk menggambar garis dengan mode *Pen* (klik dan drag)

7. *Text Tool*

Digunakan untuk menambahkan teks ke *Stage*. Teks dalam Adobe Animate nantinya akan dibedakan menjadi 3 jenis yaitu *Static Text*, *Dynamic Text* dan *Input Text* yang penggunaannya sangat spesifik. Dalam buku ini akan digunakan sebuah *Textfield* yang bersifat dinamis dan digenerasi oleh kode.

8. *Line Tool*

Digunakan untuk membuat garis.

9. *Rectangle Tool*

Digunakan untuk membuat kotak.

10. *Oval Tool*

Digunakan untuk membuat lingkaran atau oval.

11. *Polystar Tool*

Digunakan untuk membuat poligon atau poligon mode bintang.

12. *Pencil Tool*

Digunakan untuk membuat garis dengan mode pensil (drag). Perbedaan utama dengan *Pen Tool* terletak pada teknik mengontrol titik *bezier*, dimana *Pen Tool* memiliki pengaturan titik *bezier* dengan metode klik dan drag, sedangkan *Pencil Tool* hanya membentuk garis berdasarkan gerakan kursor mouse.

13. *Paint Brush Tool*

Digunakan untuk membuat sebuah *stroke* (garis seperti halnya *Pen Tool* dan *Pencil Tool*), hanya saja *Paint Brush Tool* memiliki fitur yang lebih banyak diantaranya fitur penekanan (*pressure*) dan fitur bentuk *brush*.

14. *Brush Tool*

Mode sederhana dari *paint brush tool* dimana hanya menghasilkan *fill* standar.

15. *Bone Tool*

Digunakan untuk menambahkan sistem kinetik *armature* (pertulangan) dalam sebuah animasi.

16. *Paint Bucket Tool*

Digunakan untuk menambahkan warna isi ke dalam sebuah kurva “tertutup”.

17. *Ink Bottle Tool*

Digunakan untuk menambahkan warna *stroke* (garis tepi) ke dalam sebuah *fill*.

18. *Eyedropper Tool*

Digunakan untuk mengambil sampel warna yang ada di layar.

19. *Eraser Tool*

Digunakan untuk menghapus objek gambar yang belum diubah menjadi *symbol*.

20. *Width Tool*

Digunakan untuk memanipulasi garis tepi (*Stroke*). Dekatkan pada *stroke*, klik dan drag untuk mengubah bentuk *stroke*.

21. *Camera Tool*

Digunakan untuk membuat *layer* kamera dan mengaktifkan mode kamera dalam animasi.

22. *Hand Tool*

Digunakan untuk mengeser *Stage* untuk mempermudah melihat bagian tertentu pada *Stage*.

23. *Zoom Tool*

Digunakan untuk memperbesar dan memperkecil tampilan *Stage*.

d. *Library*

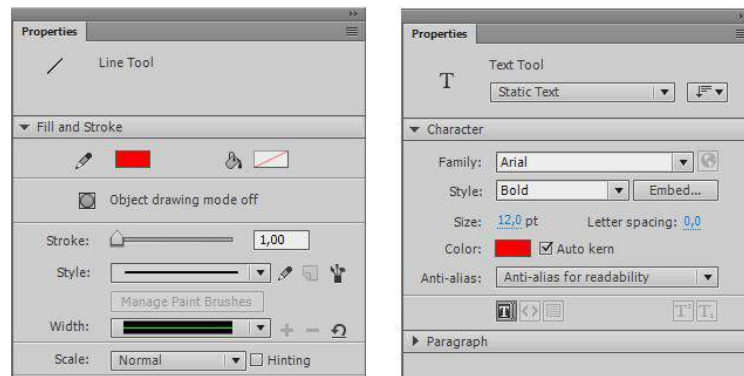
Library adalah sebuah fitur yang menyimpan semua *symbol* yang telah dibuat. *Library* ditampilkan dalam panel yang dapat diakses dengan menekan menu *Window > Library* atau menekan *Ctrl+L*. Untuk menggunakan objek yang berada di *Library* dapat dilakukan dengan melakukan *drag and drop* dari *Library* ke *Stage*.



Gambar 3.6 *Library*

e. *Properties*

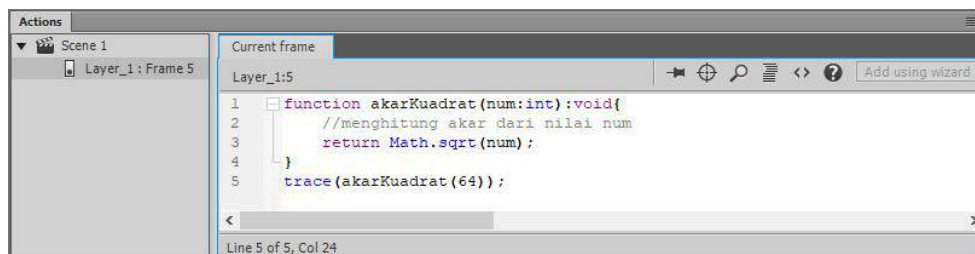
Merupakan sebuah panel dinamis yang berfungsi untuk menampilkan status, informasi atau properti dari suatu objek atau tool yang sedang aktif. Sebagai contoh ketika kita menekan *Line Tool*, maka *Properties* akan menampilkan opsi pengaturan garis seperti warna, ketebalan dan tipe garis. Sedangkan ketika kita menekan *Text Tool* maka *Properties* akan berubah dengan menampilkan opsi atau pengaturan teks mulai dari jenis teks, warna, jenis huruf dan sebagainya. Panel tersebut akan berubah lagi ketika kita menekan menu lain.



Gambar 3.7 Panel *properties*

f. *Action Panel*

Merupakan sebuah panel yang digunakan untuk meletakkan kode. Kode dalam Adobe Animate digunakan untuk mengatur interaktifitas objek, kode tersebut perlu dituliskan menggunakan aturan tertentu pada action panel. Untuk mengakses *action* panel dapat menekan menu *Window > Actions* atau dengan menekan tombol *F9*. Berbeda dengan *Actionscript* versi 1 dan 2 dimana kode dapat diletakkan pada *symbol* maupun pada *timeline*, pada versi 3 *actionscript* hanya dapat dituliskan pada *frame* melalui panel *action*.



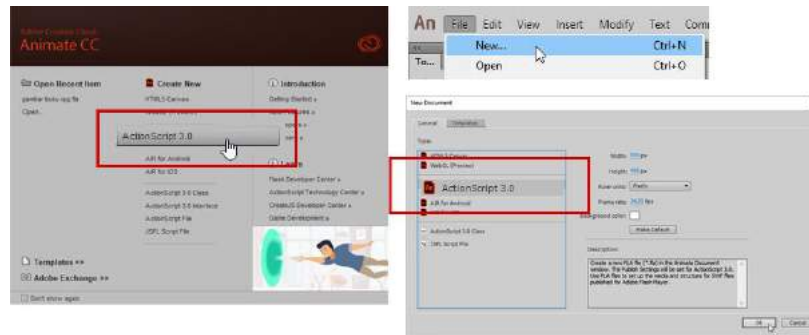
Gambar 3.8 Panel *action*

Tutorial 1

3.2 Menuliskan Kode *Actionscript*

Untuk mengenal pemrograman menggunakan *actionscript*, kita dapat memulai dengan membuat sebuah file baru dan menuliskan sebaris kode. Perhatikan langkah berikut :

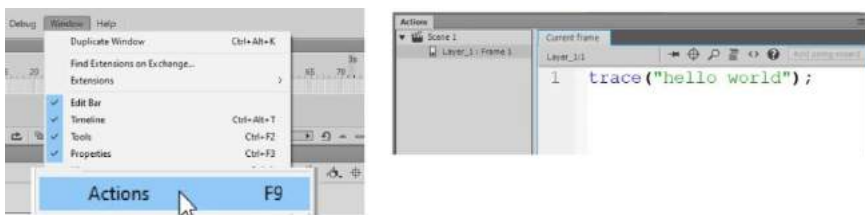
1. Buka aplikasi Adobe Animate/Flash. Kemudian buatlah sebuah file baru dengan memilih opsi **Actionscript 3.0** pada menu *startup*, atau memilih menu **File > New** dan memilih opsi **Actionscript 3.0** lalu tekan **OK**.



Gambar 3.9 Membuat File baru bertipe *Actionscript 3*

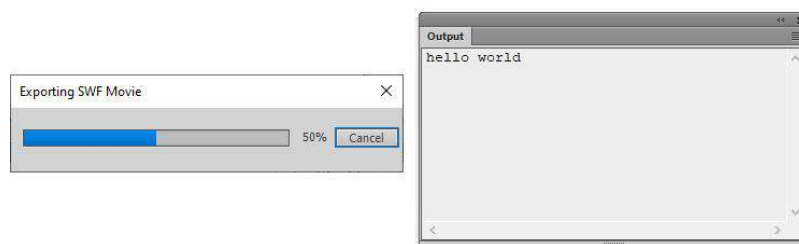
2. Klik **frame 1**. Buka panel **Action** dengan menekan menu **Window > Action** atau menekan tombol **F9**. Kemudian panel action akan muncul, lalu ketikkan satu baris kode berikut :

```
1. trace("hello world");
```



Gambar 3.10 Menuliskan kode di panel *action*

3. Jalankan aplikasi dengan menekan tombol **Ctrl+Enter**. Maka pada **output panel** akan muncul kata "*hello world*".



Gambar 3.11 Kata *Hello World* pada panel *output*

Flash Tips

Dalam dunia pemrograman, seringkali proses pembelajaran diawali dengan pengenalan kode “*hello world*”. Hal ini bertujuan untuk memastikan bahwa aplikasi yang paling sederhana dapat berjalan dengan sukses.

Kode *trace* pada aplikasi di atas tidak akan menghasilkan luaran apapun, hanya menampilkan tulisan “*hello world*” di panel output. Apabila pada tahapan ini anda sukses, maka anda pada langkah yang tepat dalam belajar pemrograman. Namun jika anda masih kesulitan dan belum menemukan apa maksud dari kode ini, berarti anda harus mengulangi langkah-langkah yang ada dan memastikan kata “*hello world*” muncul.

3.3 Dasar Pemrograman *Actionscript 3*

Dalam pembuatan aplikasi atau game terdapat proses dasar yaitu penulisan kode. Kode dalam hal ini, menggunakan bahasa pemrograman tertentu yang akan dijalankan secara prosedural sehingga mampu menghasilkan interaktifitas sebuah aplikasi atau game. Dalam buku ini, bahasa pemrograman yang digunakan adalah **ActionScript**. *Actionscript* pertamakali dikenalkan pada Macromedia Flash 5 dan selanjutnya mengalami perkembangan yang sangat signifikan sampai dengan saat ini. Pada saat ini *actionscript* berada pada versi 3.0 dan memiliki kapabilitas dalam menghasilkan aplikasi atau game untuk *multiplatform* baik itu untuk PC maupun untuk *mobilephone*

Dalam mempelajari *actionscript*, diperlukan pengetahuan dasar dalam menuliskan kode *Actionscript*, dimana terdapat beberapa istilah yang perlu diketahui sekaligus dipatuhi yaitu :

a. Syntax

Dalam pemrograman komputer sintaks berarti seperangkat aturan yang harus dipenuhi dalam penulisan kode. Sintaks meliputi simbol, kata, dan fungsi yang dapat digunakan serta bagaimana menggunakannya dalam sebuah struktur kode. Kesalahan umum yang sering dilakukan oleh programmer adalah kesalahan sintaks (*Syntax error*), dimana kesalahan sintaks akan menyebabkan kode tidak bisa dijalankan oleh aplikasi. Berikut adalah aturan dalam penulisan *Actionscript 3* (AS3).

1. Case Sensitive

Penulisan kode AS3 menggunakan format sensitif case, sehingga setiap kode harus ditulis tepat sebagaimana mestinya. Satu perbedaan karakter akan

menghasilkan perbedaan, sebagai contoh 2 baris kode berikut berbeda karena menggunakan karakter X besar dan x kecil.

```
1. var namaVariabel1:int;  
2. var namaVariabel2:int;
```

Untuk mempermudah penulisan kode maka digunakan metode *camelCase* yaitu menulis dua kata atau lebih secara sambung menyambung, namun digunakan awalan kata huruf besar.

2. Semicolons

Semicolons (;) atau titik koma dalam AS3 digunakan sebagai akhir dari sebuah baris kode. Meskipun dituliskan dalam satu baris, jika ditambahkan tanda titik koma dapat diartikan sebagai baris perintah baru.

```
1. kodeBaris1;  
2. kodeBaris2;kodeBaris3;
```

3. Parentheses

Parentheses atau tanda kurung () digunakan untuk mengubah urutan dalam sebuah operasi/fungsi. Operasi yang diberikan tanda kurung akan dijalankan terlebih dahulu oleh program. Perhatikan contoh berikut :

```
1. var jarak:int = Math.sqrt((titikA.x - titikB.x)*( titikA.x  
- titikB.x)+(titikA.y - titikB.y)*( titikA.y - titikB.y));
```

Pada baris tersebut variabel *jarak* merupakan perhitungan *Phytagoras* yang secara umum digunakan untuk menghitung jarak antara *titikA* dan *titikB*. Keberadaan *parentheses* akan memulai operasi $(titikA.x - titikB.x)$ terlebih dahulu, dilanjutkan dengan perkalian, dan pada akhirnya kode *Math.sqrt* akan menghitung akar dari kuadrat jarak.

Dalam pemrograman komputer, sistem perhitungan matematika akan dilakukan sesuai dengan peraturan yang ada seperti lebih perkalian didahulukan jika dibandingkan dengan penjumlahan atau pengurangan, dan operasi yang berada pada tanda kurung, akan dikerjakan terlebih dahulu. Untuk lebih jelasnya perhatikan contoh sederhana berikut :

```
1. trace(1 + 2 * 3); // akan menghasilkan nilai 7  
2. trace((1 + 2) * 3); // akan menghasilkan nilai 9
```

Parentheses juga dapat digunakan untuk memberikan masukan pada sebuah fungsi. Perhatikan contoh berikut :

```
1. karakter.addEventListener(Event.ENTER_FRAME, gerak);
```

Pada baris tersebut objek *karakter* diberikan perintah (*listener*) untuk menjalankan fungsi *gerak*.

4. *Code blocks*

Dalam pemrograman AS3, satu baris atau lebih yang terletak di dalam kurung kurawa { } disebut sebagai satu blok kode. Satu blok kode merupakan sebuah paket yang biasanya digunakan dalam sebuah fungsi, *class*, *loop* dan sebagainya. Perhatikan baris berikut :

```
1. function akarKuadrat(num:int):void{  
2.     //menghitung akar dari nilai num  
3.     return Math.sqrt(num);  
4. }
```

Dalam baris kode di atas fungsi *akarKuadrat* memiliki satu kode blok yaitu kode di baris 2-3.

5. *Whitespace*

Whitespace merupakan istilah untuk jarak penulisan, tab, enter dan spasi yang ditujukan untuk mempermudah penulisan kode. Perhatikan kode di atas, pada baris 2 dan 3 ditulis menjorok masuk, hal tersebut diistilahkan sebagai *whitespace* (ruang kosong). Pada kasus yang lain, programmer akan memberikan jarak penulisan (satu kali enter) antar blok kode. *Whitespace* diabaikan oleh aplikasi saat mengkompilasi kode, namun sangat bermanfaat bagi programmer untuk membantu mengecek struktur kode.

6. *Comments*

Comments atau komentar merupakan catatan yang bisa ditambahkan pada kode untuk mempermudah penjelasan kode. Ketika menulis kode dalam jumlah banyak (misal 2000 baris kode) dibutuhkan penjelasan-penjelasan pada baris kode untuk mempermudah proses pengecekan. Program secara otomatis akan mengabaikan baris komentar, sehingga kita tidak perlu khawatir dalam menggunakan komentar. Perhatikan baris 2 pada kode berikut :

```

1. function akarKuadrat(num:int):void{
2.     //menghitung akar dari nilai num
3.     return Math.sqrt(num);
4. }

```

Kata apapun yang berada di belakang karakter `//` akan dianggap sebagai komentar, dan tidak akan dijalankan oleh program. Apabila ingin menuliskan komentar lebih dari satu baris dapat menggunakan karakter `/* komentar */`

Penggunaan tanda `/* .... */` pada tahapan selanjutnya akan sangat membantu programmer untuk melakukan pengecekan kode. Beberapa baris kode yang diawali dengan karakter `/*` dan diakhiri dengan karakter `*/` tidak akan dijalankan oleh aplikasi, sehingga sangat berguna untuk mengetahui blok kode mana yang mengalami kesalahan.

b. Variable

Sistem kerja komputer adalah dengan menyimpan data dan mengolahnya. Dalam mengelola data komputer akan menyimpan sebuah nilai ke dalam memori, dimana seorang programmer dapat mengatur nilai tersebut dan memberikannya nama. Nama dan nilai yang akan disimpan ke dalam memori itulah yang disebut sebagai **variabel**. Sebagai contoh, dibuat sebuah variabel untuk menyimpan nilai, yaitu **var score:int;** ini berarti komputer akan menyimpan sebuah data bernama **score** bertipe **Integer** (angka). Variabel ini akan terus disimpan oleh memori komputer sepanjang aplikasi berjalan.

```

1. var score:int = 0;

```

c. Data Types

Data types atau jenis data adalah beberapa klasifikasi data yang digunakan dalam pemrograman AS3. Secara mendasar jenis data di AS3 ada 7 yaitu *String*, *Number*, *int*, *uint*, *Boolean*, *Null*, dan *void*.

1. String

String adalah data yang bernilai tekstual dan penulisannya dalam AS3 menggunakan tanda petik, misalnya

```
var nama:String = "Izzan";
```

2. Number

Number adalah data yang bernilai angka tanpa batasan. Misal :

```
var nilaiUjian:Number = 97;
```

3. *Int*

Int adalah kependekan dari integer yaitu data yang bernilai angka dari -2,147,483,648 sampai dengan 2,147,483,647 (32 *bit* data). Penggunaan variable bertipe *int* lebih efisien dari segi kecepatan pengolahan data dibanding dengan variabel bernilai *Number*.

4. *Uint*

Berbeda dengan *int* yang bernilai negatif sampai positif, *uint* hanya bernilai positif yaitu mulai dari angka 0 sampai dengan 4,294,967,295.

5. *Boolean*

Boolean merupakan nilai benar atau salah. Dalam AS 3 bernilai *true* atau *false*.

6. *Null*

Null merupakan nilai kosong atau tidak bernilai, namun bukan tidak terdefiniskan (*undefined*).

7. *Void*

Void adalah sesuatu yang tidak menghasilkan nilai, pada umumnya digunakan untuk fungsi yang tidak mengembalikan/menghasilkan nilai spesifik.

Pada pengembangan berikutnya akan ditemukan tipe data yang lebih kompleks seperti *Array*, *Object*, *Date* dan sebagainya.

d. Operators

Operators merupakan simbol yang digunakan untuk operasi matematika seperti penjumlahan (+), pengurangan (-), perkalian (*), pembagian (/), sama dengan (=) dan sebagainya. Pada contoh kode berikut digunakan operator penjumlahan.

```
1. score += 10
2. score = score + 10;
```

kedua baris kode di atas memiliki fungsi yang sama, yaitu nilai variabel *score* ditambah 10. Penulisan pada baris 1 lebih lazim dilakukan oleh programmer, karena lebih menghemat karakter.

e. Function

Function atau fungsi merupakan satu atau beberapa baris kode dalam satu blok, yang ditujukan untuk menyederhanakan suatu operasi dan dapat digunakan beberapa kali. Sebagai contoh, ketika membuat aplikasi yang melibatkan perhitungan jarak antara objek dan mouse, maka fungsi menghitung jarak tersebut ditulis dalam sebuah blok kode yang berisi beberapa baris. Untuk menghitung jarak tentunya ada beberapa

variabel yang diperlukan yaitu titik kordinat awal dan titik kordinat akhir, selanjutnya dilakukan perhitungan menggunakan rumus pitagoras dan terakhir hasil perhitungan ditampilkan oleh fungsi. Perhatikan contoh berikut :

```
1. function akarKuadrat(num:int):void{
2.     //menghitung akar dari nilai num
3.     return Math.sqrt(num);
4. }
```

Untuk mengaktifkan fungsi tersebut cukup memanggil nama fungsi diikuti dengan nilai parameter yang ada.

```
5. var jarak:int = akarKuadrat(64); //akan menghasilkan nilai 8
```

f. Conditional if

Kondisi if merupakan logika dasar sebagian besar bahasa pemrograman. Dengan kode *if* sebuah kondisi dicek kebenarannya dan blok kode akan dieksekusi berdasarkan kondisi tersebut. Perhatikan baris berikut :

```
1. if (jarak < 10){ //jika jarak kurang dari 10 pixel
2.     dapatKoin();
3. }else{
4.     //jika jarak lebih dari 10 pixel gerakan karakter
5.     gerakKarakter();
6. }
```

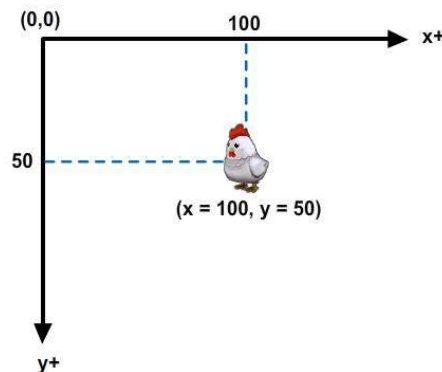
Selain condisional *if*, juga terdapat beberapa logika dasar yang digunakan dalam pemrograman seperti *while*, dan *do..case*.

3.4 Sistem Kordinat Layar

Untuk memahami pembuatan game, maka diperlukan pemahaman dasar tentang bagaimana sebuah kode bekerja dalam mengendalikan objek-objek yang ada di layar. Oleh karena itu, diperlukan pemahaman tentang sistem kordinat layar. Sistem kordinat layar pada *actionsript* 3.0 memiliki sedikit perbedaan dengan sistem kordinat kartesian, dimana pusat dari sistem kordinat atau titik (0,0), berada di pojok kiri atas layar.

Pada sistem kordinat layar, sumbu horisontal disebut sebagai sumbu x, dengan sumbu x positif ke arah kanan dan sumbu x negatif ke arah kiri. Sumbu vertikal disebut

sebagai sumbu y, dengan sedikit perbedaan dengan sistem kartesian. Sumbu y positif ke arah bawah dan sumbu y negatif ke arah atas. Perhatikan gambar berikut :



Gambar 3.12 Sistem kordinat layar

Pada gambar di atas, terdapat gambar ayam yang diletakkan pada posisi kordinat $x = 100$ pixel dan $y = 50$ pixel, maka dalam penulisan kode *actionsript* dapat ditulis sebagai berikut :

```
1. score += 10
2. score = score + 10;
```

Pada tahapan selanjutnya akan dijelaskan teknik penambahan objek dan pengaturan kordinatnya di layar.

Tutorial 2

3.5 Menambahkan Objek ke Layar dengan Kode

Dalam pengembangan game, terdapat beberapa tahapan dasar yang harus dikuasai. Tahapan yang paling awal adalah menambahkan objek ke layar permainan. Pada umumnya terdapat 2 teknik menambahkan objek ke layar, yaitu :

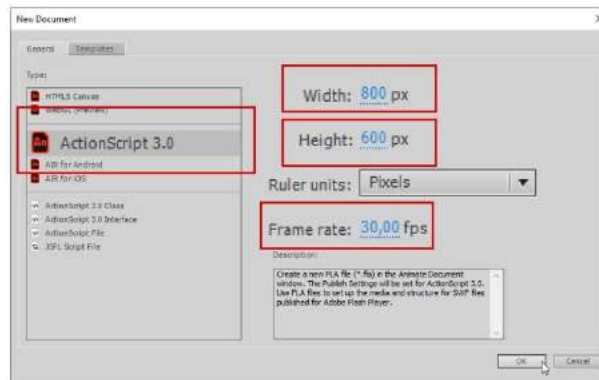
1. Melakukan *drag and drop* sebuah objek dari *Library* ke *Stage*.
2. Menambahkan menggunakan kode.

Pada langkah pertama, setelah objek dibuat (dapat berupa *Movieclip* atau tombol) objek akan masuk ke dalam *Library* yang mana untuk menampilkannya ke layar, objek secara langsung didrag ke *stage*. Metode ini sangat mudah dilakukan bahkan oleh pemula, namun untuk membuat game yang relatif kompleks, metode ini lebih baik dihindari.

Buku ini secara umum akan menggunakan langkah yang kedua, dikarenakan pengaturan objek pada tahapan selanjutnya akan lebih mudah, pengkondisian juga menjadi lebih mudah untuk dilakukan. Pada tutorial berikut, akan dijelaskan secara

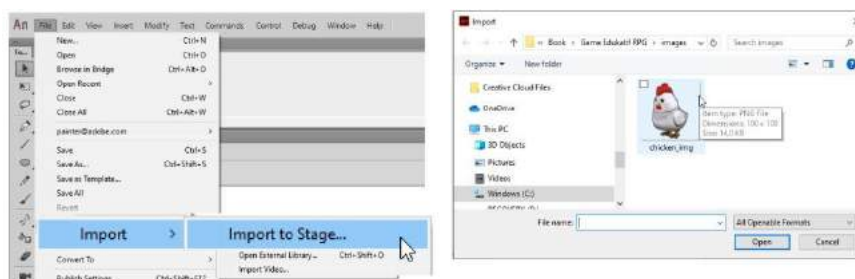
spesifik tentang proses penambahan objek ke *stage*/layar menggunakan kode. Untuk lebih jelasnya ikutilah langkah-langkah berikut :

1. Buatlah sebuah file baru dengan memilih menu **File > New** dan pilih opsi **ActionScript 3.0**.
2. Sebelum menekan tombol **OK**, lakukan pengaturan ukuran aplikasi, yaitu 800 pixel x 600 pixel dan 30 fps. Klik tombol **Make Default** untuk menyimpan pengaturan tersebut, barulah anda tekan tombol **OK**.



Gambar 3.13 Pengaturan awal dokumen aplikasi

3. Import gambar **chicken_img.png** yang terdapat pada folder *images* dengan cara memilih menu **File > Import > Import to Stage**. Buka folder *images* lalu pilih file **chicken_img.png** lalu tekan **OK**. Maka gambar tersebut akan muncul pada *Stage*.



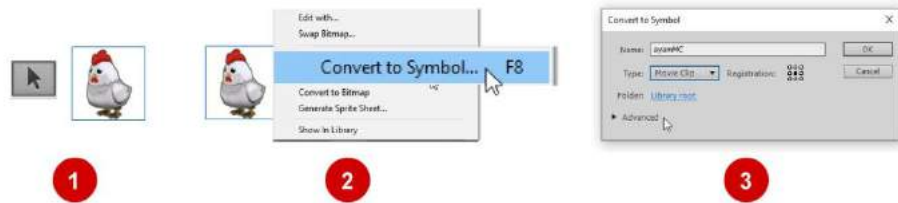
Gambar 3.14 Mengimport gambar ke *Stage*

Flash Tips

Membuat objek pada Adobe Animate, dapat dilakukan secara langsung menggunakan *drawing tool* yang tersedia. Namun, apabila anda terbiasa dengan aplikasi lain, anda dapat membuat gambar dengan menggunakan *software* grafis lain (Photoshop misalnya) dan pastikan menyimpannya dengan format **.PNG** untuk mendapatkan *background* transparan.

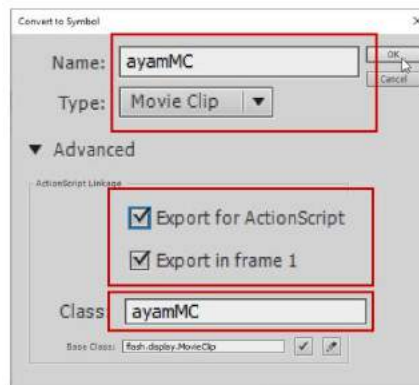
4. Agar objek dikenali oleh Animate, maka objek tersebut harus diubah menjadi **Symbol**. Animate memiliki 3 jenis *symbol*, yaitu *Graphic*, *Movieclip* dan *Button*. Ketiganya memiliki perilaku dan kegunaan yang berbeda. Dalam tutorial ini

dibutuhkan sebuah *symbol* bertipe *Movieclip*. Untuk itu, gambar ayam yang telah diimport pada langkah 3 perlu diubah menjadi *Movieclip* dengan cara menyeleksi gambar (dengan *Selection Tool*), klik kanan gambar, lalu pilih opsi **Convert to Symbol** atau tekan tombol **F8**. Sebelum menekan tombol **OK**, lakukan pengaturan seperti pada langkah 5.



Gambar 3.15 Tahapan membuat *Movieclip*

5. Sebuah *symbol* memerlukan pengaturan untuk mempermudah proses penulisan kode. Maka sebelum menekan tombol OK, *symbol* perlu ditambahkan nama yang detail, dalam hal ini diberikan nama **ayamMC** (akhiran MC digunakan sebagai identifikasi bahwa *symbol* tersebut bertipe *Movieclip*). Selanjutnya pada opsi *type* dipilih **Movieclip**. Titik registrasi di tengah. Kemudian klik teks **Advanced** untuk mengekspansi panel ke mode *advanced*. Centang opsi **Export for Action Script**, maka secara otomatis opsi **export in frame 1** akan tercentang dan kolom nama **Class** akan terisi sama dengan nama *symbol*. Setelah itu baru tekan **OK**.



Gambar 3.16 Pengaturan *movieclip* ayamMC.



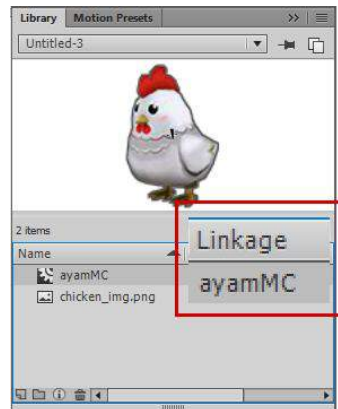
Pada saat anda menekan tombol OK, peringatan akan keberadaan *Class* kemungkinan muncul. Anda dapat mengabaikannya dengan menekan tombol OK.

6. Klik **ayamMC** yang berada di *Stage*, lalu hapus dengan menekan tombol **delete**. Hal ini dilakukan karena penambahan objek ke *stage* akan dilakukan dengan menggunakan kode, sehingga sebisa mungkin *Stage* dalam keadaan bersih (kosong). Anda tidak perlu khawatir *movieclip* ayamMC tersebut hilang, karena setelah sebuah *symbol* dibuat, secara otomatis akan masuk ke dalam **Library**.

Klik menu **Window > Library** atau tekan tombol **Ctrl+L**, maka akan didapati **ayamMC** berada pada panel *Library*.



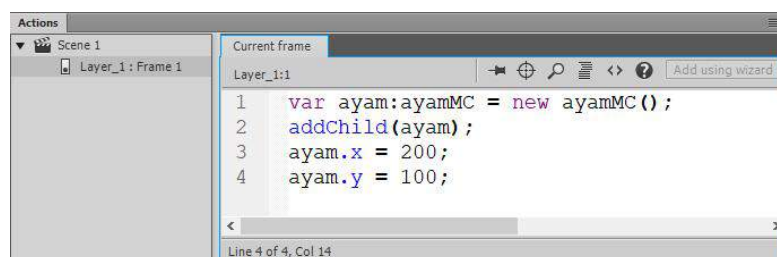
Hal yang perlu diperhatikan adalah, bahwa pada panel tersebut terdapat kata **Linkage** yang merupakan nama **Class** dari *symbol*. Nama **Class** inilah yang nantinya akan kita gunakan dalam kode, bukan nama *symbol*.



Gambar 3.17 Movieclip ayamMC pada panel *Library*

7. Selanjutnya untuk menambahkan kode, klik **frame 1**. Buka panel *Action* dengan menekan menu **Window > action** atau menekan tombol **F9**. Kemudian ketikkan kode berikut :

```
1. var ayam:ayamMC = new ayamMC ();  
2. addChild(ayam);  
3. ayam.x = 200;  
4. ayam.y = 100;
```



Gambar 3.18 Tampilan kode pada panel *action*

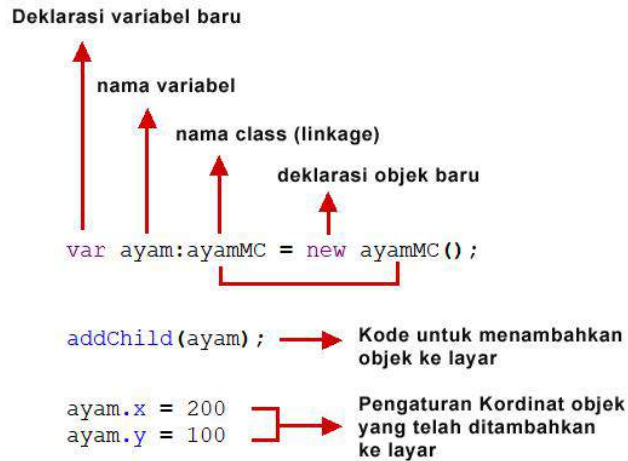
8. Simpan file dengan menekan tombol **Ctrl+S**, kemudian jalankan aplikasi dengan menekan tombol **Ctrl+Enter**. Maka gambar ayam akan muncul di layar aplikasi.



Gambar 3.19 Hasil tutorial

Penjelasan Program

Empat baris kode tersebut di atas, dapat dijelaskan sebagai berikut :



Pada awalnya diperlukan sebuah deklarasi variabel baru, yaitu dengan menggunakan kode **var**. Variabel baru tersebut membutuhkan nama, dimana penamaan variabel tidak boleh sama dengan nama *class*, sehingga digunakan nama **ayam**. Tipe variabel tersebut selanjutnya ditentukan dengan mengacu kepada *class* ayamMC yaitu **linkage** dari movieclip **ayamMC**. Kode **new**, digunakan untuk menyatakan bahwa objek tersebut adalah objek baru yang akan dibuat oleh aplikasi.

Pada baris 2, kode **addChild** berfungsi untuk menambahkan objek ke layar. Objek yang baru saja dibuat yaitu **ayam** selanjutnya ditambahkan dengan memasukkannya ke dalam parameter fungsi **addChild**. Penambahan ini akan meletakkan objek pada kordinat x = 0 dan y = 0;

Pada baris ke-3 dan ke-4, sistem kordinat diatur menjadi x = 200 dan y = 100, sehingga posisi ayam akan bergeser ke kanan 200 *pixel* dan ke bawah 100 *pixel*.

Flash Tips

Pada tahapan ini, anda dapat bereksperimen dengan menambahkan beberapa kode untuk memanipulasi objek, misal :

```
ayam.scaleX = 2 ; // pembesaran ayam kesumbu X sebesar 200%
ayam.rotation = 45; // rotasi ayam
ayam.alpha = 0.25; // transparansi gambar menjadi 25%
ayam.visible = false; // menghilangkan gambar ayam (bukan
menghapus, hanya tidak tampak saja)
```

3.6 Membuat Gerakan Objek

Setelah memahami proses penambahan objek ke layar menggunakan kode **addChild**, maka pengetahuan dasar dalam pembuatan game adalah menggerakkan objek. Gerakan objek di dalam game terdiri dari 2 jenis, yaitu :

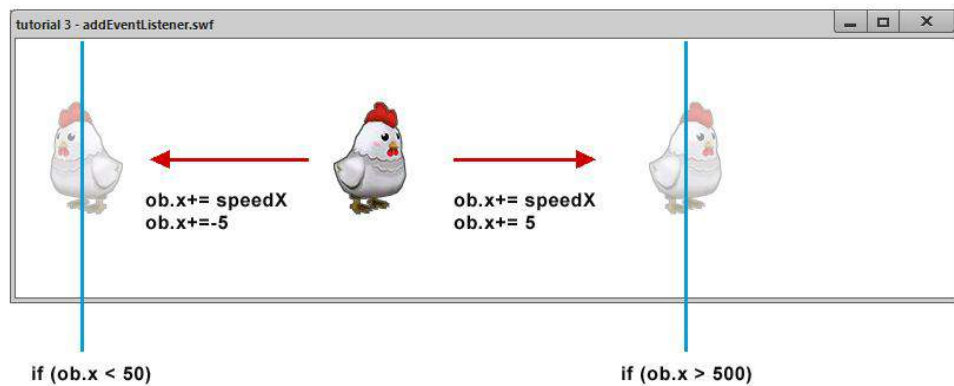
1. Gerakan otomatis, yang dilakukan secara terus menerus atau ketika ada kondisi tertentu.
2. Gerakan yang dipicu oleh masukan (*input*) yang dilakukan oleh pemain seperti adanya tombol keyboard ditekan, atau input dari mouse.

Pada tutorial ini akan dijelaskan cara menambahkan gerakan otomatis yang dilakukan secara terus menerus. Perhatikan langkah berikut:

1. Buatlah sebuah file baru, seperti pada pengaturan **tutorial 2**.
2. Import gambar ayam, kemudian **convert to symbol** menjadi movieclip **ayamMC** dengan opsi **export for actionscript** tercentang (langkah 3 – 5 pada **tutorial 2**).
3. Hapus *movieclip ayamMC* dari Stage.
4. Klik **frame 1**. Buka panel **action** dan ketikkan kode berikut :

```
1. import flash.events.Event;
2.
3. var ayam:ayamMC = new ayamMC();
4. addChild(ayam);
5. ayam.x = 200;
6. ayam.y = 100;
7.
8. var speedX:int = -5;
9.
10. ayam.addEventListener(Event.ENTER_FRAME, gerak);
11.
12. function gerak(e:Event):void{
13.     var ob:Object = e.currentTarget;
14.     ob.x+=speedX;
15.     if (ob.x < 50 || ob.x > 500){
16.         speedX*=-1;
17.         ob.scaleX*=-1;
18.     }
19. }
```

5. Jalankan aplikasi dengan menekan tombol **Ctrl+Enter**. Maka ayam akan bergerak dari kanan ke kiri dan berbalik secara terus menerus.



Gambar 3.20 Hasil tutorial 3

Penjelasan Program

Perhatikan gambar di atas, ketika aplikasi dijalankan ayam akan bergerak secara terus menerus ke kanan dan ke kiri diikuti dengan perubahan bentuk yang mengikuti arah. Hal ini dikarenakan sebuah *event* atau kejadian yang diset untuk berulang secara terus menerus (**ENTER_FRAME**).

```
1. import flash.events.Event;
```

Baris tersebut sebenarnya tidak perlu diketikkan secara manual, karena akan otomatis muncul ketika baris 10 diketikkan. Baris ini berarti aplikasi sedang meng*import* satu paket fungsi yang berada pada kelompok **Event**.

```
3. var ayam:ayamMC = new ayamMC();
4. addChild(ayam);
5. ayam.x = 200;
6. ayam.y = 100;
```

Baris 3-6 merupakan metode untuk menambahkan objek ke layar (seperti pada tutorial 2)

```
8. var speedX:int = -5;
```

Untuk membuat gerakan dinamis ke kanan dan kekiri, diperlukan variabel kecepatan yang dapat berubah jika mengalami kondisi tertentu. Pada awal aplikasi dijalankan, variabel *speedX* bernilai negatif, sehingga ayam akan bergerak ke kiri (*x* negatif).

```
10. ayam.addEventListener(Event.ENTER_FRAME, gerak);
```

Inti dari gerakan objek adalah penambahan **Event**. Dalam hal ini objek **ayam** diberikan **listener** (perintah) untuk melaksanakan fungsi **gerak** secara berulang-ulang

(ENTER_FRAME). Perhatikan kata-kata **gerak**, ini adalah fungsi yang akan dijalankan secara terus menerus oleh objek **ayam**, dan perlu dijabarkan pada baris selanjutnya.

```
12. function gerak(e:Event):void{
13.     var ob:Object = e.currentTarget;
14.     ob.x+=speedX;
15.     if (ob.x < 50 || ob.x > 500){
16.         speedX*=-1;
17.         ob.scaleX*=-1;
18.     }
19. }
```

Fungsi **gerak** selanjutnya dituliskan dengan satu blok kode (baris 13-18). Pada tahapan awal diperlukan sebuah variabel untuk mendefinisikan objek yang diperintah (dideklarasikan dengan nama **ob** / kependekan dari objek).

Selanjutnya posisi horisontal (sumbu **x**) dari **ob** akan ditambahkan dengan nilai variabel **speedX**, yang mana pertamakali aplikasi dijalankan bernilai **-5** sehingga **ob** akan bergerak ke arah kiri.

Condisional **if** pada baris 15, menyatakan bahwa **jika** kordinat **x** dari **ob** kurang dari 50 **atau** (disimbolkan dengan karakter **||**) lebih dari 500, maka blok kode baris 16 -17 akan dijalankan.

Perhatikan gambar 3.20. Ayam bergerak ke kiri, dan suatu ketika kordinat **x** dari ayam kurang dari 50, maka blok kode 16-17 akan aktif, sehingga nilai variabel **speedX** akan dikalikan dengan **-1** sehingga berubah menjadi **+5**. Demikian halnya dengan **scaleX** dari ayam dikalikan **-1** sehingga gambar menjadi terbalik. Kondisi ini terjadi satu kali, dan ayam bergerak ke kanan, sampai kondisi **if (ob.x>500)** tercapai.

Fungsi **gerak** akan dilakukan secara terus menerus sepanjang aplikasi berjalan, dan kode blok baris 13-18 akan berjalan, sehingga ayam akan bergerak ke kiri dan ke kanan, sesuai dengan kecepatan **speedX** dan arah gerakan **scaleX**.

3.7 Membuat Gerakan Objek dengan Input *Keyboard*

Membaca input dari pemain merupakan syarat umum dalam sebuah game. Input yang berasal dari *keyboard*, *mouse*, *touch*, dan *gesture* menjadi poin utama dalam membangun interaktifitas antara pemain dengan game.

Pada tutorial ini dijelaskan tentang gerakan objek berdasarkan input dari tombol *Keyboard*. Perhatikan langkah-langkah berikut :

1. Buatlah sebuah file baru, seperti pada pengaturan **tutorial 2**.
2. Import gambar ayam, kemudian **convert to symbol** menjadi movieclip **ayamMC** dengan opsi **export for actionscript** tercentang (langkah 3 – 5 pada tutorial 2).
3. Hapus movieclip **ayamMC** dari *Stage*.
4. Klik **frame 1**. Buka panel **action** dan ketikan kode berikut :

```
1. import flash.events.KeyboardEvent;
2.
3. var ayam:ayamMC = new ayamMC();
4. addChild(ayam);
5. ayam.x = 200;
6. ayam.y = 200;
7.
8. var speed:int = 10;
9.
10. stage.addEventListener(KeyboardEvent.KEY_DOWN,
    tombol_ditekan);
11.
12. function tombol_ditekan(e:KeyboardEvent):void{
13.     if (e.keyCode == 37){
14.         ayam.x -= speed;
15.         ayam.scaleX = 1;
16.     }
17.     if (e.keyCode == 39){
18.         ayam.x += speed;
19.         ayam.scaleX = -1;
20.     }
21.     if (e.keyCode == 38){
22.         ayam.y -= speed;
23.     }
24.     if (e.keyCode == 40){
25.         ayam.y += speed;
26.     }
27. }
```

5. Jalankan aplikasi dengan menekan tombol **Ctrl+Enter**. Klik layar permainan, kemudian tekan tombol panah, maka ayam akan bergerak sesuai dengan input dari *keyboard*.

Penjelasan Program

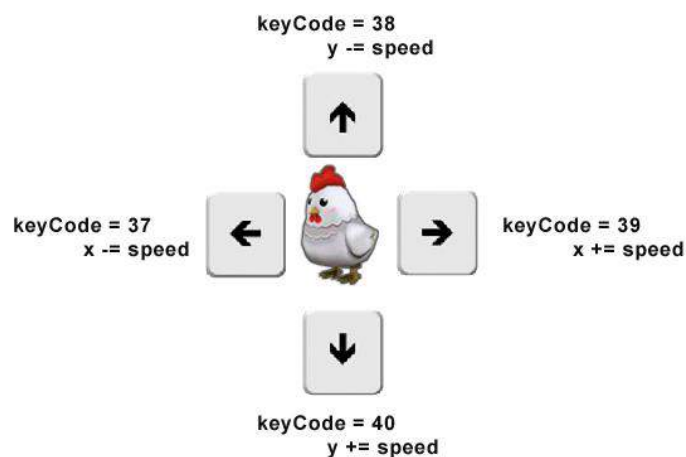
Untuk membaca masukan (input) dari *keyboard*, digunakan kode **KeyboardEvent** yang *listenernya* ditambahkan langsung ke **stage**.

```
10. stage.addEventListener(KeyboardEvent.KEY_DOWN, tombol_ditekan);
```

Pada baris tersebut, **stage** akan membaca ketika terdapat *event keyboard* ditekan (**KEY_DOWN**), yang mana jika hal tersebut terjadi maka fungsi **tombol_ditekan** akan dijalankan.

```
12. function tombol_ditekan(e:KeyboardEvent):void{  
13.     if (e.keyCode == 37){  
14.         ayam.x -= speed;  
15.         ayam.scaleX = 1;  
16.     }  
17.     ....  
  
27. }
```

Fungsi **tombol_ditekan** akan membaca kode tombol yang sedang aktif ditekan (**e.code**). Kode tombol ini merupakan kode *unicode* standar dari *keyboard*, dimana tombol panah memiliki kode secara berurutan **37**, **38**, **39** dan **40**. Sebagai contoh ketika tombol kiri ditekan (**e.keyCode == 37**), maka kordinat x **ayam** digeser ke arah kiri sebesar nilai variabel **speed**.



Gambar 3.21 Kode *keyboard* dan gerakan ayam

Flash Tips

Untuk mengetahui kode *keyboard* secara lengkap, anda dapat melihat pada gambar 3.22

Esc 27		F1 112				F2 113		F3 114		F4 115		F5 116				F6 117		F7 118		F8 119		F9 120				F10 121		F11 122		F12 123					
~ 192		! 49		@ 50		# 51		\$ 52		% 53		^ 54		& 55		* 56		{ 57		} 48		- 189		= 187		Backspace 8				Home 36		End 35		PgUp 33	
Tab 9		Q 81		W 87		E 69		R 82		T 84		Y 89		U 85		I 73		O 79		P 80		{ 219		} 221		\ / 220				Insert 45		Delete 46		PgDn 34	
Caps Lock 20		A 65		S 83		D 68		F 70		G 71		H 72		J 74		K 75		L 76		; 186		" 222		Enter 13											
Shift 16			Z 90		X 88		C 67		V 86		B 66		N 78		M 77		< 188		> 190		? 191		Shift 16												
Ctrl 17		Win 91		Alt 18		32										Alt 18		Win 92		Menu 93		Ctrl 17													

← 37		↓ 40		→ 39	
---------	--	---------	--	---------	--

Gambar 3.22 Kode *keyboard*

BAB IV

SISTEM *TILING* DALAM GAME RPG

Sistem pengulangan gambar yang bersumber dari satu set gambar *tile* (ubin) yang sering digunakan sebagai *background*, *map* atau arena permainan

SISTEM *TILING* DALAM GAME RPG

4.1 Sistem *Tiling*/Pengulangan

Game bertipe RPG (*Role Play*) memiliki kompleksitas yang lebih dalam jika dibandingkan dengan game jenis lainnya. Salah satu kompleksitas yang hampir selalu muncul adalah penggunaan *background* atau area yang luas sebagai area aktif dari karakter yang dimainkan.

Menggerakkan sebuah gambar yang besar / luas sebagai *background* pada proses pengembangan game di era sekarang bisa jadi mudah untuk dilakukan dan tidak membutuhkan konsep yang rumit. Namun perlu diingat bahwa menggerakkan sebuah gambar membutuhkan *memory* yang cukup besar. Hal inilah yang sulit dilakukan pada masa-masa awal berkembangnya game (periode tahun 1980 – 2000).

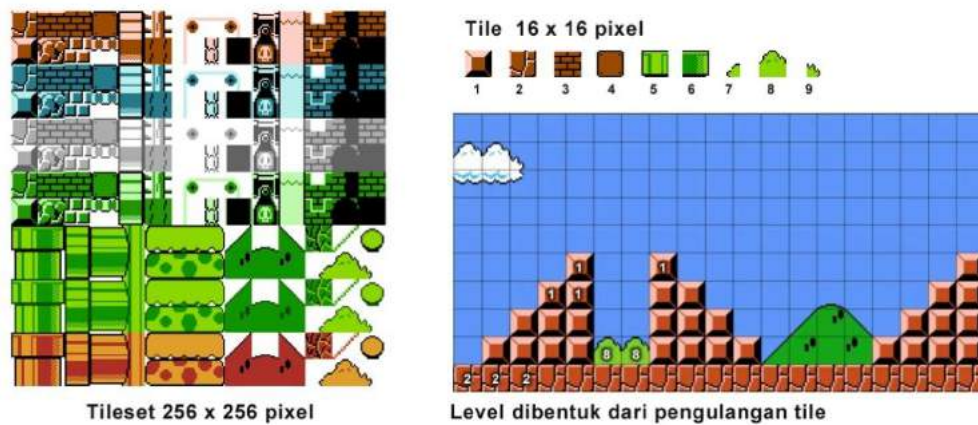
Jika anda pernah memainkan game Super Mario Bros (*Copyright* Nintendo, 1985), game klasik pada *platform* Nintendo, anda mungkin tidak mengira bahwa ukuran file game tersebut kurang dari 40 KB, sementara apabila kita memainkannya di emulator lalu kita mengambil *screenshot* dari permainan tersebut memiliki ukuran lebih dari 200 KB. Bagaimana hal tersebut bisa dilakukan?



Gambar 4.1 Game Super Mario Bros (*Copyright* Nintendo, 1985)

Game Super Mario di atas memiliki 24 level yang berbeda dengan masing-masing level memiliki grafik yang berbeda-beda, memiliki karakter yang beragam dan suara yang ikonik. Kelebihan tersebut berkebalikan dengan keterbatasan teknologi yang bisa digunakan saat itu. Sehingga pengembang (*developer*) membutuhkan suatu teknik tertentu untuk mengoptimalkan performa permainan. Salah satu cara yang dilakukan adalah dengan menggunakan sistem *tiling* (pengulangan/pengubinan).

Sistem *tiling* (pengulangan/pengubinan), merupakan sebuah metode untuk mengulang-ulang sebuah gambar berdasarkan data bertipe *array*, dimana dengan data *array* tersebut gambar sumber (disebut sebagai *tileset*) diolah dan diulang sedemikian rupa untuk digambar ulang menjadi satu gambar baru yang lebih besar.



Gambar 4.2 *Tileset* dan sistim *tiling* pada game Super Mario

Tileset atau gambar sumber pada game Super Mario terdiri dari beberapa potongan-potongan ubin (*tile*) kecil berukuran 16×16 *pixel*, yang kemudian disatukan menjadi sebuah *tileset* berukuran 256×256 *pixel*. Ubin-ubin tersebut ditata ulang menjadi sebuah level permainan. Dengan metode ini, ukuran file gambar akan selalu tetap, yaitu sebesar ukuran file *tileset*, sedangkan level dapat dibuat sebanyak mungkin dengan menggunakan editor khusus. Dengan metode ini grafik level akan menjadi sangat beragam tanpa diikuti kenaikan ukuran file yang signifikan, inilah yang disebut sebagai sistem *tiling*.

Pada game bertipe RPG, sistem *tiling* juga seringkali diadaptasi untuk mengelola level atau area permainan. Seiring teknologi yang semakin berkembang, teknik pengulangan grafik untuk keperluan area permainan juga semakin beragam. Pengulangan tidak lagi harus dalam bentuk *flat* (persegi), namun juga dapat dilakukan dengan metode *isometri*, bahkan pengulangan dalam grafik 3 dimensi.



Gambar 4.3 *Tiling* pada Game The Legend of Zelda (Copyright Nintendo, 2004) menggunakan *tiling* standar 2 dimensi



Gambar 4.4 Game Tactics Ogre (Copyright Quest, 1995)
menggunakan *tiling* isometris



Gambar 4.5 Astro Farmer (copyright Froyogames, 2013)
menggunakan grafik 3D dengan *tiling* material

Dalam game RPG modern, teknik *tiling* sudah jarang digunakan mengingat teknologi yang ada memungkinkan grafis yang memukau, ukuran file yang tidak dibatasi dan *memory* grafis yang semakin tinggi. Meskipun demikian, buku ini menggunakan teknik *tiling* untuk mempermudah proses desain arena permainan karena proses tersebut akan menggunakan editor khusus.

4.2 Mengenal Array

Pada pembahasan di atas, sering kali muncul istilah *array*. *Array* adalah struktur data yang berisi sekelompok elemen. Biasanya elemen-elemen ini semua tipe data yang sama, seperti *integer* atau *string*. *Array* biasanya digunakan dalam program komputer untuk mengatur data sehingga seperangkat nilai terkait dapat dengan mudah disortir atau dicari. Sebagai contoh, perhatikan kode berikut :

```
1. var namaHari:Array = ["senin", "selasa", "rabu", "kamis",  
    "jumat", "sabtu", "minggu"];  
2. trace(namaHari[1]); //akan didapatkan nilai selasa
```

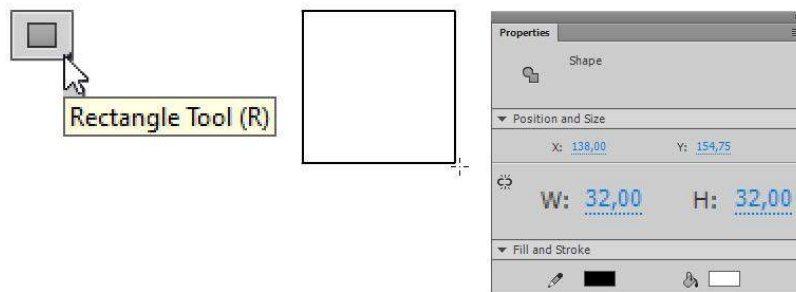
Tutorial 5

Variabel **namaHari** bertipe *array* dan berisi nama-nama hari dalam format *string* (" "). Untuk mengakses data tersebut digunakan *syntax* [], sehingga **namaHari[1]** akan bernilai data ke-2, yaitu "**selasa**" (hal ini dikarenakan perhitungan dalam kode *actionscript* dimulai dari angka 0. Array **namaHari** di atas sering disebut dengan istilah *array* 1 dimensi, sedangkan untuk pembuatan arena permainan biasanya digunakan *array* 2 dimensi.

4.3 Pemrograman Dasar Sistem Tiling

Untuk mempelajari sistem *tiling*, maka diperlukan pemahaman tentang penerapan data bertipe *array* 2 dimensi, serta operasi berulang (*for loop*). Untuk lebih jelasnya perhatikan langkah berikut :

1. Buatlah sebuah file baru sesuai dengan pengaturan file tutorial sebelumnya.
2. Pilih **Rectangle Tool** lalu buatlah sebuah persegi dengan ukuran 32 x 32 *pixel*. Untuk mempermudah proses pembuatan, buatlah sebuah kotak dengan ukuran bebas. *Double click* kotak tersebut, dan buka panel **properties**. Kemudian atur ukuran kotak dengan mengubah nilai *selection width* (W) = 32, dan *selection height* (H) = 32.

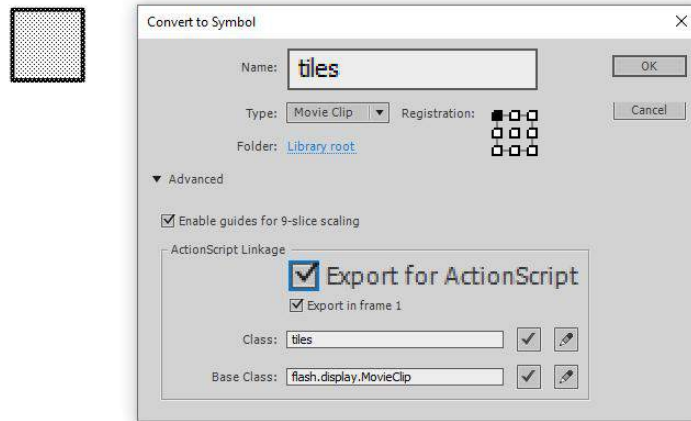


Gambar 4.6 Membuat *tile* dengan *rectangle tool*

3. Seleksi kotak tersebut, kemudian **convert to symbol** menjadi **movieclip** dengan nama **tiles**, pastikan titik registrasi berada di pojok kiri atas. Selanjutnya klik opsi **advanced** dan centang opsi **export for actionscript**, dan klik **OK**.

Flash Tips

Titik registrasi merupakan titik perhitungan koordinat objek. Peletakan titik registrasi di pojok kiri atas dimaksudkan agar perhitungan pengubinan lebih mudah.



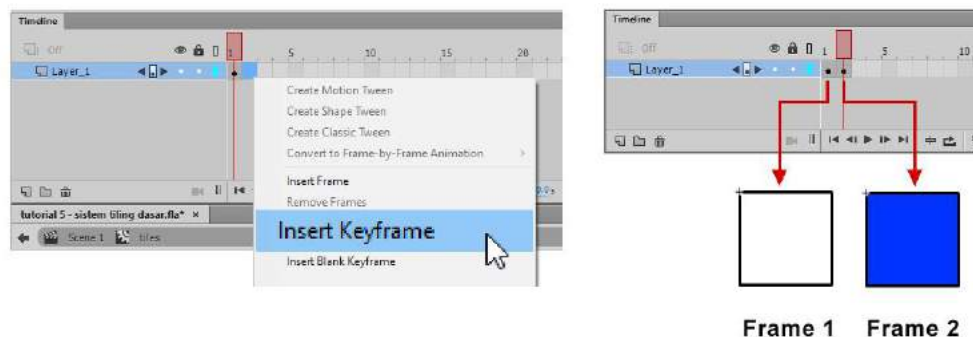
Gambar 4.7 Membuat *movieclip* tiles

4. *Doubleclick* **movieclip tiles** tersebut, untuk mengeditnya. Saat ini anda berada dalam mode **edit symbol**, apapun yang anda lakukan hanya akan berpengaruh pada *movieclip* tiles.



Gambar 4.8 Mode edit *movieclip* tiles

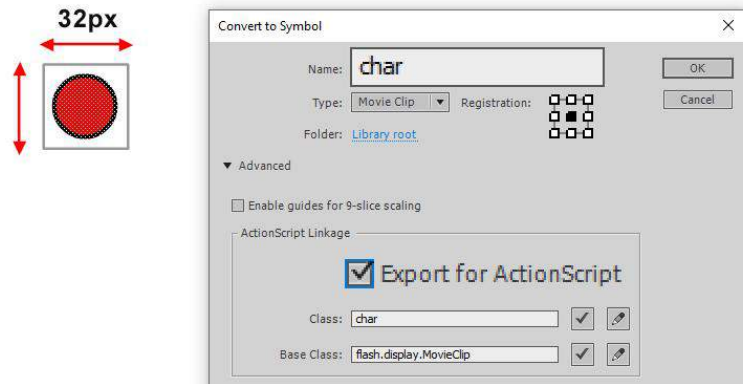
5. Klik kanan **frame 2**, kemudian pilih **Insert Keyframe**. Selanjutnya ubah warna **fill** kotak tersebut menjadi warna biru *solid*. Perbedaan warna digunakan sebagai indikator antara area yang dapat diakses dan area yang tidak dapat diakses.



Gambar 4.9 Menambahkan *keyframe* pada *movieclip* tiles

6. Keluar dari mode *edit symbol* dengan menekan tombol **Ctrl+E**, dan hapus *movieclip tiles* dari layar (*stage*).

- Selanjutnya diperlukan sebuah *symbol* untuk mempresentasikan karakter game. Pilih **Oval Tool**, kemudian buatlah sebuah lingkaran berukuran lebih kecil dari 32 x 32 *pixel*, kemudian seleksi kotak tersebut dan *convert* menjadi **movieclip** dengan nama **char**, pastikan titik registrasi berada di tengah dan pada mode **Advanced**, opsi **Export for Actionscript** tercentang. Lalu klik **OK**.



Gambar 4.10 Membuat *movieclip* char

- Hapus *movieclip* **char** dari layar, sehingga tidak ada objek apapun di layar.
- Klik **frame 1** selanjutnya buka panel **action** dan ketikkan kode berikut :

```

1. import flash.display.MovieClip;
2. import flash.events.Event;
3.
4. var map:Array = [[1,1,1,1,1,1,1,1,1,1],
5.                  [1,0,0,0,0,1,0,0,0,1],
6.                  [1,0,0,0,0,1,0,0,0,1],
7.                  [1,0,0,0,0,1,0,0,0,1],
8.                  [1,1,0,1,1,1,1,0,1,1],
9.                  [1,0,0,0,0,0,0,0,0,1],
10.                 [1,0,0,0,0,0,0,0,0,1],
11.                 [1,0,0,0,1,0,0,0,0,1],
12.                 [1,0,0,0,1,0,0,0,0,1],
13.                 [1,1,1,1,1,1,1,1,1,1]];
14.
15. var game:MovieClip;
16. var peta:MovieClip;
17. var karakter:MovieClip;
18. var tile_w:Number = 32;
19. var karakter_w:Number = 20;
20. var karakter_h:Number = 20;
21. var karakter_speed:Number = 5;
22.
23. function buat_level(level_map:Array, c_x:Number, c_y:Number):void{
24.     game = new MovieClip();
25.     addChild(game);
26.     peta = new MovieClip();

```

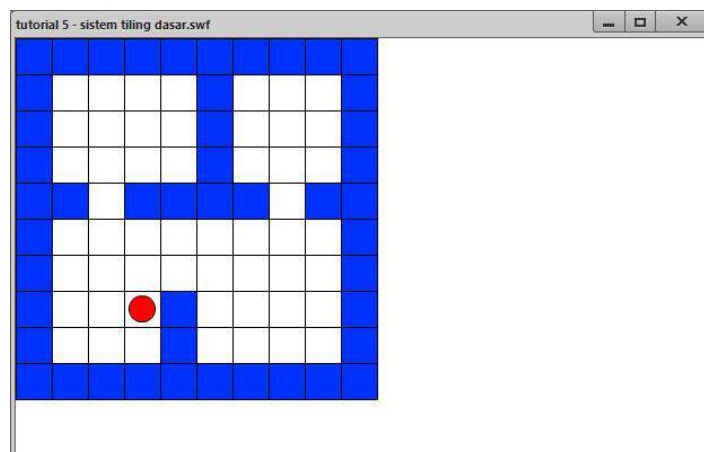


```

27.         game.addChild(peta);
28.         for (var i:Number = 0; i < level_map.length;i++){
29.             for (var j:Number = 0; j < level_map[0].length;j++){
30.                 var tile_map:tiles = new tiles();
31.                 tile_map.gotoAndStop(level_map[j][i]+1);
32.                 tile_map.x = i*tile_w;
33.                 tile_map.y = j*tile_w;
34.                 peta.addChild(tile_map);
35.             }
36.         }
37.         karakter = new char();
38.         karakter.x = c_x*tile_w+tile_w/2;
39.         karakter.y = c_y*tile_w+tile_w/2;
40.         game.addChild(karakter);
41.     }
42.
43.     buat_level(map, 3, 7);

```

10. Jalankan aplikasi dengan menekan tombol **Ctrl+Enter**, maka akan dihasilkan gambar sebagai berikut:



Gambar 4.11 Hasil sistem *tile* dengan data array

Penjelasan Program

```

4.     var map:Array = [[1,1,1,1,1,1,1,1,1,1],
5.                     [1,0,0,0,0,1,0,0,0,1],....

```

Konsep *tiling* yang harus kita pahami pertama kali adalah penggunaan variabel bertipe *Array* 2 dimensi. Variabel **map** merupakan variabel bertipe *array* 2 dimensi. Pada variabel **map** tersebut terdapat 2 nilai yaitu 0 yang merupakan area yang dapat diakses oleh karakter, atau disebut sebagai **walkable area**, dan 1 yang merupakan area yang tidak dapat diakses atau disebut sebagai *wall* atau **unwalkable area**.

Data yang bernilai 1, akan menyebabkan *movieclip tile* ditampilkan pada **frame 2** sedangkan data bernilai 0 akan menampilkan gambar kotak pada **frame 1**. Dengan konsep ini, pembangunan peta atau arena permainan akan lebih mudah dan beragam.

```
15. var game:MovieClip;
16. var peta:MovieClip;
17. var karakter:MovieClip;
```

Variabel utama selanjutnya yang dibutuhkan adalah variabel **game**. Variabel ini bertipe **movieclip**, dan berfungsi sebagai “*container*” atau menyimpan seluruh aset yang akan ditampilkan di dalam game. Teknik menambahkan aset ke dalam sebuah *movieclip* (yang dalam hal ini adalah *movieclip game*) merupakan salah satu teknik yang dapat dilakukan untuk mempermudah proses kontrol/kendali permainan. Dengan meletakkan seluruh aset ke dalam satu kontainer, maka proses menggerakkan objek, menambah dan menghapus objek, dan menghapus game secara keseluruhan akan menjadi lebih mudah.

Variabel berikutnya adalah **peta** dan **karakter**. Peta digunakan untuk menyusun masing-masing *tile* berdasarkan data dari variabel **map**. Sedangkan variabel **karakter** digunakan untuk mengatur karakter game nantinya (baris 37-40).

```
23. function buat_level(level_map:Array, c_x:Number, c_y:Number):void{
```

Fungsi **buat_level** tersebut memiliki 3 variabel yang akan digunakan, yaitu **level_map** yang merupakan data *array* dari peta yang akan digambar. **c_x** dan **c_y** yang merupakan koordinat karakter yang akan dimunculkan di dalam peta.

```
24.     game = new MovieClip();
25.     addChild(game);
```

Dua baris di atas merupakan teknik membuat dan menambahkan *movieclip* baru ke dalam sebuah *stage*. Dengan kode tersebut akan tercipta sebuah *movieclip* kosong yang siap untuk dijadikan “*container*”.

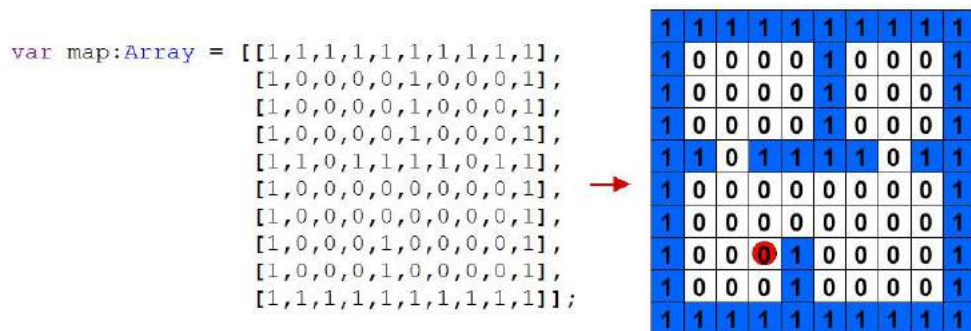
```
28.     for (var i:Number = 0; i < level_map.length;i++){
```

Baris di atas merupakan operasi pengulangan **for**. Operasi pengulangan akan menjalankan satu kode blok secara berulang ulang sampai kondisi terpenuhi. Dalam hal ini pengulangan dimulai dari data **i** ke 0 dan berakhir saat **i** tepat kurang dari lebar data **map**. Dengan adanya pengulangan ini, maka pada baris selanjutnya *movieclip*

tiles akan dibuat dan diposisikan secara berulang-ulang sesuai dengan data variabel **map**.

```
30.         var tile_map:tiles = new tiles();
31.         tile_map.gotoAndStop(level_map[j][i]+1);
32.         tile_map.x = i*tile_w;
33.         tile_map.y = j*tile_w;
34.         peta.addChild(tile_map);
```

Operasi pengulangan **for** bertingkat (**i** dan **j**), selanjutnya mengeksekusi kode blok baris 30-34. Pada baris 30 dibuat sebuah *movieclip* baru yang mengacu pada *movieclip* **tiles**. Pada *movieclip* baru tersebut dilakukan pengaturan *frame* aktif dengan kode **gotoAndStop**. Pengaturan tersebut mengacu pada data variabel **map**, misal **map[0][0]** bernilai **1**, maka *frame* akan diaktifkan menuju *frame* **2** (*tile* berwarna biru). Perhatikan gambar :



Gambar 4.12 Visualisasi data *array* dalam sistem *tiling*

```
43. buat_level(map, 3, 7);
```

Pada baris terakhir, fungsi **buat_level** diaktifkan, dengan meletakkan karakter pada koordinat (3, 7) yaitu pada kolom ke 4 dan pada baris ke 8 di arena permainan.

Tutorial 6

4.4 Mengatur Gerakan Karakter dalam Sistem *Tiling*

Setelah memahami teknik membentuk peta atau arena permainan, tahapan selanjutnya adalah menggerakkan karakter. Untuk lebih jelasnya perhatikan langkah berikut :

1. Lanjutkan file pada **tutorial 5**, simpan dengan nama lain (**tutorial 6 fla**). Kita akan melakukan beberapa penambahan fitur gerakan karakter.
2. Pada kode baris ke-42 sisipkan kode berikut :

```
44. var key_up:Boolean=false;
45. var key_down:Boolean=false;
46. var key_left:Boolean=false;
```

```

47. var key_right:Boolean=false;
48.
49. function keyDownFunction(event:KeyboardEvent):void {
50.     if (event.keyCode == 38) {
51.         key_up=true;
52.     } else if (event.keyCode == 39) {
53.         key_right=true;
54.     } else if (event.keyCode == 40) {
55.         key_down=true;
56.     } else if (event.keyCode == 37) {
57.         key_left=true;
58.     }
59. }
60. function keyUpFunction(event:KeyboardEvent) {
61.     if (event.keyCode == 38) {
62.         key_up=false;
63.     } else if (event.keyCode == 39) {
64.         key_right=false;
65.     } else if (event.keyCode == 40) {
66.         key_down=false;
67.     } else if (event.keyCode == 37) {
68.         key_left=false;
69.     }
70. }
71. function jalankan_game(e:Event):void{
72.     if (key_up) gerakkan_karakter(0, -karakter_speed);
73.     if (key_down) gerakkan_karakter(0, karakter_speed);
74.     if (key_left) gerakkan_karakter(-karakter_speed, 0 );
75.     if (key_right) gerakkan_karakter(karakter_speed, 0);
76. }
77.
78. function gerakkan_karakter(sx:Number, sy:Number):void{
79.     var p_x:Number = Math.floor(karakter.x/tile_w);
80.     var p_y:Number = Math.floor(karakter.y/tile_w);
81.     var next_x:Number = karakter.x+sx;
82.     var next_y:Number = karakter.y+sy;
83.     var c_w:Number = karakter_w/2;
84.     var c_h:Number = karakter_h/2;
85.     var up:Number = Math.floor((next_y-c_h)/tile_w);
86.     var down:Number = Math.floor((next_y+c_h)/tile_w);
87.     var left:Number = Math.floor((next_x-c_w)/tile_w);
88.     var right:Number = Math.floor((next_x+c_w)/tile_w);
89.     if (sy<0){
90.         if (map[up][p_x] == 0 && map[up][left] == 0 &&
            map[up][right] == 0) {
91.             karakter.y += sy;
92.         }
93.     }else{

```

```

94.         if (map[down][p_x] == 0 &&
              map[down][left] == 0 && map[down][right] == 0){
95.             karakter.y += sy;
96.         }
97.     }
98.     if (sx<0){
99.         if (map[p_y][left] == 0 && map[up][left] == 0 &&
              map[down][left] == 0){
100.             karakter.x += sx;
101.         }
102.     }else{
103.         if (map[p_y][right] == 0 && map[up][right] == 0 &&
              map[down][right] == 0) {
104.             karakter.x += sx;
105.         }
106.     }
107. }
108.
109.     buat_level(map, 3, 7);
110.     stage.addEventListener(KeyboardEvent.KEY_DOWN,
                             keyDownFunction);
111.     stage.addEventListener(KeyboardEvent.KEY_UP, keyUpFunction);
112.     game.addEventListener(Event.ENTER_FRAME, jalankan_game);

```



baris 109 sudah ditulis pada tutorial sebelumnya, sehingga perlu dipastikan tidak tertulis 2 kali

3. Jalankan kembali dengan menekan **Ctrl+Enter**, klik *stage* dan tekan tombol panah untuk menggerakkan karakter.

Flash Tips

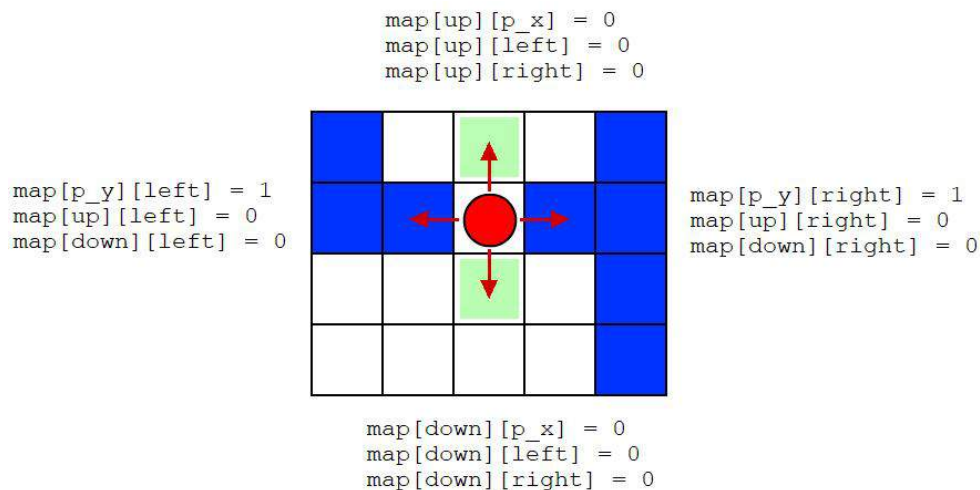
Dalam menjalankan aplikasi yang melibatkan *input Keyboard*, *stage* harus di klik terlebih dahulu agar aplikasi merespon. Hal ini dikarenakan fokus aplikasi belum aktif. Hal ini tidak perlu dilakukan jika anda menjalankan file SWF atau EXE secara langsung melalui Windows Explorer, atau ketika aplikasi dipublish dengan output APK (untuk Android) .

Penjelasan Program

Penambahan fitur gerakan karakter pada dasarnya adalah dengan menambahkan listener untuk membaca input keyboard (baris 110-111). Pembacaan input *keyboard* dilakukan secara berulang sepanjang aplikasi berjalan melalui fungsi ***jalankan_game***. Pada fungsi tersebut, masing-masing input *keyboard* akan mengaktifkan fungsi ***gerakan_karakter***. Dimana gerakan karakter dihitung berdasarkan posisinya terhadap data peta (variabel ***map***). ketika karakter bergerak maka dilakukan

perhitungan gerakan terlebih dahulu, apabila karakter bergerak ke **unwalkable tile** maka karakter tidak jadi dijalankan. Sebaliknya karakter baru akan berjalan, jika langkah selanjutnya menuju ke area yang masih kosong (**walkable tile**).

Sebagai contoh, perhatikan gambar 4.13. Saat karakter bergerak, maka karakter akan melakukan deteksi gerakan selanjutnya. Karakter hanya dapat bergerak, ke area yang ke semua arah gerakan selanjutnya memiliki nilai 0. Sehingga dalam contoh gambar, karakter hanya bisa bergerak ke atas atau ke bawah saja.



Gambar 4.13 Sistem deteksi gerakan dalam sistem *tiling*

Dengan penjelasan dasar tentang sistem *tiling*, pada tahapan selanjutnya sistem *tiling* akan diintegrasikan dengan *tileset* serta animasi karakter. Oleh karena itu pastikan anda memahami konsep *tiling* terlebih dahulu sebelum melangkah pada tahapan selanjutnya.

Tutorial 7

4.5 Menggambar Peta dengan Menggunakan *Bitmap Data*

Pada tutorial 6 telah dijelaskan teknik menambahkan *tile* dengan dua jenis yaitu *walkable* dan *unwalkable tiles*. Dalam game, masing-masing *tile* tersebut akan menampilkan grafik yang berbeda-beda yang berasal dari *tileset* yang ada (lihat konsep *tile* game Super Mario pada gambar 4.2).

Terdapat beberapa teknik dalam mengatur *tileset* menjadi sebuah arena permainan. Dalam pengalaman saya membuat game dengan teknik *tiling*, beberapa teknik yang pernah saya gunakan antara lain :

1. Teknik *frame by frame*

Teknik *frame by frame* dilakukan dengan meletakkan masing-masing *tile* ke dalam sebuah *movieclip*. Hal ini dilakukan dengan memecah *tileset* melalui

aplikasi grafis seperti Photoshop, kemudian mengimport gambar secara *frame by frame* ke dalam sebuah *movieclip*. Contoh dari game buatan saya yang menerapkan ini adalah game Desert Eagle (Copyright FOG, 2006), Sim Taxi series (Copyright GameNinja dan Froyogames, 2010), dan Ace Gangster (Copyright Froyogames, 2012).



Gambar 4.14 Game Sim Taxi menggunakan teknik *frame by frame tiling*

2. Teknik *Bitmap Data*

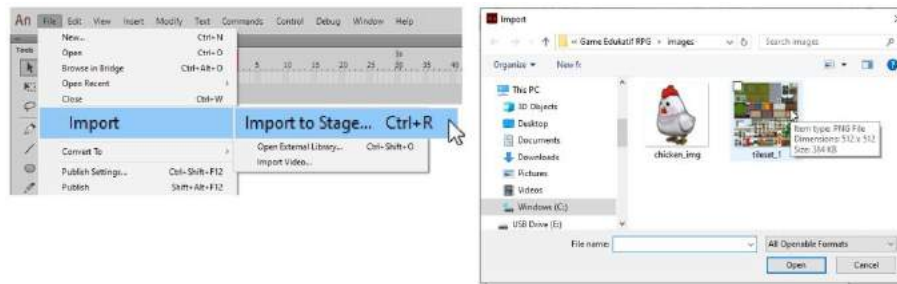
Teknik ini merupakan teknik menggambar ulang *pixel by pixel* untuk membentuk gambar baru. Dalam hal ini satu bagian kecil (*tile*) yang bersumber dari *tileset* disalin ke dalam gambar baru. Dengan operasi berulang, data yang bertipe *array* dijadikan sebagai pedoman dalam mengambil warna *pixel* untuk membuat sebuah gambar *bitmap* baru.



Gambar 4.15 Proyeksi *tileset* dengan data *array* untuk membentuk *BitmapData*

Dalam buku ini, akan digunakan teknik *BitmapData* untuk mempermudah proses pembuatan arena permainan, pengaturan asset dan pemrograman permainan. Sebagai contoh pada tutorial ini akan ditunjukkan teknik merubah *tileset* menjadi sebuah area permainan. Untuk lebih jelasnya perhatikan langkah berikut :

1. Buatlah sebuah file baru dengan pengaturan seperti pada tutorial sebelumnya.
2. Import gambar **tileset_1.png** dari *folder images* dengan memilih opsi **File > Import to Stage. Tileset_1.png** merupakan satu *tileset* siap pakai berukuran 512x512 *pixel* dengan satu *tile* berukuran 32x32 *pixel*.



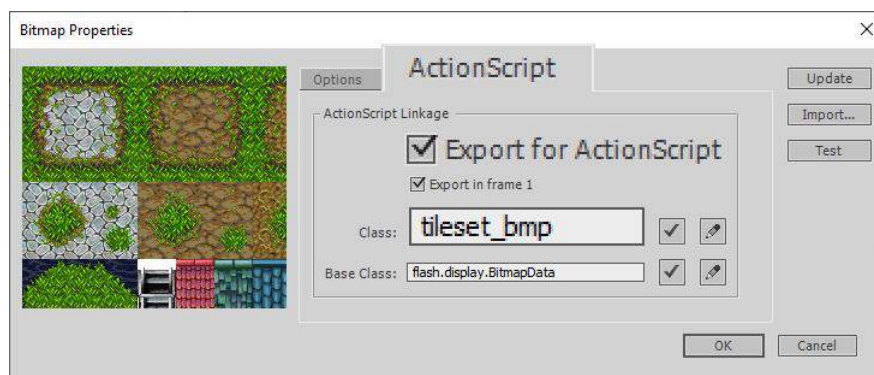
Gambar 4.16 Mengimport *tileset* ke *stage*

3. Buka panel **library** dengan menekan **Ctrl+L**, kemudian klik kanan **tileset_1** kemudian pilih **Properties**. Pada panel **properties** ubah opsi *compression* menjadi **Lossless (PNG/GIF)** dan matikan opsi **Allow smoothing** hal ini bertujuan untuk mempertahankan karakteristik *pixelart* dan mempertahankan kualitas gambar.



Gambar 4.17 Pengaturan *properties bitmap*

4. Kemudian pada panel **Bitmap Properties** tersebut, pindah ke panel **Actionscript** (untuk Adobe Animate), atau klik tombol **Advanced** (untuk Adobe Flash CS). Centang opsi **Export for Actionscript**, kemudian ubah nama *class* menjadi **tileset_bmp**. Kemudian klik **OK**.



Gambar 4.18 Mengatur panel *Actionscript* untuk *symbol bitmap*

5. Dalam tutorial ini digunakan data *array map* yang dihasilkan oleh aplikasi editor peta. Data tersebut disimpan dalam file **map-1.as**. File berekstensi **.as** merupakan kode *actionscript*, yang dapat diimport oleh aplikasi.

Struktur data yang ada dalam file **map-1.as** pada dasarnya mirip dengan variabel **map** pada tutorial sebelumnya, hanya saja karena banyaknya *tile* yang ada di dalam *tileset*, sehingga dibutuhkan editor khusus untuk membuatnya. Penjelasan tentang editor terdapat pada Bab selanjutnya.

```
var map_1:Array =
[[[0,97,0],[0,97,0],[0,97,0],[0,97,0],[0,97,0],[0,97,0],[0,97,0],
[0,97,0],[0,97,0],[0,97,0]], [[0,97,0],[0,0,0],[0,16,0],[0,16,0],[
0,16,0],[0,32,0],[0,6,0],[0,22,0],[0,38,0],[0,97,0]], [[0,97,0],[0,
1,0],[0,17,151],[0,17,0],[0,50,0],[0,33,0],[0,8,0],[0,70,0],[0,3
9,0],[0,97,0]], [[0,84,0],[0,100,0],[0,116,0],[0,132,0],[0,17,0],[
0,33,0],[0,97,0],[0,8,0],[0,40,0],[0,97,0]], [[0,84,0],[0,100,0],[
0,117,0],[0,133,0],[0,17,0],[0,33,0],[0,97,0],[0,97,224],[0,97,0]
,[0,97,0]], [[0,84,0],[0,100,0],[0,117,201],[0,133,217],[0,17,0],[
0,49,0],[0,32,0],[0,97,0],[0,97,0],[0,97,0]], [[0,84,0],[0,100,0],
[0,118,0],[0,134,0],[0,17,0],[0,17,0],[0,33,0],[0,97,0],[0,97,0],
[0,97,0]], [[0,97,0],[0,1,0],[0,50,0],[0,17,0],[0,17,0],[0,17,0],[
0,33,0],[0,97,0],[0,97,0],[0,97,0]], [[0,97,0],[0,2,0],[0,18,0],[0,
18,0],[0,18,0],[0,18,0],[0,34,0],[0,97,0],[0,97,172],[0,97,0]], [
[0,97,0],[0,97,0],[0,97,0],[0,97,0],[0,97,0],[0,97,0],[0,97,0],[0,
97,0],[0,97,0],[0,97,0]]];
```

Gambar 4.19 Struktur data yang dihasilkan oleh editor



Dikarenakan cukup rumit untuk mengetik kode di atas, maka anda cukup mengcopy file **map-1.as** ke *folder* tempat anda menyimpan tutorial ini.

6. Klik **frame 1** kemudian buka panel **action** dan ketikkan kode berikut :

```
1. import flash.display.MovieClip;
2. import flash.display.Bitmap;
3.
4. var tile_set:BitmapData;
5. var game:MovieClip;
6. var peta:MovieClip;
7. var canvas:MovieClip;
8. var tile_bmp:Bitmap;
9. var item_bmp:Bitmap;
10.
11. var map_w:Number = 10; //lebar peta
12. var map_h:Number = 10; //tinggi peta
13. var t_size:Number = 32; //tile size : ukuran tile
14. var m_size:Number = 512; //master size : ukuran tileset
15. var tile_num:Number = Math.floor(m_size/t_size);
16.
17. include "map-1.as"; //membuka file external
18.
19. var map:Array = map_1;
20.
21. function set_canvas():void{
22.     //menjadikan tileset sebagai gambar sumber
23.     var bmp:Bitmap = new Bitmap(new tileset_bmp());
```

```

24.     tile_set = bmp.bitmapData;
25.     //membuat container untuk potongan tile (background)
26.     canvas = new MovieClip();
27.     addChild(canvas);
28.     var bmp_handler:MovieClip = new MovieClip();
29.     canvas.addChild(bmp_handler);
30.     tile_bmp = new Bitmap(new BitmapData(map_w * t_size,
                                           map_h * t_size,true,0x00000000));
31.     bmp_handler.addChild(tile_bmp);
32.     //membuat container untuk potongan tile (foreground)
33.     var item_handler:MovieClip = new MovieClip();
34.     canvas.addChild(item_handler);
35.     item_bmp = new Bitmap(new BitmapData(map_w * t_size,
                                           map_h * t_size,true,0x00000000));
36.     item_handler.addChild(item_bmp);
37.     //membuat container untuk potongan tile
38.     var tile_handler:MovieClip = new MovieClip();
39.     canvas.addChild(tile_handler);
40.     tile_bmp = new Bitmap(new BitmapData(map_w * t_size,
                                           map_h * t_size,true,0x00000000));
41.     var bmd1:BitmapData = tile_set;
42.     var bmd2:BitmapData = tile_set;
43.     for (var i:Number = 0; i < map_w; i++){
44.         for (var j:Number = 0; j < map_h; j++){
45.             var t_type:Number = map[i][j][1];
46.             //background
47.             if (t_type > -1){
48.                 var sy:Number = Math.floor(t_type / tile_num);
49.                 var sx:Number = t_type-(sy*tile_num);
50.                 var rect:Rectangle = new Rectangle(sx * t_size,
                                                    sy * t_size,t_size,t_size);
51.                 var pt:Point = new Point(i * t_size,j * t_size);
52.                 tile_bmp.bitmapData.copyPixels(bmd1, rect, pt);
53.             }
54.             var i_type:Number = map[i][j][2];
55.             //foreground
56.             if (i_type > 0){
57.                 var iy:Number = Math.floor(i_type / tile_num);
58.                 var ix:Number = i_type-(iy*tile_num);
59.                 var i_rect:Rectangle = new Rectangle(ix*t_size,
                                                    iy * t_size,t_size,t_size);
60.                 var i_pt:Point = new Point(i * t_size,
                                                    j * t_size);
61.                 item_bmp.bitmapData.copyPixels(bmd1,i_rect,
                                                    i_pt);
62.             }
63.         }
64.     }
65.     bmp_handler.addChild(tile_bmp);

```

```

66.     item_handler.addChild(item_bmp);
67. }
68.
69. function buat_level():void{
70.     //mengatur properti game
71.     game = new MovieClip();
72.     addChild(game);
73.     peta = new MovieClip();
74.     game.addChild(peta);
75.     //menambahkan canvas
76.     set_canvas();
77.     //menjadikan canvas sebagai gambar sumber
78.     var peta_bd:BitmapData = new BitmapData(map_w*t_size,
                                                map_h * t_size,true,0x00000000);
79.     var peta_bmp:Bitmap = new Bitmap(peta_bd);
80.     peta.addChild(peta_bmp);
81.     peta_bd.draw(canvas);
82.     //hapus canvas setelah digambar ulang
83.     removeChild(canvas);
84. }
85.
86. buat_level();

```

7. Jalankan aplikasi dengan menekan **Ctrl+Enter**. Jika tidak ada kesalahan, maka akan didapatkan gambar berikut :



Gambar 4.20 Hasil sistem *tiling* dengan metode *bitmapData*

Penjelasan Program

Perhatikan baris 52 berikut :

```
52. tile_bmp.bitmapData.copyPixels(bmd1, rect, pt);
```

Disinilah letak aspek teknis utama penggunaan metode *BitmapData*. Sebuah *tile* dengan ukuran yang didefinisikan sebelumnya (dengan variabel **rect**) mengacu pada titik **pt** yang diperoleh dari data *array* **map**. Gambar *tile* tersebut dimasukkan ke dalam *movieclip* **canvas** (baris 34 dan 36).

```
81. peta_bd.draw(canvas);
```

Setelah semua *tile* tergambar ke dalam *movieclip* **canvas**, selanjutnya *canvas* akan dijadikan *bitmap* dan digambar ke dalam satu *bitmap* besar yaitu **peta_bd**.

Flash Tips

Pada bab selanjutnya, penulisan kode pembuatan level permainan tidak terlalu panjang, karena fungsi-fungsi utama telah diringkas dan dijadikan menjadi sebuah file kode siap pakai. Jadi anda tidak perlu khawatir karena tidak perlu menulis kode yang panjang

Bab ini hanya untuk menunjukkan kode dasar yang digunakan untuk membangun sebuah area permainan game RPG dengan teknik *tiling* dan *bitmapData*. Apabila anda masih mengalami kesulitan, anda tetap dapat melanjutkan ke bab selanjutnya.

BAB V

PENGEMBANGAN ARENA PERMAINAN

Arena permainan merupakan salah satu elemen penting dalam game RPG. Untuk mengembangkannya diperlukan sebuah editor khusus yang dapat memfasilitasi desain arena dengan sistem *tiling* dan teknik *BitmapData*.

PENGEMBANGAN PETA / ARENA PERMAINAN

5.1 Aplikasi Editor untuk Mengembangkan Peta / Arena Permainan

Salah satu elemen terpenting dalam game RPG adalah peta atau area permainan. Menurut pengalaman, bagi pemula membuat peta atau arena permainan merupakan kesulitan terbesar yang harus dilakukan jika ingin game yang dihasilkan dapat dikategorikan sebagai game RPG. Oleh sebab itu beberapa *tool* untuk mengembangkan game seperti RPG Maker, Game Maker, Construct, Unity dan sebagainya memiliki fitur *level editor* untuk membuat peta secara visual. Sayangnya Adobe Flash atau Adobe Animate tidak memiliki editor visual berbasis *tile*, sehingga banyak game Flash bertipe RPG yang menggunakan teknik *art base* (menggerakkan gambar secara keseluruhan).

Sesuai dengan penjelasan pada bab sebelumnya, tutorial membuat game RPG dalam buku ini menggunakan teknik *tiling*, dimana gambar yang menyusun sebuah peta/area permainan disusun dari potongan-potongan kecil yang diulang-ulang peletakkannya berdasarkan variabel bertipe *Array*. Pada tutorial sebelumnya dijelaskan bahwa metode ini bertujuan untuk menghemat *memory* sekaligus mengoptimalkan kecepatan game. Namun demikian menyusun peta cukup merepotkan karena penggunaan variabel bertipe *array* yang cukup kompleks.

Pada tahun 2002 ketika belajar Flash, sebagai pemula saya membuat peta secara manual, menggunakan bantuan kertas grafik dan menuliskan kode satu demi satu (contoh kasus desain peta game Sim Taxi). Dalam game tersebut, peta seluas 100x100 *tile* membutuhkan beberapa hari untuk membuatnya. Hal ini membutuhkan ketekunan dan ketelitian, serta menyita begitu banyak waktu, sehingga sering mematahkan semangat seorang pemula dalam mengembangkan game.



Gambar 5.1 Game Sim Taxi (2004) dengan pengembangan peta secara manual

Seiring dengan pengalaman dalam mengembangkan game, saya membuat **editor** level untuk mempermudah proses kerja saya. Pada awalnya saya mengembangkan sistem editor sederhana untuk membuat data *Array*, seperti yang saya gunakan pada game Black Eagle (FOG.com, 2006) dan generasi baru Sim Taxi (FOG.com, 2006). Pada tahapan selanjutnya, editor dilengkapi dengan sistem gerakan NPC (*Non Payable Character*), sehingga dapat digunakan untuk game yang melibatkan gerakan kendaraan umum dan keramaian publik secara terstruktur, seperti yang saya gunakan dalam game Gangster Life (FOG.com, 2007), City Driver (FOG.com, 2007), Sim Taxi 2 (GameNinja, 2010), Sim Taxi Series (GamesBook.com, 2012) dan Ace Gangster (Froyogames.com, 2012). Pada pengembangan selanjutnya dibuat editor untuk game dengan grafik 3D, yaitu untuk game Sim Taxi 3D (Froyogames.com, 2013), dan Ace Gangster 2 (Froyogames.com, 2014). Sedangkan khusus untuk mengembangkan game RPG dengan grafik 2D, saya kembangkan editor khusus yang akan dibahas lebih detail pada subbab selanjutnya.



Gambar 5.2 Map Editor untuk pengembangan game berbasis *tile*.

5.2 Map Editor RPG berbasis *Bitmap Data*

Untuk mempermudah pembuatan level/peta/area untuk game berbasis *tile*, saya membuat sebuah editor visual berbasis *tileset bitmap*. Berbeda dengan 3 level editor yang saya buat di tahun-tahun sebelumnya yang menggunakan pengaturan file

berdasarkan *frame* sebuah *movieclip*, editor ini saya buat berdasarkan teknik *tileset*. Artinya dalam membangun sebuah peta digunakan 1 buah file *bitmap* sebagai acuan. Penggunaan *tileset* sangat umum ditemui dalam pembuatan game RPG seperti dalam aplikasi RPG Maker.

Secara default map editor yang saya buat memiliki *tileset* yang saya peroleh dari situs opengameart.org, yang selanjutnya dapat diatur sesuai dengan kebutuhan developer. Adapun *interface* dari aplikasi editor yang dimaksud adalah sebagai berikut :



Gambar 5.3 *Interface* map editor FlashRPG

Flash Tips

Pembuatan *tileset* 2D membutuhkan skill grafis seperti teknik *pixel art*, *digital painting*, atau *graphic editing* menggunakan *software* grafis. Biasanya skill ini dimiliki secara khusus oleh tim pengembang grafis. Oleh karena itu, bagi pemula yang ingin membuat game dengan grafik *tileset* dapat memanfaatkan gambar siap pakai yang tersedia di beberapa situs, salah satunya dari situs opengameart.org. Dalam penggunaan grafik siap pakai, anda harus mengetahui jenis lisensi dari grafik tersebut. Secara umum pada grafis yang bertipe *open/freelicense* mewajibkan pengembang untuk mencantumkan nama pembuat grafik di bagian *game's credit*.

5.3 Membuat Peta / Arena Permainan dengan Map Editor

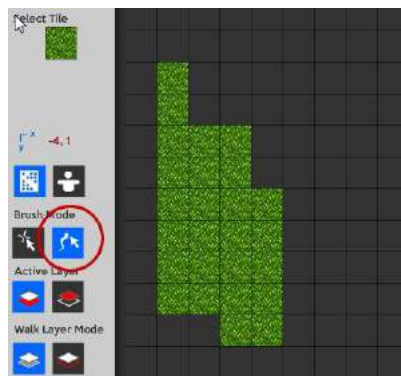
Untuk memahami fungsi masing-masing tools yang terdapat pada aplikasi map editor di atas, ikuti tutorial berikut:

1. Bukalah aplikasi map editor yang terdapat pada file tutorial, atau buka editor online pada link <http://www.wandah.org/map-editor>
2. Secara default ukuran peta adalah **10x10 tile**, masing-masing *tile* memiliki ukuran **32 pixel** dan menggunakan *tileset* **default** (file **tileset_1.png** pada *folder* file tutorial)
3. Klik tombol **new** untuk membuat file baru. Ketikkan **Map ID 2** (karena map id 1 sudah digunakan pada tutorial sebelumnya. Langkah ini akan menghasilkan variabel *array* bernama **map_2**. Ketikkan **32** pada map size, dan *tileset* kita gunakan *default*. Namun jika anda ingin menggunakan *tileset* lain, klik tombol **browse**, pilih *tileset* yang anda gunakan dan sesuaikan nilai **tile size** sesuai dengan *tileset* yang anda gunakan.



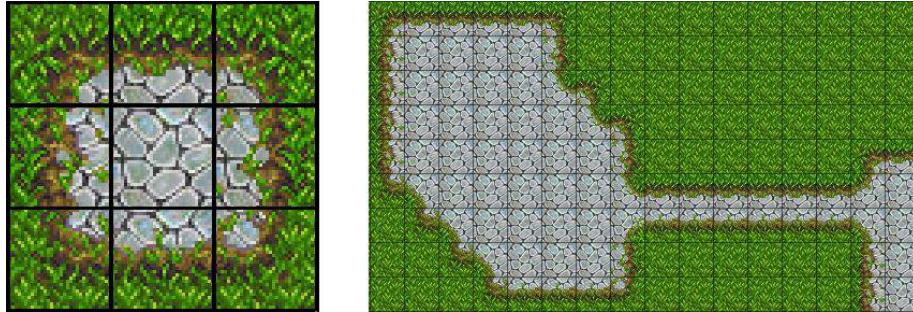
Gambar 5.4 Membuat file baru

4. Fitur editor peta yang pertama adalah mode **Click** dan mode **brush**. Mode **click** berarti setiap *grid* dapat diubah dengan mengklik *tile*, sedangkan mode *brush* *tile* akan berubah jika tombol kiri mouse ditekan (didrag). Pilih *tile* aktif, kemudian klik editor untuk menambahkan *tile*. Anda dapat menggerakkan area editor dengan tombol keyboard **ASDW** atau dengan **Space+Drag**.



Gambar 5.5 Mode *click* dan *brush* pada *tile* editor

5. Dalam membuat sebuah arena permainan, diperlukan kesabaran dan ketelitian. Cobalah untuk membuat *background* rumput yang luas terlebih dahulu, kemudian tambahkan jalan setapak atau jalan batuan. Konsep dari *tileset* pada umumnya menggunakan 9 **grid** (9 kotak *tile*), dimana dengan kombinasi 2x9 petak ini anda dapat membuat berbagai kombinasi. Perhatikan gambar :



Gambar 5.6 Konsep 9 *grid* dalam membentuk arena permainan

6. Aplikasi map editor memiliki 2 *layer* (lapisan), *layer* bawah (*layer 1*) dan *layer* atas (*layer 2*). Secara *default*, *layer* aktif adalah **Layer 1** yang merupakan *layer* dengan posisi di bawah. Mode **Layer 1** ini efektif jika digunakan untuk menambahkan *tile background* seperti jalan, rumput, dan sebagainya.



Gambar 5.7 Mode *layer 1* untuk *background*

7. Sedangkan **Layer 2** yang merupakan *layer* dengan posisi di atas efektif jika digunakan untuk menambahkan objek-objek dengan *background* transparan seperti batu, rumput, pintu, dan item-item lainnya.



Gambar 5.8 Mode *layer 2* untuk objek di atas *background*

8. Gunakan kreatifitas anda dalam mendesain arena permainan. Tambahkan beberapa variasi seperti beberapa bangunan, semak-semak, pepohonan, jalan setapak, jalan bebatuan dan sebagainya, sesuai dengan *tileset* yang digunakan.



Gambar 5.9 Gambar hasil pengeditan *tile*.

9. Secara *default*, ketika anda mengedit peta, anda sedang berada pada mode **Edit All Layer**, sehingga perubahan tidak hanya terjadi pada grafis saja, tetapi juga pada mode *tile* (**walkable** atau **wall/unwalkable**). Secara default tipe *tile* adalah **walkable**, namun demikian setelah anda selesai dengan proses *editing* grafis, anda dapat memilih tombol **walk layer** untuk melakukan *editing mode tile*.



Gambar 5.10 Mode *edit walk layer*

10. Klik **Tile Mode : Wall**, untuk membuat area yang tidak dapat diakses oleh karakter. Dengan mode **walk layer** anda dapat melakukan editing tanpa merubah grafis yang ada.

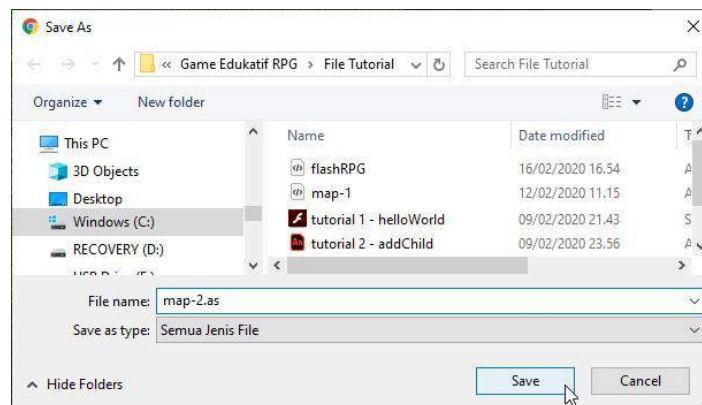


Langkah ini bertujuan untuk menentukan apakah nantinya area dapat dilalui oleh karakter atau tidak. Sebagai contoh jika *tile* berupa batuan, pohon, air, atau dinding maka karakter tidak dapat melaluinya, sehingga pada editor kita harus merubah mode tile menjadi tipe **wall/unwalkable**, yang ditandai dengan warna merah.



Gambar 5.11 Menambahkan *tile mode unwalkable/wall*

11. Setelah peta selesai dibuat / dalam progres, anda dapat menyimpannya dengan menekan tombol **Save**. Gunakan ekstensi file **.as** untuk mempermudah proses *editing* selanjutnya, misal berikan nama **map-2.as**.



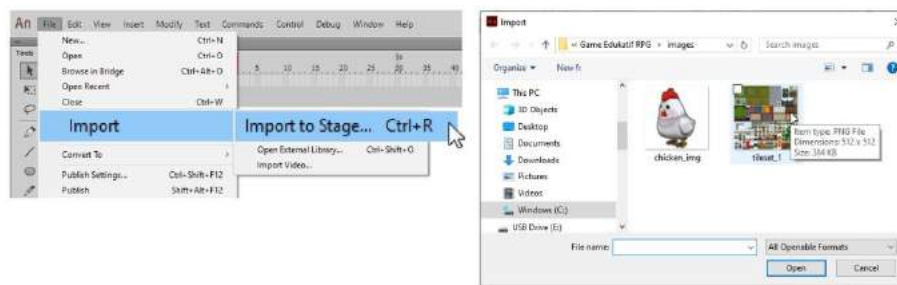
Gambar 5.12 Menyimpan file arena permainan

Tutorial 8

5.4 Menguji Coba Peta / Arena Permainan

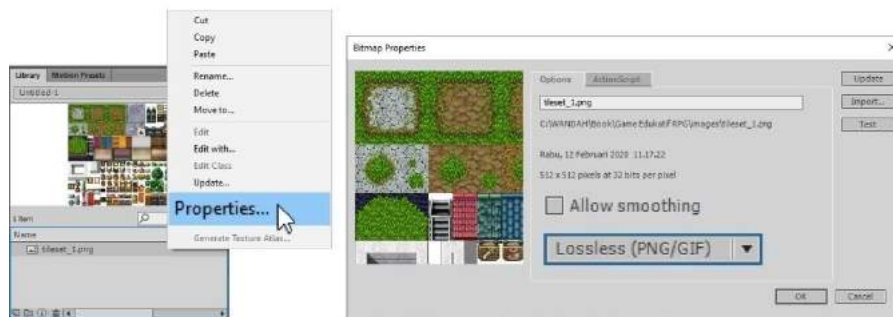
Setelah peta selesai dibuat, peta perlu diuji coba untuk mengetahui apakah peta bekerja dengan baik atau tidak. Ikuti langkah-langkah berikut:

1. Buatlah sebuah file baru dengan pengaturan seperti pada tutorial sebelumnya. Simpan file dengan nama **tutorial-8 fla**.
2. *Copy* dua file yaitu file peta permainan **map-2.as** dan file **flashRPG.as** dari *folder* file tutorial ke satu *folder* yang sama dengan tempat anda menyimpan file tutorial ini.
3. Import gambar *tileset* yang anda gunakan untuk membuat peta (dalam hal ini adalah file **tileset_1.png** dari *folder images*) dengan memilih opsi **File > Import to Stage**.



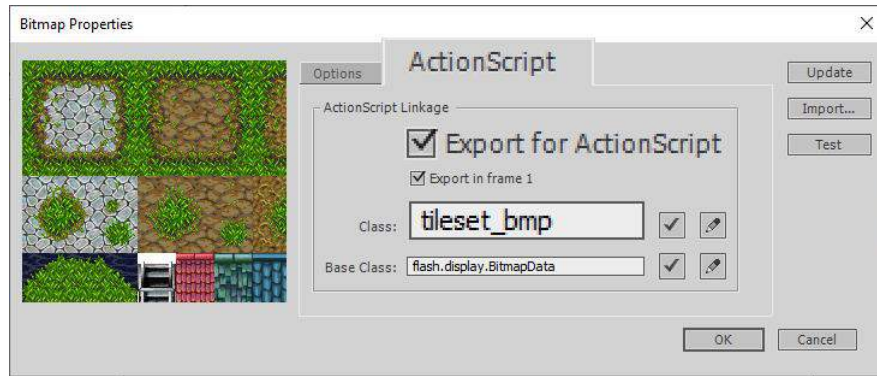
Gambar 5.13 Mengimport *tileset* ke *stage*

4. Buka panel **library** dengan menekan **Ctrl+L**, kemudian klik kanan **tileset_1** kemudian pilih **Properties**. Pada panel **properties** ubah opsi *compression* menjadi **Lossless (PNG/GIF)** dan matikan opsi **Allow smoothing** hal ini bertujuan untuk mempertahankan karakteristik *pixelart* dan mempertahankan kualitas gambar.



Gambar 5.14 Pengaturan *properties bitmap*

5. Kemudian pada panel *Bitmap Properties* tersebut, pindah ke panel **Actionscript** (untuk Adobe Animate), atau klik tombol **Advanced** (untuk Adobe Flash CS). Centang opsi **Export for Actionscript**, kemudian ubah nama *class* menjadi **tileset_bmp**. Kemudian klik **OK**.

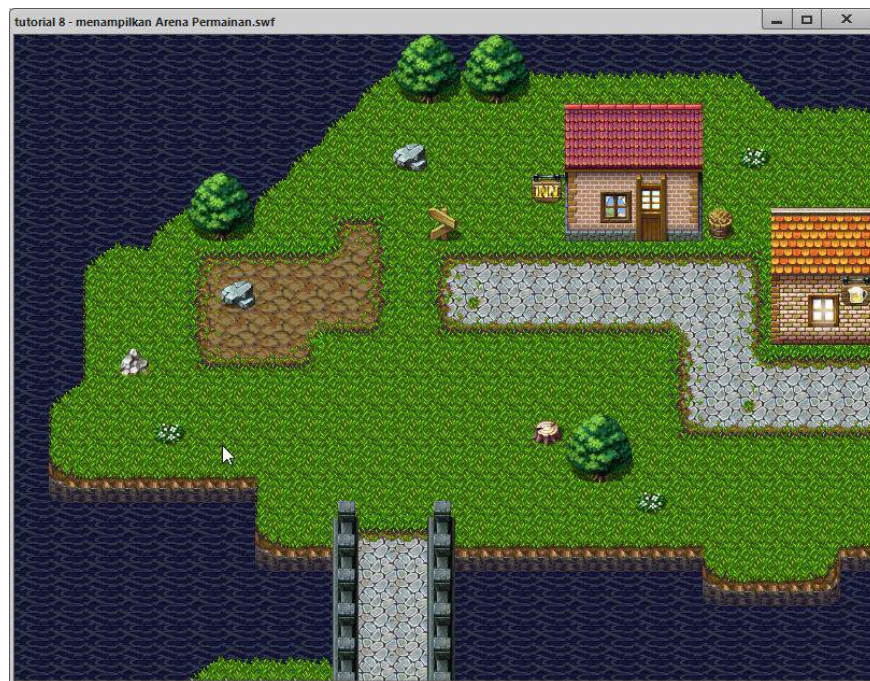


Gambar 5.16 Mengatur panel *Actionscript* untuk *symbol bitmap*

6. Seleksi gambar *tileset* yang ada pada *stage*, dan hapus *tileset* dari *stage* dengan menekan tombol **Delete**, sehingga tidak ada objek apapun di *stage*.
7. Klik **frame 1** kemudian buka panel **Action**. Ketikkan kode berikut :

```
1. include "map-2.as";
2. include "flashRPG.as";
3.
4. buat_level (map_2, "tileset_bmp");
```

8. Jalankan aplikasi dan arena permainan akan tampak di layar.



Gambar 5.17 Hasil ujicoba arena permainan

Penjelasan Kode

Pada tutorial di atas hanya terdapat 3 baris kode. Hal ini dikarenakan kumpulan kode fungsi telah dituliskan ke dalam file **flashRPG.as**, sehingga cukup dengan memanggilnya dengan kode `include "flashRPG.as";`.

Sedangkan pada baris ke-4 terdapat aktivasi untuk fungsi **buat_level** dimana terdapat variabel **map_2** yang diperoleh dari file **map-2.as** hasil editor peta, dan variabel bertipe *String* bernilai **"tileset_bmp"** yang merupakan nama *Class (linkage)* dari gambar *tileset* yang telah dilakukan pada langkah no 5.

5.5 Menambahkan Pintu dan Item ke dalam Arena Permainan

Salah satu fitur dari arena permainan game RPG adalah pintu dan item permainan. Pintu digunakan untuk mengakses area lain, seperti memasuki ruangan tertentu atau pindah ke arena yang lain, sedangkan item merupakan barang-barang tersembunyi maupun terlihat, yang dapat diakses oleh pemain. Map editor memfasilitasi dua fitur tersebut, sebagai contoh perhatikan langkah berikut :

1. Buka aplikasi map editor, dan buka file **map-2.as**, dengan cara menekan tombol **open** dan memilih file.
2. Tekan opsi **edit walk mode** dan tekan tombol **door**. Pada panel **Door id**, ketikkan angka **3**. Hal ini berarti apabila karakter masuk ke pintu tersebut, karakter akan berpindah menuju ke peta **map-3.as**.



Gambar 5.18 menambahkan fitur pintu pada arena permainan

3. Klik salah satu pintu pada arena permainan, maka pintu tersebut akan ditandai dengan angka yang sesuai dengan **door id**. Untuk menambahkan id lain anda dapat menekan kembali tombol **door** dan memasukkan angka baru.



Gambar 5.19 Hasil dari penambahan door id.

Pintu akan memindah karakter ke arena baru yang sesuai dengan **door id**, sehingga dengan **door id** yang sama kurang direkomendasikan. Sedangkan

untuk menghapus **door id** anda dapat menggunakan **walkable tile** (icon sepatu).

4. Untuk menambahkan item ke arena permainan, tekan tombol **item** kemudian masukkan **item id** dan klik pada tile yang ingin ditambahkan item.



Gambar 5.20 Menambahkan item ke arena permainan

5. **Item id** dalam arena permainan nantinya akan digunakan sebagai identitas untuk pengaturan *database* item yang dapat diakses oleh pemain. Sebagai contoh pada gambar terdapat 3 id. Item id 1 merupakan penunjuk jalan, dimana nantinya jika diakses oleh pemain, maka akan muncul popup penunjuk jalan. Item id 2 merupakan keranjang berisi uang, dimana jika diakses oleh pemain, maka akan didapatkan uang yang tersembunyi. Item id 3 merupakan guci yang berisi uang dan ketika telah diambil akan muncul keterangan “tidak ditemukan apapun”.



Gambar 5.21 Contoh peletakan item id pada arena permainan.

Perencanaan item id ini nantinya dapat berubah-ubah seiring dengan berkembangnya peta dan konsep permainan, sehingga anda dapat fleksibel menggantinya di lain waktu.

Pada tahapan ini belum dijelaskan tentang teknik penambahan karakter baik itu karakter utama maupun NPC (*Non Playable Character*), karena akan dijelaskan secara khusus pada bab selanjutnya.

BAB VI

KARAKTER DALAM GAME RPG

Role Play berarti bermain peran. Para pemain diajak untuk memerankan tokoh/karakter tertentu. Karakter menjadi bagian vital karena merupakan representasi dari pemain, sehingga konsep pembentukannya harus dilakukan dengan serius.

KARAKTER DALAM GAME RPG

6.1 Pembentukan Karakter dalam Game RPG

Karakter dalam RPG merupakan sebuah konsep yang merujuk pada suatu cerita. Karakter dalam game dianggap sebagai objek aktif (hidup), sehingga proses pembentukannya tidak terbatas pada konsep visual saja, namun terdapat konsep penting lainnya yang harus dikonstruksi seperti identitas, karakteristik atau perwatakan, keberadaan waktu dan peristiwa. Sebuah game memiliki aturan dan batasan-batasan yang membawa pemain masuk ke dalamnya, dalam artian pemain menjadi aktor utama dalam sebuah permainan. Pemain memiliki atau diberi kebebasan dalam menentukan peran dan jalan cerita yang ingin dilalui meskipun program akan memberikan kondisi dan batasan tertentu.

Merujuk buku *Genre and the Video Game* karya Mark J.P. Wolf (2000), empat komponen yang harus diperhatikan dalam memahami game RPG, yaitu:

1. *Establishment*

meliputi narasi/jalan cerita, ikon dan konflik yang dialami oleh karakter dalam game. Sebagai contoh game RPG Simulasi Pertanian Astro Farmer memiliki konsep cerita seorang petani yang berusaha menyelamatkan lahan pertaniannya dengan pertanian sistem organik. Yang menarik, pendekatan karakter yang digunakan adalah seorang astronot di planet tertentu. Astronot dan bertani adalah sebuah hal yang relatif berbeda, namun di dalam game dua hal tersebut dijadikan sebuah konsep logis, sehingga pemain diposisikan sebagai petani dengan sifat-sifat visual seperti astronot.



Game RPG Simulasi Pertanian Astro Farmer dengan karakter Astronot Petani
(Copyright FroyoGames, 2014)

2. Animation

Merupakan kumpulan satuan karakter dan perilaku dalam game yang bisa dilakukan oleh sebuah karakter. Secara teknis disebut sebagai *character sprites*, dimana semua kemampuan karakter ditampilkan dalam bentuk visual 2D atau 3D dengan teknik animasi. Animasi dasar yang secara umum digunakan adalah animasi berjalan, maka secara visual dalam format 2 dimensi karakter akan digambarkan secara *frame by frame* berjalan ke berbagai arah. Seiring dengan kemampuan lain, maka *character sprites* akan semakin kompleks.



Gambar 6.2 *Character Sprites* Link dalam Game Zelda
(Copyright Nintendo, 1992)

3. Intensification

Merupakan irama/intensitas konflik yang hadir antar tokoh. Dalam game Ace Gangster (Froyogames, 2012) karakter diceritakan merupakan seorang kriminal yang baru datang ke kota tertentu, dan berusaha mendapatkan pengakuan dari kriminal lain yang ada disana. Konflik muncul dalam bentuk cerita dan misi yang harus diselesaikan oleh pemain.



Gambar 6.3 Karakter dalam Ace Gangster (Copyright Froyogames, 2012)

4. *Resolution*

Merupakan jalan keluar dalam menyelesaikan masalah dan *reward* yang didapat setelah masalah itu terselesaikan. Di dalam game RPG pengembangan karakter sering kali bergantung dengan hal ini. Level pemain, atau pengalaman pemain biasanya akan naik seiring penyelesaian satu masalah tertentu. Seperti ada game Astro Farmer, misi akan diberikan secara bertahap dan pengalaman pemain akan meningkat ketika menyelesaikannya. Dengan level yang semakin meningkat, maka pemain dapat menanam tanaman yang lebih sulit untuk ditanam.



Gambar 6.4 Pemberian misi dan *reward* pada Astro Farmer.

Dalam menentukan penokohan karakter game RPG terdapat beberapa variabel yang mempengaruhi pembentukan karakter, yaitu :

1. Tema dan Narasi/ Jalan Cerita (*referenciality*)
Pembentukan karakter akan terpengaruh oleh konstruksi budaya, seting lingkungan, dan sejarah.
2. Perwujudan Visual (*canocity and breach*)
Merepresentasikan struktur tema, pengarakteran, motivasi, dan ekspresi.
3. Sifat Dasar (*idem-identity*)
Merupakan karakteristik yang dilekatkan ke dalam karakter yang dibentuk, seperti jenis kelamin, warna kulit, gaya rambut, dan sebagainya.
4. Spesifikasi Keahlian (*ipse-identity*)
Merupakan keahlian dasar yang dimiliki oleh karakter. Dalam beberapa game RPG dapat ditemui keahlian yang bercabang, dimana satu karakter dapat berkembang menjadi karakter lain yang memiliki kemampuan lebih baik.

Sebagai contoh, dalam mengembangkan karakter dalam game **My New School**, 4 karakter yang dikembangkan memiliki tema anak pramuka (*referenciality*) dengan 4 visualisasi yang berbeda yaitu anak laki-laki pemberani, anak gemuk yang senang makan, anak perempuan yang cerdas dan solutif, serta anak laki-laki yang pandai tentang ilmu alam (*canocity, breach, idem-identity* dan *ipse-identity*).

Dalam konsep perancangan karakter game My New School, diawali dengan penjelasan deskriptif, pengumpulan referensi visual, sketsa kasar (*rough sketch*), sketsa final (*final sketch*), pewarnaan (*coloring*) dan pembuatan *sprite* animasi.



Gambar 6.5 Referensi gambar anak pramuka
(Nadia F, <https://rencanamu.id/>, 2017)



Gambar 6.6 Sketsa kasar karakter



Gambar 6.7 Sketsa final karakter



Gambar 6.8 Hasil digitalisasi dan pewarnaan karakter



Gambar 6.9 *Sprite* Karakter dengan animasi berjalan

Flash Tips

Teknik pembuatan grafis sangat beragam dan tergantung kemampuan masing-masing pengembang. Dalam contoh di atas digunakan aplikasi Photoshop dan Autodesk 3D studio Max untuk membuat dan menganimasikan gambar.

6.2 Karakter dalam Game RPG sebagai Agen Pedagogis

Agen pedagogis merupakan konsep yang dipinjam dari ilmu komputer dan kecerdasan buatan (AI) yang diterapkan untuk pendidikan. Agen pedagogis berupa antarmuka yang biasanya dipresentasikan secara visual mirip dengan manusia dan mensimulasikan hubungan antara pelajar dan konten edukasi, dalam lingkungan pendidikan. Agen pedagogis dirancang untuk memodelkan jenis interaksi antara siswa dan orang lain atau digambarkan sebagai karakter yang berinteraksi dengan pengguna (pemain game) dengan cara yang menarik.

Agen pedagogis secara umum divisualisasikan dalam bentuk avatar (perwakilan sebuah karakter), yang menyampaikan pesan-pesan edukasi tertentu, dibuat semenarik mungkin agar dapat berinteraksi dengan pengguna (dalam hal ini adalah pemain). Berbagai penelitian telah membuktikan bahwa keberadaan agen pedagogis sangat efektif dalam meningkatkan daya serap seorang pengguna multimedia pembelajaran interaktif. Hal inilah yang selanjutnya dapat dijadikan landasan bahwa karakter dalam game RPG bisa diposisikan perannya sebagai agen pedagogis.

Dalam aplikasi pembelajaran agen-agen pedagogis sering dirancang untuk memperoleh dan merespons tindakan afektif dari pengguna melalui berbagai modalitas seperti bicara, ekspresi wajah, dan gerakan tubuh. Lain halnya dalam game RPG, respon afektif seperti pada aplikasi interaktif masih sulit untuk didapatkan, maka peranan agen pedagogis dalam hal ini adalah menyampaikan konten-konten edukasi secara tersirat dalam sebuah cerita atau misi yang disampaikan kepada karakter utama (pemain). Sebagai contoh dalam game Astro Farmer, konsep pertanian organik disampaikan secara tersirat mulai dari cara menanam yang tidak boleh menggunakan pupuk kimia, cara mengatasi hama tikus dengan pemangsa alami, cara memahami cuaca dan jenis tanaman sesuai cuaca dan sebagainya. Materi-materi edukatif tersebut disampaikan dalam waktu-waktu tertentu sesuai dengan jalan cerita yang ada di dalam game.



Gambar 6.10 Contoh agen pedagogis dalam game Astro Farmer

Pada game *My New School*, agen pedagogis divisualisasikan dalam bentuk teman-teman dari karakter utama dan guru. Guru dalam hal ini memberikan materi edukasi dalam bentuk ucapan verbal yang didukung dengan teks penjelas. *Avatar* ditampilkan dalam bentuk animasi dengan gerakan tangan dan mulut.



Gambar 6.11 Agen pedagogis dalam game *My New School*

6.3 Menambahkan Karakter ke Arena Permainan

Pada tutorial 8, arena permainan berhasil dibuat. Tahapan selanjutnya adalah menambahkan gerakan karakter utama ke dalam arena permainan tersebut. Untuk lebih jelasnya, ikuti langkah-langkah berikut:

1. Buka file tutorial 8, kemudian **save as** dengan nama lain (**tutorial-9 fla**). Maksud dari langkah ini adalah untuk *membackup* file sekaligus menunjukkan riwayat dari pembuatan game.
2. Siapkan sebuah *sprites* karakter yang terdiri dari animasi gerakan berjalan karakter. Dalam tutorial ini digunakan *sprite* yang berukuran sama dengan ukuran *tile*, yaitu 32 x 32 *pixel* dan 3 x 4 *grid*, sehingga total ukuran dari *sprites* adalah 96 x 128 *pixel*. *Sprite* ditata sedemikian rupa sehingga membentuk baris animasi berjalan menghadap ke bawah (depan), ke kiri, ke kanan dan ke atas (belakang). *Sprites* semacam ini sangat umum digunakan dalam *software* pembuat game RPG seperti RPG Maker. Simpan file dengan nama **karakter_1.png**



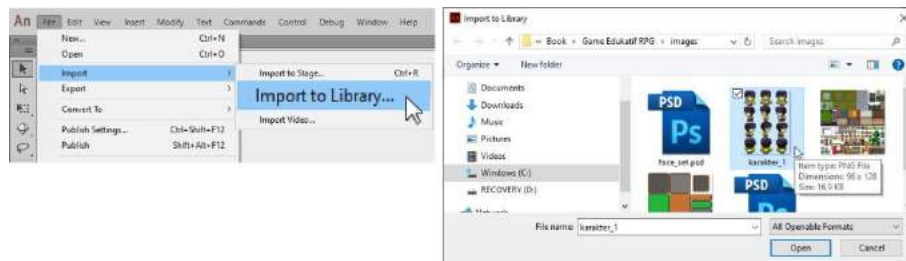
Gambar 6.12 *Sprites* karakter yang akan ditambahkan

Flash Tips

Apabila anda mengalami kesulitan dalam pembuatan karakter, anda dapat menggunakan *sprites* yang terdapat pada file tutorial, atau mendownload *sprites* gratis yang tersedia di beberapa situs pengembangan game.

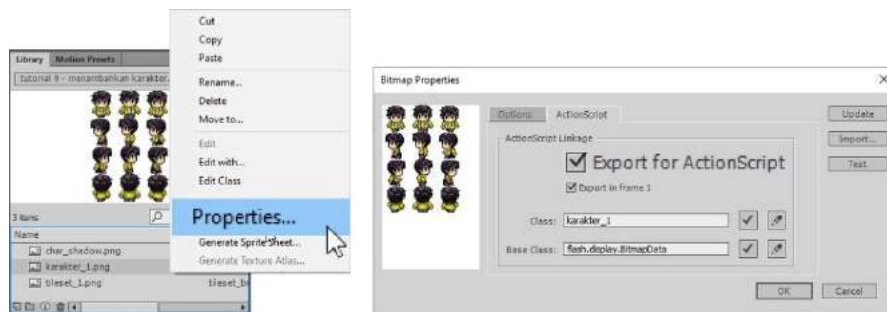
Format file yang paling umum adalah PNG atau GIF karena memiliki fitur transparansi, sehingga *background* nantinya dapat terlihat. File dapat dibuat secara terpisah (1 gambar demi 1 gambar), akan tetapi menyatukan beberapa gambar menjadi satu kesatuan *sprites* akan lebih efektif dalam proses pengerjaan.

3. Selanjutnya import karakter *sprites* tersebut ke **library**, dengan memilih opsi **File > Import to Library**.



Gambar 6.13 Mengimport karakter *sprites* ke *library*

4. Buka panel **Library (Ctrl+L)**, kemudian klik kanan gambar karakter_1.png yang baru saja diimport dan pilih opsi **properties**. Pada panel **advanced** atau panel **Actionscript**, klik opsi **Export for Actionscript** agar gambar dapat dikenali oleh kode.



Gambar 6.14 Menambahkan *linkage* pada gambar.

5. Selanjutnya, untuk membuat karakter terlihat lebih realistis, siapkan gambar bayangan yaitu sebuah lingkaran hitam dengan transparansi 40%. Simpan dengan nama **char_shadow.png**. Import gambar bayangan tersebut ke **library** dan berikan **linkage** (seperti pada langkah 4).
6. Agar aplikasi berjalan, klik **frame 1 layer 1** kemudian buka panel **Action**, dan tambahkan kode menjadi sebagai berikut :

```

1. include "map-2a.as";
2. include "flashRPG.as";
3.
4. buat_level(map_2, "tileset_bmp");
5. char = set_karakter("karakter_1", 18, 7);
6. jalankan_game();

```

7. Jalankan aplikasi dengan menekan **Ctrl+Enter**. Klik pada aplikasi dan tekan tombol panah untuk menggerakkan karakter.



Gambar 6.15 Hasil penambahan karakter

Penjelasan Program

```

5. char = set_karakter("karakter_1", 18, 7);
6. jalankan_game();

```

Pada baris 5, digunakan variabel **char** untuk mendefinisikan karakter utama dalam game. Variabel ini bertipe **movieclip** dan merupakan hasil dari fungsi **set_karakter**. Pada fungsi tersebut terdapat variabel **"karakter_1"** yang merupakan nama *linkage* dari karakter *sprites* yang telah diimport pada langkah no 3.

Angka **18, 7** merupakan kordinat awal tempat keluarnya karakter di arena permainan, yang berarti karakter akan muncul di kolom ke 18, dan baris ke 7. Angka ini dapat dengan mudah dilihat di **map editor**.

Pada baris 6, dijalankan fungsi **jalankan_game()**; yang berisi seperangkat fungsi untuk menggerakkan karakter.

BAB VII

PERPINDAHAN AREA DALAM RPG

Arena permainan dalam game RPG pada umumnya bervariasi. Karakter dapat bergerak dari satu arena ke arena yang lain. Berarti, perencanaan terhadap arena permainan dalam game sangat diperlukan.

PERPINDAHAN AREA PERMAINAN

7.1 Desain Arena Permainan

Dalam sebuah game RPG sering kali melibatkan beberapa arena permainan. Pemain dapat berpindah dari satu tempat ke tempat yang lain, dan dapat memasuki area baru seperti memasuki ruangan tertentu, membuka peta baru yang lebih luas, membuka area rahasia dan sebagainya. Dalam hal ini area atau peta permainan sangat terkait dengan plot cerita yang ada di dalam game.

Dalam tutorial di Bab 5 telah dijelaskan teknik pembuatan arena atau peta permainan menggunakan aplikasi map editor. Pada peta yang telah dibuat sebelumnya (**map-2**), terdapat 3 buah rumah dan satu jalan ke arah kanan. Telah dijelaskan pula terdapat beberapa peletakan id pintu (**door_id**) dan id item (**item_id**). Peletakan pintu dan id-nya akan dijadikan acuan dalam perpindahan ruang/area permainan.



Gambar 7.1 Perencanaan peta dengan door_id dan item_id

Dalam proses perancangan area permainan, perlu diperhatikan faktor-faktor utama yang akan mempengaruhi jalannya permainan, yaitu :

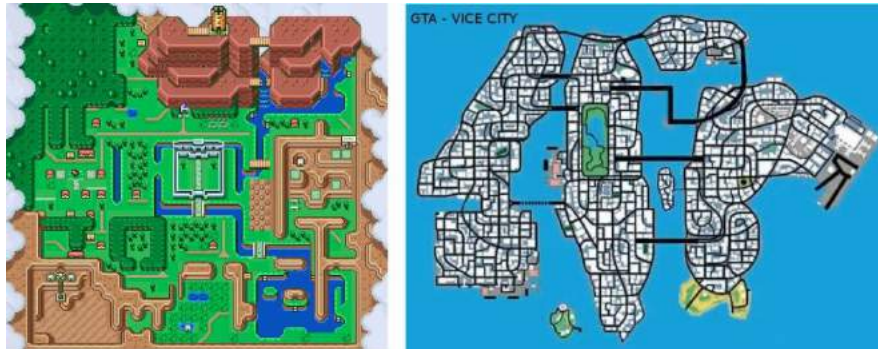
1. Plot cerita

Cerita memegang peranan utama dalam desain arena permainan. Tema cerita akan menentukan tampilan visual dari arena, sekaligus properti yang berada di lingkungan area tersebut. Sebagai contoh, pada arena yang dibuat ditahapan sebelumnya, digunakan *tileset* dengan properti bertema medieval. Jalan setapak, dinding bangunan, dan barang-barang pelengkap didesain sedemikian rupa mengikuti tema medieval.

Plot cerita juga menentukan kemana karakter akan bergerak, sehingga beberapa arena permainan harus direncanakan struktur dan pemetaannya.

Pada umumnya di awal permainan, area yang dapat diakses hanya area tertentu saja, namun seiring waktu area yang dapat diakses semakin meluas.

Plot cerita juga menentukan seberapa luas area yang dibutuhkan. Sebuah game *open world modern* seperti Grand Theft Auto, memiliki luas area sebesar kota dalam artian yang sebenarnya (skala 1:1). Namun ada juga game RPG yang membutuhkan beberapa area saja, karena cerita yang dibangun tidak menuntut area yang luas.



Gambar 7.2 Contoh ukuran peta
Game Zelda Link to the Past (Copyright Nintendo, 1991)
dan GTA Vice City (Copyright Rockstar Games, 2002)

2. Kebutuhan karakter

Sebuah arena permainan harus memfasilitasi kebutuhan karakter. Karakter dalam game RPG akan mengalami perkembangan baik itu kemampuan maupun level, sehingga waktu dan penempatan karakter dalam arena permainan harus terencana dengan baik.

Dalam contoh peta (**map-2**) yang digunakan di dalam tutorial misalnya, terdapat 4 bangunan yang terdiri dari *bar* (tempat makan), *inn* (tempat istirahat), *item shop* (tempat jual beli barang) dan sebuah rumah tua. 4 Bangunan ini memfasilitasi kebutuhan karakter dalam hal pemulihan energi, pemberian misi atau tugas. Selain itu dalam peta juga ditampilkan *door_id* 7, dimana akan dibuat sebuah area yang luas untuk meletakkan karakter NPC musuh.

Contoh lain dalam game Pokemon Fire Red (Copyright GameFreak, Nintendo, 2004), salah satu game RPG populer pada platform GBA, menggunakan arena permainan yang sangat terencana dan memfasilitasi perkembangan karakter. Pada awal cerita, arena hanya menampilkan sedikit bangunan dan sebuah lab, selanjutnya terdapat ruang terbuka dimana pemain dapat mencari pokemon,

dan pada tahapan selanjutnya ketika pemain sudah memiliki beberapa pokemon (secara *default* setelah beberapa menit permainan, pemain akan mulai mengkoleksi pokemon lewat beberapa pertempuran), maka arena akan dilengkapi dengan toko peralatan, pokemon center, dan *gym*.



Gambar 7.3 Contoh peta yang memfasilitasi perkembangan karakter pada game Pokemon FireRed (Copyright Nintendo, 2004)

Arena dengan bangunan pendukung karakter(kiri)

Arena terbuka untuk berburu pokemon dan melakukan *battle* dengan NPC (kanan)

3. Penempatan NPC

NPC atau *Non Playable Character* dalam peta akan melengkapi arena permainan. Merekalah yang nantinya berbicara, memberikan informasi, menjual belikan perlengkapan, memberikan misi atau hanya sebagai pemanis tampilan. Maka, peletakan NPC pada arena permainan juga harus terencana sedemikian rupa untuk mendukung jalannya permainan.



Gambar 7.4 Peletakan NPC pada game Shining Force 2

(Copyright SEGA, 1993)

4. Penempatan Item

Seperti halnya NPC, item-item tertentu diletakkan pada objek-objek yang berada pada arena permainan. Sering kali item-item ini “disembunyikan” agar menjadi rahasia yang harus diungkap oleh pemain. Bahkan seringkali sebuah game RPG populer akan memiliki banyak pemain yang menuliskan petunjuk permainan (*walkthrough*) yang di dalamnya terdapat lokasi item-item yang dapat ditemukan dalam permainan.



Gambar 7.6 Peletakan item pada game Final Fantasy 2
(Copyright Square, Nintendo, 1991)

Perencanaan peta atau arena permainan, berikut item-item yang akan ditambahkan ke dalamnya, akan mempengaruhi konten dari *tileset*. Pada peta **map-2** yang dikembangkan di Bab 5, *tile* pada *tileset* yang digunakan merupakan *tile* yang sifatnya *outdoor* seperti jalan setapak, rumput, pohon, dinding dan atap bangunan. Ketika akan membuat peta atau arena untuk interior (dalam ruang), maka akan dibutuhkan variasi item seperti meja, kursi, lemari, kotak harta karun, rak dan sebagainya. Dalam contoh *tileset* yang digunakan untuk membuat peta di Bab 5, telah dilengkapi dengan beberapa *tile* item yang dimaksud.

Flash Tips

Tutorial dalam buku ini, memfasilitasi pembangunan peta permainan dengan menggunakan satu *tileset* saja untuk mempermudah proses penjelasan. Anda dapat menambahkan fitur penggunaan *tileset* yang unik untuk tiap-tiap map dengan memahami dan mengedit fungsi-fungsi yang ada dalam file **flashRPG.as**

7.2 Mengembangkan Peta Permainan

Dalam tutorial pengembangan peta yang ada pada bab 5, terdapat beberapa peta tambahan yang perlu dibuat. Setidaknya terdapat 4 interior bangunan dan 1 area tambahan jika pemain menelusuri jalan ke arah kanan. 4 interior dan 1 arena ini perlu dibuat melalui map editor, dengan beberapa perencanaan. Anda dapat memberikan variasi ukuran peta dan variasi *tile-tile* yang menyusun peta tersebut.

Kita mulai dengan interior bangunan yang pertama atau **door_id 3**. Bangunan pertama merupakan penginapan (*inn*), yang nantinya digunakan untuk memulihkan energi pemain. Secara sederhana bisa divisualisasikan dengan interior yang menampilkan tempat resepsionis untuk NPC, dan beberapa kamar tidur. Ukuran peta

yang dibuat juga tidak perlu luas, mengingat ruangan yang dibutuhkan hanya sebagai simbol penginapan saja, sehingga kita tidak perlu menampilkan 10 kamar tidur misalnya, cukup dengan yang mewakili saja.



Gambar 7.7 Perencanaan peta permainan untuk **door_id 3** (penginapan)

Untuk mewujudkan interior tersebut, bukalah aplikasi map editor dan buatlah file baru dengan identitas **map id** = 3 dan ukuran file 14 x 16 *tile*. Hal terpenting disini adalah anda memberikan **id** peta sesuai dengan id pintu (**door_id**), hal ini dimaksudkan agar map editor nantinya membuat *database* peta dengan nama variabel **map_3** yang secara otomatis akan dideteksi oleh kode saat permainan berjalan.



Gambar 7.8 Membuat peta baru dengan Map ID 3

Selanjutnya buatlah peta interior sesuai konsep di atas, perhatikan area yang dapat diakses dan area yang tidak. Tempatkan juga beberapa item, dengan item id yang direncanakan (misalnya item id = 3). Poin terpenting selanjutnya adalah meletakkan **door_id = 0** pada pintu ruangan. Door id yang bernilai 0 akan mengembalikan karakter ke lokasi peta sebelumnya. Dalam tutorial di buku ini misalnya, permainan dimulai dengan peletakan karakter di **map-2**, setelah pemain masuk ke pintu **door 3**, maka arena akan berpindah ke **map-3**, dan jika keluar dari ruang akan kembali ke **map-2**.

Secara visual di map editor, peta interior untuk **door id 3**, dapat dilihat pada gambar berikut :



Gambar 7.9 Pengembangan interior inn (**map-3**) pada map editor.

Flash Tips

Pada map editor di atas, di bagian bawah terdapat satu baris kosong dan hanya berisi *door tile*. Hal ini dimaksudkan agar pemain dapat melewati celah pintu ketika masuk atau keluar, sehingga terlihat lebih natural. Baris kosong ini nantinya tidak akan terlihat di dalam permainan.

Setelah peta tersebut jadi, simpan dengan nama **map-3.as** dan jika anda buka file tersebut akan terlihat struktur *array* sebagai berikut :

```
var map_3:Array = [[ [1,17,144], [1,17,160], ..... , [0,-1,0] ]];
```

Tutorial 10

7.3 Menguji Coba Perpindahan Area Permainan

Setelah peta selesai dibuat, peta perlu diuji coba untuk mengetahui apakah peta bekerja dengan baik atau tidak. Ikuti langkah-langkah berikut:

1. Buka file **tutorial 9**, kemudian save as dengan nama lain (**tutorial-10 fla**).
2. Klik **frame 1 layer 1** kemudian buka panel **Action**, dan tambahkan kode menjadi sebagai berikut :

```
1. include "map-2.as";
2. include "map-3.as";
3. include "flashRPG.as";
```

```

4.
5. buat_level(map_2, "tileset_bmp");
6. char = set_karakter("karakter_1", 18, 7);
7. jalankan_game();

```

3. Jalankan aplikasi dengan menekan **Ctrl+Enter**. Klik pada aplikasi dan tekan tombol panah untuk menggerakkan karakter, kemudian berjalanlah mengarah ke pintu *inn* dan tekan **Spasi** untuk memasuki ruangan tersebut.

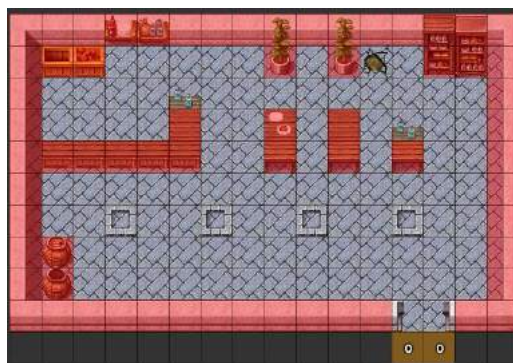


Gambar 7.10 Hasil tutorial 10

Pada tutorial tersebut, terdapat efek *wipe-in* dan *wipe-out* (lingkaran mengecil dan membesar), yang merupakan efek tambahan untuk memberikan kesan pergantian ruang. Pada tahapan ini, akan muncul *error* ketika anda mencoba memasuki pintu yang lain, hal ini dikarenakan aplikasi tidak menemukan variabel *data base* peta yang belum dibuat.

7.4 Pengembangan peta lebih lanjut

Setelah berhasil dengan penambahan interior bangunan pertama, selanjutnya kita dapat mengembangkan peta untuk bangunan lain dan untuk area lain. Prinsip yang dikembangkan tetap sama seperti pada tahapan sebelumnya. Dalam hal ini peta berikutnya yang dikembangkan adalah peta ke 4 (**map-4**), yaitu interior bangunan restoran / bar. Di tempat ini nantinya pemain bisa mendapatkan informasi dari NPC.



Gambar 7.11 Pengembangan peta **map-4**

Peta selanjutnya adalah peta ke 5 (**map-5**) yang merupakan tempat membeli perlengkapan karakter. Pada tahap berikutnya akan diletakkan sebuah NPC yang bertugas sebagai pedagang item. Pada bangunan di bawah peta, yang secara konsep adalah bangunan terbengkalai, dibuatkan peta ke 6 (**map-6**), dimana di dalamnya hanya terdapat beberapa item.



Gambar 7.12 Desain peta untuk **map-5** (kiri) dan **map-6** (kanan).

Sedangkan untuk **map-7**, yaitu arena di sebelah kanan peta utama, dibuat sebuah arena terbuka yang relatif luas (32x20 *tile*), yang akan digunakan sebagai arena untuk menemukan NPC musuh. Pada area ini diberikan ruangan yang cukup luas, sehingga pemain dapat mengeksplorasi dan menemukan musuh. NPC musuh juga dapat bergerak lebih leluasa di area yang relatif luas.



Gambar 7.13 *Open Area* pada map-7

Untuk menguji peta, buka kembali file tutorial-10, dan ubah kodenya menjadi sebagai berikut :

```
1. include "map-2.as";
2. include "map-3.as";
3. include "map-4.as";
4. include "map-5.as";
5. include "map-6.as";
6. include "map-7.as";
7. include "flashRPG.as";
8.
9. buat_level(map_2, "tileset_bmp");
10. char = set_karakter("karakter_1", 18, 7);
11. jalankan_game();
```

Setiap penambahan peta baru, perlu dilakukan penambahan kode `include` agar *database* peta dapat dikenali oleh kode. Pada tahapan ini, permainan RPG sudah mulai terbentuk, dimana karakter pemain bisa bergerak mengeksplorasi lokasi yang ada di arena permainan. Pada tahapan selanjutnya, akan ditambahkan sistem dialog atau percakapan dan NPC.

BAB VIII

GUI DAN SISTEM DIALOG

Dalam RPG, cerita seringkali ditampilkan dalam bentuk dialog, selain itu juga perlu ditampilkan status pemain seperti energi, level, score dan sebagainya. Hal ini membutuhkan tampilan visual yang disebut dengan *Graphical User Interface (GUI)*

GUI DAN SISTEM DIALOG

8.1 *Graphical User Interface (GUI)* dalam game RPG

Sebelum menambahkan karakter pendukung (NPC), perlu direncanakan sebuah sistem percakapan yang secara umum digunakan untuk menyampaikan cerita, atau menunjukkan dialog antara karakter pemain dan karakter pendukung (NPC). Dialog biasanya ditampilkan dalam bentuk panel dengan kata-kata, tidak jarang pula ditampilkan *avatar* (representasi wajah) dari karakter yang sedang melakukan dialog. Panel-panel untuk menampilkan dialog secara visual merupakan salah satu dari *Graphical User Interface (GUI)*.

GUI merupakan antar muka pada sebuah aplikasi yang ditampilkan secara visual untuk mendukung interaksi antara pengguna dan aplikasi tersebut. Dalam RPG, GUI dapat ditemui pada bagian berikut :

1. *Game cover / tittle screen* (halaman pembuka), pada umumnya terdapat sebuah judul game, ilustrasi, dan seperangkat tombol untuk memulai, membuka atau mengatur permainan.



Gambar 8.1 *Tittle Screen* game Baldur's Gate (Copyright Beamdog, 1998)

2. *New Game Setup* (halaman awal permainan), pada umumnya game RPG akan menanyakan nama karakter dan beberapa properti lainnya. Pada halaman ini seringkali terdapat panel-panel visual yang tematik, sesuai dengan permainan yang akan dimunculkan.



Gambar 8.2 Seting awal game Pokemon Emerald (Copyright Gamefreak, 2004)

3. Panel *Heroes Properties* (properti karakter), digunakan untuk menampilkan statistik karakter yang ada dalam permainan, seperti kondisi kesehatan, level, dan perlengkapan yang digunakan oleh karakter.



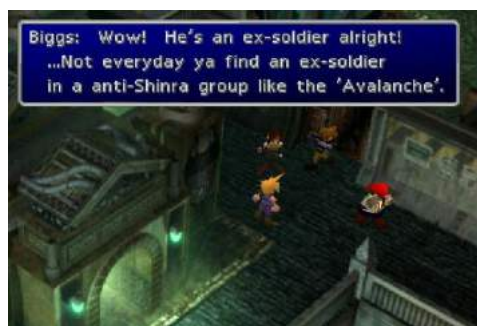
Gambar 8.3 Panel *heroes properties* game Shining Force
(Copyright Sega, 1993)

4. Panel *trading* (jual beli item), digunakan untuk menampilkan list barang yang dijual oleh NPC tertentu. Pada panel ini sering kali ditunjukkan ilustrasi item, deskripsi item, harga item dan statistik kegunaan yang dimiliki oleh item tersebut.



Gambar 8.4 Panel jual beli game Secrets of Grindea
(Copyright Pixel Ferrets, 2012)

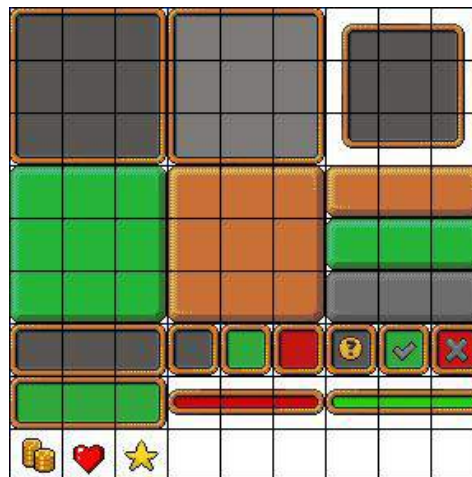
5. Panel dialog, digunakan untuk menampilkan dialog/percakapan antar karakter dalam permainan, atau menampilkan pesan-pesan tertentu yang membentuk jalannya cerita.



Gambar 8.5 Panel dialog dalam game FF7
(Copyright Square Enix, 1997)

Pengembangan GUI sebuah permainan tentunya membutuhkan desain yang baik serta komposisi yang tepat. Berbagai teknik dapat digunakan dalam mengembangkan GUI untuk game dengan Adobe Animate / Flash, seperti menggambar langsung dengan menggunakan *drawing tools* yang tersedia, mengimport grafik GUI siap pakai, atau mengatur GUI menggunakan kode.

Teknik pengaturan GUI yang digunakan dalam tutorial di buku ini adalah teknik pengaturan menggunakan kode, dimana sebuah grafik *GUI set* dijadikan acuan dalam membentuk panel-panel yang akan dimunculkan dalam game. *GUI set* yang dimaksud memiliki konsep yang sama dengan *tileset* dimana secara umum terdiri dari 3x3 grid dengan ukuran masing-masing *tile* sebesar 32 x 32 *pixel*. Adapun *GUI set* yang dimaksud memiliki tampilan sebagai berikut :



Gambar 8.6 *GUI set* yang digunakan dalam tutorial

Selain *GUI set*, dalam tutorial di buku ini juga digunakan *Face set*, yaitu kumpulan *avatar* yang ditampilkan dalam sistem GUI pada saat panel dialog muncul. *Face set* dapat dikembangkan menggunakan aplikasi *face generator* seperti pada aplikasi RPG Maker. *Face set* yang digunakan pada buku ini, masing-masing berukuran 64x64 *pixel*.

Flash Tips

Seperti halnya *Tileset*, pembuatan *GUI set* melibatkan kemampuan pengelolaan grafis. Bagi yang belum terbiasa dengan pengelolaan grafik, bisa memperoleh *UI set* melalui situs pengembangan game baik itu berbayar maupun gratis. Hal ini dapat menghemat waktu dan biaya pengembangan.

8.2 Membuat Panel GUI

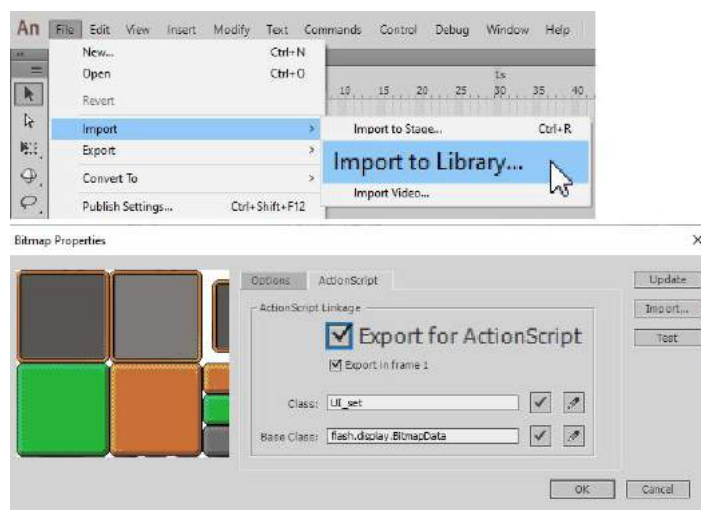
Pada dasarnya kode yang terdapat pada file **flashRPG.as** sudah memiliki fitur untuk mengatur GUI secara standar baik itu untuk dialog, *trading*, maupun UI pada main game seperti untuk menampilkan energi, level, score dan sebagainya. Namun demikian, anda perlu mengetahui konsep dasar dari sistem pengaturan GUI yang ada pada file tersebut, agar nantinya anda dapat melakukan pengeditan sendiri dan membangun GUI sesuai konsep anda sendiri. Untuk lebih memahami konsep GUI ikutilah langkah-langkah berikut:

1. Buatlah sebuah file baru. Simpan file satu folder dengan file **flashRPG.as**.
2. Siapkan file **UI set** dan **Face set**, anda dapat menyiapkannya sendiri atau meenggunakan file **UI_set.png** dan **Face_set.png** yang terdapat pada folder tutorial.



Gambar 8.7 File *UI_set* dan *Face_set*

3. *Import* gambar *UI_set* ke **Library** dengan memilih menu **File > Import to Library**. Selanjutnya buka panel **library** klik kanan gambar **UI_set** dan pilih opsi **properties**. Agar dapat dikenali oleh kode, centang opsi **Export for Actioscript** pada mode *Advanced* atau pada panel *Action*.



Gambar 8.8 Mengimport file UI Set

4. Dengan cara yang sama **import to library** file **Face_set.png**. Buka panel **library** klik kanan gambar **Face_set** dan pilih opsi **properties**. Agar dapat dikenali oleh kode, centang opsi **Export for Actionscript** pada mode *Advanced* atau pada panel *Action*.
5. Klik **frame 1** kemudian buka panel **action** dan ketikkan kode berikut :

```

1. import flash.display.MovieClip;
2. import flash.text.TextField;
3.
4. include "flashRPG.as";
5.
6. UIset_name = "UI_set";
7. faceset_name= "face_set";
8.
9. var bg_1:MovieClip = draw_panel(0, 0, 20, 4, "3x3", 1);
10. var bg_2:MovieClip = draw_panel(0, 0, 3, 3, "3x3", 3);
11. var ava:MovieClip = draw_from(faceset_name, 64, 0, 64, 64);
12. var teks_dialog:TextField =
    add_text("Haii.. ini uji coba panel dialog", 500, 100);
13.
14. addChild(bg_1);
15. addChild(bg_2);
16. addChild(ava);
17. ava.x = 16;
18. ava.y = 16;
19. addChild(teks_dialog);
20. teks_dialog.x = 100;
21. teks_dialog.y = 16;

```

6. Jalankan aplikasi, dan akan dihasilkan tampilan sebagai berikut :



Gambar 8.9 Hasil tutorial pengaturan GUI

Penjelasan Program

```

6. UIset_name = "UI_set";
7. faceset_name= "face_set";

```

Dua baris tersebut digunakan untuk mendefinisikan grafik *UI set* dan *faceset* yang akan digunakan dalam aplikasi. “**UI_set**” dan “**face_set**” merupakan nama **linkage** hasil dari langkah 3.

```
9. var bg_1:MovieClip = draw_panel(0, 0, 20, 4, "3x3", 1);
```

Fungsi **draw_panel** akan membuat sebuah panel antar muka pada kordinat (0,0), sebesar **20 x 4 tile** (1 tile = 32 *pixel*), menggunakan tipe grid “3x3” dan visualisasi grid yang digunakan adalah 9 grid no **1** (pojok kiri atas). Kode ini akan mengambil sampel gambar dari **UI_set** yang hanya 9 grid, kemudian menggambar dengan metode *tiling* dan *Bitmap data*. Proses ini dilakukan secara berulang, sehingga terbentuk panel yang memanjang.

Metode **draw_panel** akan menghasilkan Movieclip baru, sehingga dapat ditambahkan ke layar dengan metode **addChild** (baris 14).

```
10. var bg_2:MovieClip = draw_panel(0, 0, 3, 3, "3x3", 3);
```

Seperti halnya baris 9, baris di atas juga menggambar sebuah panel, bedanya panel yang dibuat lebih kecil dengan ukuran 3x3 *tile*, dan menggunakan visualisasi *grid* ke 3 (pojok kanan atas). Sehingga langkah ini menghasilkan sebuah *frame* untuk menempatkan *avatar*.

```
11. var ava:MovieClip = draw_from(faceset_name, 64, 0, 64, 64);
```

Fungsi **draw_from** digunakan untuk mengambil gambar sumber yaitu “**face_set**”. Pengambilan sampel gambar dimulai pada kordinat (64, 0) yaitu *avatar* ke 2, dan selebar **64x64 pixel**. Metode **draw_from** ini cukup efektif digunakan untuk membentuk gambar-gambar baru yang bersumber pada “**UI_set**” maupun gambar lain. Sebagai contoh pada kode berikut :

```
1. var mc:MovieClip = draw_from("UI_set", 32, 257, 32, 32);
2. addChild(mc);
3. mc.x = 80;
4. mc.y = 100;
5. var bg_hp:MovieClip = draw_panel(100, 100, 4, 1, "1x3", 6);
6. addChild(bg_hp);
```

Akan dihasilkan sebuah gambar sebagai berikut :



Gambar 8.10 Hasil metode draw_from.

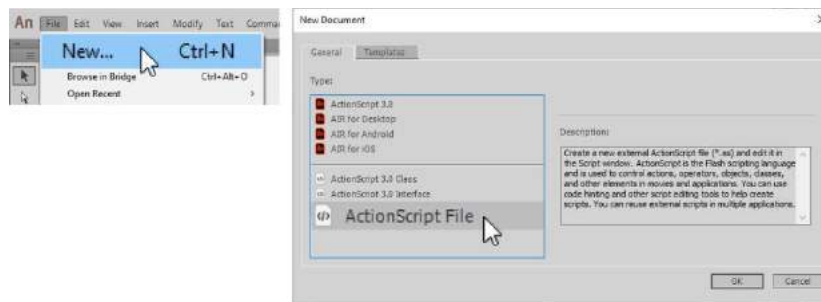
Penggunaan metode **draw_panel** dan **draw_from** menjadikan penyusunan GUI jauh lebih mudah untuk dilakukan, tanpa harus membuat aset secara satuan. Selain menghemat waktu, metode ini pada akhirnya juga menghemat ukuran file.

Tutorial 12

8.3 Membuat Sistem Dialog dalam Permainan

Elemen berikutnya yang terkait dengan GUI adalah sistem dialog. Sistem dialog akan digunakan oleh karakter utama maupun karakter pendukung untuk menampilkan jalannya cerita. Kode **flashRPG.as** telah memfasilitasi fitur dialog dengan menggunakan data bertipe *Array*. Untuk mempermudah pemahaman, ikuti langkah-langkah berikut :

1. Buatlah sebuah file baru, namun kali ini anda pilih opsi **Actionscript File**, karena kita akan membuat file baru khusus untuk kode. Simpan dengan nama **dialog-12.as**. (angka 12 merujuk angka pada tutorial ke-12).



Gambar 8.11 Membuat file bertipe *actionscript*

2. Pada panel **action** ketikkan kode dialog sebagai berikut :

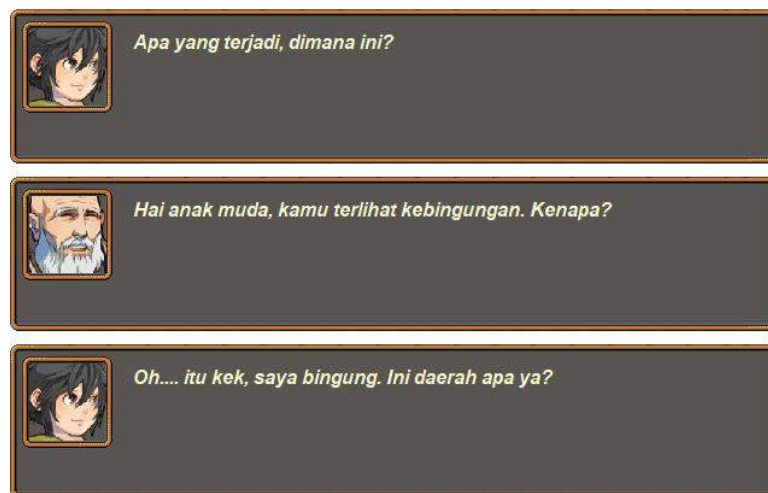
```
1. var cerita:Array = new Array();
2. cerita[0] = [0, "Apa yang terjadi, dimana ini?", "next"];
3. cerita[1] = [0, "Seingatku aku tertidur di bangku kelas.
   Kenapa tiba-tiba aku disini?", "next"];
4. cerita[2] = [0, "Lebih baik, aku bertanya ke seseorang",
   "next"];
5. cerita[3] = [3, "Hai anak muda, kamu terlihat kebingungan.
   Kenapa?", "next"];
6. cerita[4] = [0, "Oh.... itu kek, saya bingung. Ini daerah apa
   ya?", "next"];
7. cerita[5] = [3, "Ini Desa Ketpi, kamu berasal dari mana
   nak?", "next"];
8. cerita[6] = [0, "Saya dari Semarang.", "next"];
9. cerita[7] = [3, "Semarang... hmmm.. belum pernah kudengar,
   pasti negeri yang jauh", "next"];
10. cerita[8] = [0, "Nah itulah kek, kenapa saya bingung. Kok
    bisa sampai sini.", "next"];
11. cerita[9] = [3, "Cobalah kau pergi ke penginapan, siapa tahu
    ada yang bisa membantu", "close"];
```

3. Simpan file **dialog-12.as** tersebut.
4. Buka kembali file **tutorial-11**. Lalu **save as** dengan nama lain (**tutorial-12 fla**). Maksud dari langkah ini adalah agar kita tidak perlu mengimport ulang aset visual untuk UI dan Face set.
5. Klik **frame 1**, kemudian buka panel **action**, dan hapus semua kode yang ada pada panel tersebut.

6. Kemudian, ketikkan kode di bawah ini, untuk menggantikan kode yang telah kita hapus:

```
1. include "flashRPG.as";
2. include "dialog-12.as";
3.
4. UIset_name = "UI_set";
5. faceset_name= "face_set";
6.
7. setup_GUI();
8. tambah_dialog(cerita);
9.
10. stage.addEventListener(KeyboardEvent.KEY_DOWN, keyDownFunction);
11. stage.addEventListener(KeyboardEvent.KEY_UP, keyUpFunction);
```

7. Jalankan aplikasi, maka akan didapatkan panel dialog dengan animasi teks dan percakapan yang saling berganti sesuai dengan variabel **cerita**, yang telah dibuat pada langkah ke-2.



Gambar 8.12 Hasil sistem dialog

Penjelasan Program

```
2. include "dialog-12.as";
```

Baris ini digunakan untuk membuka file dialog yang dibuat secara terpisah. Teknik memisah file *actionscript* menjadi beberapa file *.as*, sebenarnya merupakan pilihan yang didasarkan atas kenyamanan saja. Memisah kode ke dalam beberapa file mempermudah proses pengembangan, karena secara spesifik kita dapat mengedit satu buah sistem tanpa mengganggu sistem lain. Telah dijelaskan pada bab sebelumnya, bahwa di buku ini tidak digunakan pemrograman dengan sistem *Class*, meskipun

demikian banyak programmer yang lebih menyukai sistem *Class* karena lebih terstruktur dan lebih berorientasi objek (OOP).

Dengan memisah file dialog, pada tahapan selanjutnya kita dapat secara spesifik membuat sebuah cerita dengan mengedit file tersebut, tanpa harus mengganggu kode yang lain.

```
7. setup_GUI();  
8. tambah_dialog(cerita);
```

Pada baris 7, fungsi **setup_GUI()**; secara spesifik akan membentuk tampilan untuk dialog, UI untuk menampilkan kondisi energi dari pemain dan pengalaman (*exp*) dari pemain. Pada fungsi **tambah_dialog(cerita)**; akan menjabarkan variabel *array* yaitu **cerita** dan menampilkannya dalam panel.

```
1. cerita[0] = [0, "Apa yang terjadi, dimana ini?", "next"];
```

Array cerita memiliki setidaknya 3 data. Data pertama (cerita[0][0]) merupakan nilai yang digunakan untuk menampilkan *avatar*. Nilai **0** akan mengaktifkan fungsi untuk menggambar *avatar* dari **face_set** untuk gambar *avatar* pertama (pojok kiri atas), sedangkan pada kode dialog baris ke 5, 7, 9 dan 11, *avatar* yang ditampilkan adalah *avatar* ke 4 yaitu gambar wajah seorang kakek.

Data kedua dari *array* cerita (cerita[0][1]) merupakan variabel bertipe *String* yaitu kata-kata yang akan dimunculkan pada panel dialog. Kata-kata ini akan ditampilkan secara bertahap huruf demi huruf, sampai seluruh kalimat selesai.

Data ketiga dari *array* cerita (cerita[0][2]) merupakan variabel bertipe *String* yang digunakan untuk menentukan langkah selanjutnya. Isian data dapat bernilai sebagai berikut :

1. “next” untuk melanjutkan dialog selanjutnya.
2. “close” untuk menutup sistem dialog ketika sudah selesai.
3. “trade” untuk membuka sistem jual beli.
4. “sleep” untuk membuka sistem istirahat.
5. “story” untuk merubah variabel jalannya cerita.

Pada tahapan selanjutnya akan ditunjukkan pemanfaatan data *array* di atas.

BAB IX

NON PLAYABLE CHARACTER (NPC)

Karakter pendukung atau NPC menjadi bagian penting dalam game RPG dalam hal pembentukan cerita. Penambahan karakter pendukung berikut penyusunan *database* dialog dan cerita menjadi poin utama dalam penambahan NPC.

NON PLAYABLE CHARACTER (NPC)

9.1 Konsep NPC atau Pemain Pendukung dalam RPG

Istilah *Non Playable Character* (NPC) menggambarkan semua karakter game yang tidak bisa dikendalikan oleh pemain game, dalam artian dikendalikan sepenuhnya oleh aplikasi. NPC berperan sebagai karakter yang membantu, membimbing, dan berkomunikasi dengan pemain untuk mencapai tujuan permainan. Dalam beberapa referensi, pembagian NPC dalam game meliputi :

1. *Villain* (NPC musuh)

Yaitu karakter musuh yang berada di dalam permainan, yang secara sistemik dapat menyerang karakter pemain. Ketika karakter musuh menyerang, pada umumnya permainan akan berubah ke mode *battle*. Pada game RPG bergenre simulasi, biasanya tidak menggunakan karakter bertipe *villain*, hanya saja akan dimunculkan karakter dengan sifat antagonis sebagai gantinya.



Gambar 9.1 Karakter *villain* dalam game FF7

(Copyright Square Enix, 1997)

2. *Services NPC*

Yaitu NPC yang memberikan pelayanan kepada karakter utama, seperti melakukan jual beli item dengan pemain, membuatkan artifak tertentu, membantu memulihkan energi dan memberikan jasa pelayanan dengan timbal balik variabel tertentu seperti *gold* atau uang.



Gambar 9.2 NPC pada game Golden Sun: Dark Down

(Copyright Camelot Software Planning, 2010)

3. *Story / Quest NPC*

Yaitu NPC yang akan menunjukkan informasi jalannya cerita. Memberikan latarbelakang atau konsep utama yang ada dalam permainan, menunjukkan arah atau petunjuk tertentu, dan sering kali menjadi kunci untuk melanjutkan cerita dalam permainan.



Gambar 9.3 NPC Quest dalam game RDR2
(Copyright Rockstar game, 2018)

4. *Ordinary NPC*

Yaitu NPC yang keberadaannya hanya untuk membuat arena permainan tampak aktif. Karakter ini akan bergerak atau diam dan dapat diajak berkomunikasi, namun tidak memberikan informasi kunci yang signifikan untuk perkembangan cerita permainan.



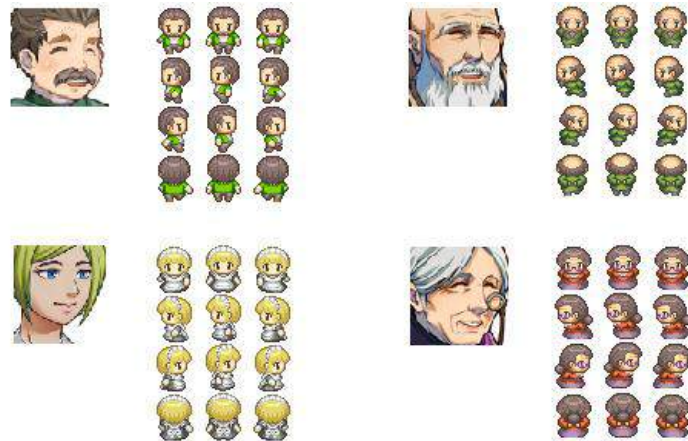
Gambar 9.4 *Ordinary NPC* dalam Game Pokemon Omega Ruby
(Copyright Game Freak, Nintendo, 2014)

9.2 Mengembangkan Grafis NPC (*NPC sprites*)

Seperti halnya grafis untuk karakter utama (karakter *sprites*), grafis untuk NPC juga dikembangkan dengan metode yang sama, yaitu dengan menggunakan teknik *sprites* dengan jumlah gambar 3 x 4 *tile*. Dalam kode **flashRPG.as** pengembangan karakter utama maupun karakter NPC akan dibuat secara otomatis, setelah grafis *sprites* dari karakter-karakter tersebut di *import* ke *library* dan diberikan *linkage*.

Hal teknis yang perlu diperhatikan dalam mengembangkan *NPC sprites* adalah keserasian gambar 1 dengan gambar yang lain. Sebagai contoh pada semua tutorial di buku ini digunakan grafis berjenis *pixel art*, dengan ukuran satu *tile* 32x32 *pixel*.

Karakteristik gambar karakter memiliki *outline* hitam, dengan sedikit perubahan proporsi tubuh. Maka, NPC harus menyesuaikan gaya tersebut, sekaligus menyesuaikan grafis *avatar face set*. Perhatikan contoh gambar NPC *sprites* berikut :



Gambar 9.5 Contoh NPC *sprites* yang disesuaikan dengan *avatar* NPC

Dalam membuat *sprites* tersebut, perlu diperhatikan bahwa masing-masing gambar harus tetap berada dalam *tile grid*. Hal ini dimaksudkan agar animasi yang terbentuk nantinya tidak mengalami distorsi. Perhatikan gambar di bawah ini, untuk memahami peletakan grafis karakter pada *grid*.



Gambar 9.6 Posisi grafis pada *tilegrid*.

Pada gambar di atas, posisi kaki melangkah di letakkan di *grid* kanan dan kiri, sedangkan pada *grid* tengah karakter berada pada posisi normal. Selain itu, anda juga dapat melihat bahwa pada saat melangkahkan kaki ke depan, karakter akan turun sekitar 1-2 *pixel*, untuk menghasilkan animasi melangkah yang logis dan realis. Dengan teknik ini animasi yang akan dibentuk oleh kode hanya perlu membutuhkan 3 grafis saja. Dalam aplikasi lain, seperti RPG Maker dan aplikasi lainnya, sering kali ditemukan *sprites* yang menggunakan 4 gambar atau lebih untuk menghasilkan animasi melangkah.

Untuk mempermudah proses selanjutnya, simpan *sprites* dengan nama yang berurutan seperti **npc_sprites_1.png**, **npc_sprites_2.png**, dan seterusnya.

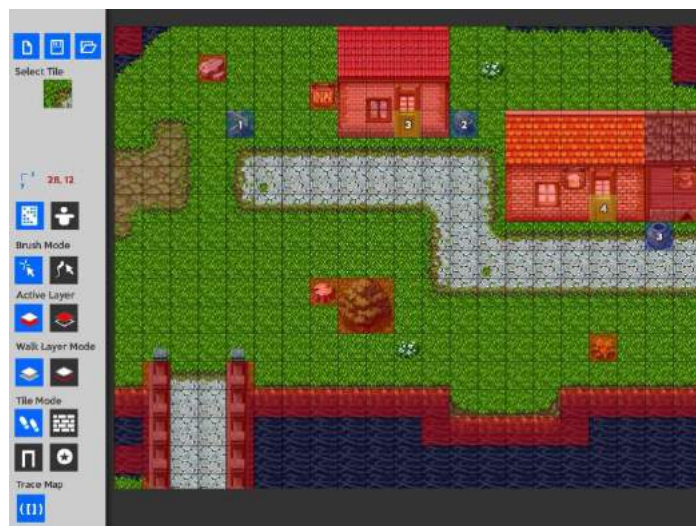
Flash Tips

Dalam beberapa situs pengembangan game, seringkali ditemukan satu paket *tileset* dan karakter *sprites* yang siap pakai baik gratis maupun berbayar. Satu paket yang dimaksud, akan menjamin keserasian grafis dari segi penggayaan, tema dan warna.

9.3 Menambahkan *Ordinary NPC* pada Arena Permainan

Setelah melalui tahapan pengembangan *sprites* untuk NPC, selanjutnya adalah menambahkan NPC tersebut ke arena permainan menggunakan **Map Editor**. Untuk melakukannya, perhatikan langkah berikut :

1. Buka aplikasi **map editor**, lalu buka peta **map-2.as**.



Gambar 9.7 Mengedit map-2 dengan map editor

2. Klik tombol **Character Mode**, kemudian klik tombol **NPC**. Pada panel NPC, ketikkan **NPC id = 1** dan **dialog id = 1**, lalu tekan **OK**.



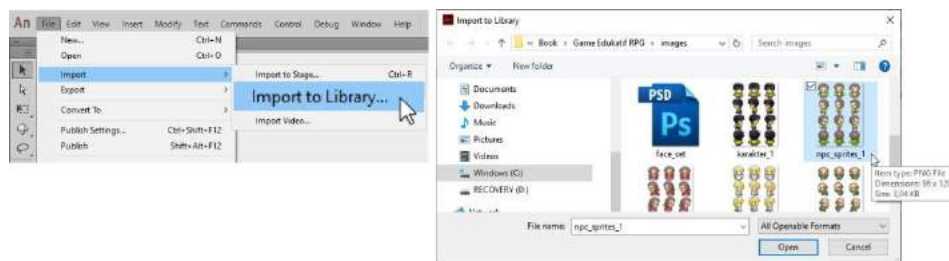
Gambar 9.8 Mengatur penambahan NPC

3. Klik di kordinat **21, 12** (lihat pada gambar), untuk menambahkan NPC ke arena permainan.



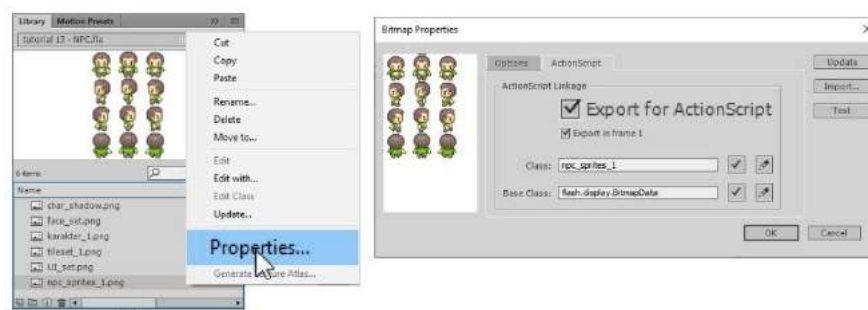
Gambar 9.9 Posisi peletakan NPC pada arena permainan.

4. Simpan kembali peta dengan nama lain (**map-2c.as**), hal ini dimaksudkan agar kita memiliki *backup* agar jika terjadi kesalahan, kita tidak harus mengulang semuanya.
5. Buka aplikasi Adobe Animate/Flash, dan buka file **tutorial-10** tentang perpindahan ruang / arena permainan. **Save as** dengan nama lain (**tutorial-13 fla**).
6. Siapkan file untuk *sprites* NPC pertama, yaitu file **npc_sprites_1.png** yang ada pada file tutorial (yang sudah anda siapkan sendiri sebelumnya). Kemudian import ke *library* dengan memilih menu **File > Import to Library (Ctrl+L)**.



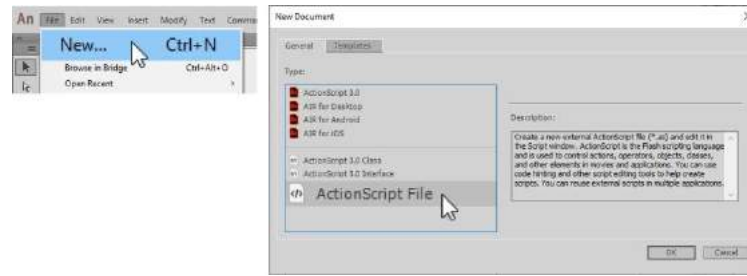
Gambar 9.10 Mengimport NPC *sprites*.

7. Pada panel *library*, klik kanan gambar **npc_sprites_1** tersebut, kemudian pilih opsi **Properties**. Pada panel *advanced* atau panel *actionscript*, centang opsi **Export for Actionscript** agar gambar dapat dikenali oleh kode.



Gambar 9.11 Menambahkan *linkage* pada NPC *Sprites*.

8. Untuk membuat dialog karakter, buatlah sebuah file *actionscript* baru dengan menekan menu **File > New** dan pilih opsi **Actionscript File**.



Gambar 9.12 Membuat file *actionscript* baru

9. Pada panel **Action**, buatlah *database* dialog untuk karakter NPC sebagai berikut :

```
1. var talk_1:Array = new Array();
2. talk_1[0] = [0, "Permisi pak... dimanakah ini?", "next"];
3. talk_1[1] = [5, "Ini desa Ketpi, kamu berasal dari
   mana?", "next"];
4. talk_1[2] = [0, "Saya dari Semarang pak", "next"];
5. talk_1[3] = [5, "Semarang, mmmmm.... saya belum pernah
   dengar", "next"];
6. talk_1[4] = [5, "Cobalah kamu ke resto, mungkin kamu bisa
   bertanya-tanya", "next"];
7. talk_1[5] = [5, "Tapi, kamu jangan pergi ke hutan! Disana
   banyak monster", "next"];
8. talk_1[6] = [0, "Baik pak, terimakasih infonya",
   "close"];
```

10. Simpan file tersebut dengan nama **dialog.as**. File dialog.as ini pada nantinya akan digunakan sebagai tempat untuk menyusun dialog antar karakter NPC. Penjelasan tentang variabel *array* pada file dialog tersebut terdapat pada bab sebelumnya.
11. Kembali buka file **tutorial-13**, kemudian klik **frame 1** kemudian buka panel **Action** dan tambahkan kode baris 9 untuk menambahkan file dialog, menjadi sebagai berikut :

```
1. include "map-2c.as";
2. include "map-3.as";
3. include "map-4.as";
4. include "map-5.as";
5. include "map-6.as";
6. include "map-7.as";
7.
8. include "flashRPG.as";
9. include "dialog.as";
10.
11. buat_level(map_2, "tileset_bmp");
12. char = set_karakter("karakter_1", 18, 7);
13. setup_GUI();
14. jalankan_game();
```

15. Jalankan aplikasi dengan menekan tombol **Ctrl+Enter**. Saat ini grafik NPC sudah muncul di layar dan dapat diajak berkomunikasi. Hasil dari proses ini adalah sebagai berikut :



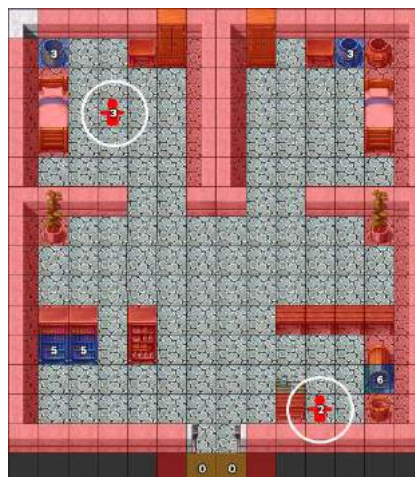
Gambar 9.13 Hasil penambahan NPC

Tutorial 14

9.4 Menambahkan *Services* NPC pada Arena Permainan

Tahapan selanjutnya adalah menambahkan karakter NPC bertipe *services* untuk tempat karakter memulihkan energi. Untuk melakukannya, perhatikan langkah berikut :

1. Buka aplikasi **map editor**, lalu buka peta **map-3.as** (peta ruangan *inn*).
2. Pada ruangan tambahkan 2 NPC, masing-masing dengan **npc id = 2** dan **npc id = 3**. NPC 2 nantinya akan digunakan sebagai NPC yang memberikan pelayanan untuk istirahat (mengembalikan energi pemain), sedangkan NPC 3 merupakan *ordinary* NPC saja. Perhatikan peletakannya pada gambar berikut:



Gambar 9.14 Penempatan NPC pada map-3

3. Simpan kembali peta dengan nama yang berbeda (**map-3c.as**). Siapkan pula gambar sprites untuk karakter NPC 2 dan 3 dengan nama **npc_sprites_2.png** dan **npc_sprites_3.png**.



Gambar 9.15 Sprites NPC 2 dan 3.

4. Buka file **dialog.as** dan tambahkan percakapan untuk NPC 2 dan 3 sebagai berikut (mulai baris ke-8):

```
1. var talk_1:Array = new Array();
2. talk_1[0] = [0, "Permisi pak... dimanakah ini?", "next"];
3. ....

8. var talk_2:Array = new Array(); //services NPC inn
9. talk_2[0] = [2, "Selamat datang di penginapan kami..",
  "sleep"];
10. talk_2[1] = [2, "Apakah anda mau istirahat, biayanya 10G
  ?", "YA", "TIDAK", "sleep", 10];
11. talk_2[2] = [2, "Terimakasih, selamat istirahat",
  "close_sleep"];
12. talk_2[3] = [2, "Maaf, anda tidak memiliki cukup uang",
  "close"];
13. talk_2[4] = [2, "Selamat pagi. Terimakasih sudah
  istirahat di penginapan kami", "close"];
14.
15. var talk_3:Array = new Array();
16. talk_3[0] = [7, "Istirahat di penginapan ini akan
  memulihkan tenagamu", "next"];
17. talk_3[1] = [7, "Hal itu bermanfaat jika kamu kelelahan
  sehabis bertempur", "next"];
18. talk_3[2] = [0, "Apakah anda habis bertempur pak?",
  "next"];
19. talk_3[3] = [7, "Iya, saya bertempur di hutan untuk
  menaikkan pengetahuan saya.", "close"]
```

5. Buka file **tutorial-13.as** dan **save-as** dengan nama lain (**tutorial-14.as**).
6. Import ke **library** dua *sprites* NPC tersebut, dan tambahkan *linkage* (**export for actionscript**) pada masing-masing *sprites*. (Lihat langkah 7 pada tutorial 13).
7. Kemudian klik **frame 1** kemudian buka panel **Action** dan edit sedikit untuk menyesuaikan peta baru, menjadi sebagai berikut:

```

1. include "map-2c.as";
2. include "map-3c.as";
3. include "map-4.as";
4. include "map-5.as";
5. include "map-6.as";
6. include "map-7.as";
7.
8. include "flashRPG.as";
9. include "dialog.as";
10.
11. buat_level(map_2, "tileset_bmp");
12. char = set_karakter("karakter_1", 18, 7);
13. setup_GUI();
14. jalankan_game();

```

8. Jalankan aplikasi dengan menekan tombol **Ctrl+Enter**. Cobalah untuk masuk bangunan *inn* dan berdialog dengan NPC yang ada, serta mencoba NPC *services* pertama, yaitu istirahat untuk memulihkan energi.

Tutorial 14a

9.5 Menambahkan *Services* NPC bertipe *Trading*

Tahapan selanjutnya adalah menambahkan NPC yang dapat menjual item (*trading*). Untuk melakukannya, perhatikan langkah berikut :

1. Buka aplikasi **map editor**, lalu buka peta **map-4.as** (peta ruangan *bar/resto*).
2. Pada ruangan tambahkan 2 NPC, masing-masing dengan **npc id = 4** dan **npc id = 5**. NPC 4 nantinya akan digunakan sebagai NPC yang menjual item, sedangkan NPC 5 merupakan *ordinary* NPC saja. Perhatikan peletakannya pada gambar berikut:



Gambar 9.16 Peletakan NPC 4 dan 5 pada map-4

3. Simpan kembali peta dengan nama yang berbeda (**map-4c.as**). Siapkan pula gambar *sprites* untuk karakter NPC 4 dan 5 dengan nama **npc_sprites_4.png** dan **npc_sprites_5.png**.



Gambar 9.17 Sprites NPC 4 dan 5

4. Untuk gambar item yang akan dijual, diperlukan sebuah grafis khusus dengan ukuran yang sama dengan ukuran *tile* yaitu 32x32 pixel, dan simpan dengan nama **item_set.png**. Pada gambar di bawah ini, file item set berukuran 128 x 128 *pixel*, dan memiliki icon-icon item yang akan dijual oleh NPC.



Gambar 9.18 gambar untuk item set

5. Buka file **dialog.as** dan tambahkan percakapan untuk NPC 4 dan 5 sebagai berikut :

```

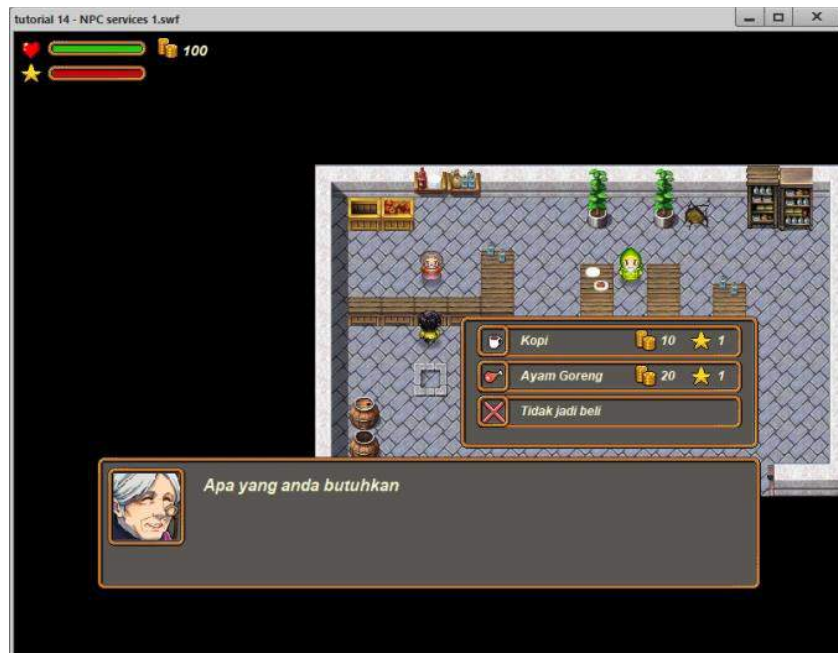
20. var talk_4:Array = new Array(); //services NPC resto
21. talk_4[0] = [4, "Selamat datang di warung kami..",
    "next"];
22. talk_4[1] = [4, "Apa yang anda butuhkan", "trade", 1];
23. talk_4[2] = [4, "Terimakasih, selamat datang kembali",
    "close"];
24. talk_4[3] = [4, "Maaf, anda tidak memiliki cukup uang",
    "close"];
25. talk_4[4] = [4, "Maaf, level anda belum cukup", "close"];
26.
27. var item_1:Array = new Array();
28. item_1[0] = [0,2, "Kopi", 1, 10, "HP", 5, 0];
29. item_1[1] = [1,2, "Ayam Goreng", 1, 20, "HP", 10, 0];
30. item_1[2] = [0,3, "Tidak jadi beli", 0, 0, "close", 0];
31.
32. var talk_5:Array = new Array();
33. talk_5[0] = [1, "Sepertinya kamu orang baru disini",
    "next"];
34. talk_5[1] = [1, "Bersiaplah, jika kamu ingin pergi ke
    hutan", "next"];
35. talk_5[2] = [1, "Belilah perlengkapan di toko sebelah",
    "close"]

```

5. Buka file **tutorial-14.as** dan lanjutkan prosesnya.
6. Import ke **library** dua *sprites* NPC 4 dan 5, serta *item_set* tersebut, dan tambahkan *linkage* (**export for actionscript**) pada masing-masing *sprites*. (Lihat langkah 7 pada tutorial 13).
7. Kemudian klik **frame 1** kemudian buka panel **Action** dan edit sedikit untuk menyesuaikan peta baru, menjadi sebagai berikut:

```
2. include "map-3c.as";
3. include "map-4c.as";
4. include "map-5.as";
```

5. Jalankan aplikasi dengan menekan tombol **Ctrl+Enter**. Cobalah untuk masuk bangunan *bar* dan berdialog dengan NPC yang ada, serta mencoba NPC *services trading*, yang akan menampilkan barang-barang sesuai variabel **item_1** pada file **dialog.as**.



Gambar 9.19 Hasil tutorial NPC *trading*

Penjelasan program

```
22. talk_4[1] = [4, "Apa yang anda butuhkan", "trade", 1];
```

Pada baris tersebut, pada nilai *array* **talk_4[1][2]** bernilai **trade** hal ini akan mengaktifasi kode untuk menambahkan panel trading. Nilai **talk_4[1][3]** bernilai **1**, hal ini berarti panel trading akan membutuhkan data item yang dijual, data ini ditulis dalam variabel *array* dengan nama **item_1**.


```
28.item_1[0] = [0,2, "Kopi", 1, 10, "HP", 5, 0];
```

Pada baris tersebut, item pertama yang dijual menggunakan **icon** yang diperoleh dari grafik **item_set**. Icon yang digunakan merupakan icon dengan kordinat (0 x 32, 2 x 32). Item bernama **Kopi** (item_1[0][2]), membutuhkan level **1** (item_1[0][3]), seharga **10G** (item_1[0][4]), berfungsi untuk menaikkan energi atau **HP** sebesar **5** poin (item_1[0][5] dan item_1[0][6]) dan item ini tidak masuk ke *inventory* pemain ((item_1[0][7] = 0).

Tutorial 14b

9.6 Menambahkan *Services* NPC bertipe *Trading* untuk item

Tahapan selanjutnya adalah menambahkan NPC yang dapat menjual item (*trading*) untuk item yang dapat digunakan oleh pemain seperti pedang, perisai dan sebagainya . Untuk melakukannya, perhatikan langkah berikut :

1. Buka aplikasi **map editor**, lalu buka peta **map-5.as** (peta ruangan *penjual item*).
2. Pada ruangan tambahkan 1 NPC, masing-masing dengan **npc id = 6**. NPC 6 nantinya akan digunakan sebagai NPC yang menjual item. Perhatikan peletakkannya pada gambar berikut:



Gambar 9.20 Peletakan NPC 6 pada map-5

3. Simpan kembali peta dengan nama yang berbeda (**map-5c.as**). Siapkan pula gambar sprites untuk karakter NPC 6 dengan nama **npc_sprites_6.png**.



Gambar 9.21 Sprites NPC 6

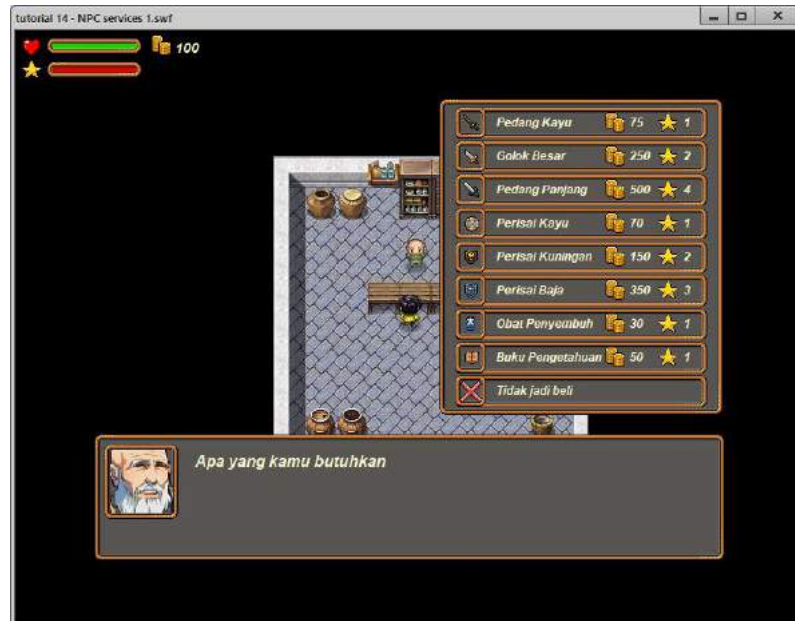
4. Untuk gambar item yang akan dijual, sudah terdapat pada file **item_set.png**, sehingga tidak membutuhkan pembuatan ulang. Pada file tersebut juga masih terdapat beberapa slot kosong, yang dapat anda isi dengan icon item lainnya dalam pengembangan game selanjutnya.
5. Buka file **dialog.as** dan tambahkan percakapan untuk NPC 6 sebagai berikut :

```
52. var talk_6:Array = new Array(); //services NPC bar
53. talk_6[0] = [3, "Selamat datang di toko peralatan ..",
    "next"];
54. talk_6[1] = [3, "Apa yang kamu butuhkan", "trade", 2];
55. talk_6[2] = [3, "Terimakasih, selamat datang kembali",
    "close"];
56. talk_6[3] = [3, "Maaf, kamu tidak memiliki cukup uang",
    "trade", 2];
57. talk_6[4] = [3, "Maaf, level kamu belum cukup", "trade", 2];
58. talk_6[5] = [3, "Kamu sudah punya barang ini", "trade", 2];
59.
60. var item_2:Array = new Array();
61. item_2[0] = [0,0, "Pedang Kayu", 1, 40, "ATK", 5, 1];
62. item_2[1] = [1,0, "Golok Besar", 2, 250, "ATK", 12, 2];
63. item_2[2] = [2,0, "Pedang Panjang", 4, 500, "ATK", 18, 3];
64. item_2[3] = [3,0, "Perisai Kayu", 1, 40, "DEF", 4, 4];
65. item_2[4] = [0,1, "Perisai Kuningan", 2, 150, "DEF", 9, 5];
66. item_2[5] = [1,1, "Perisai Baja", 3, 350, "DEF", 12, 6];
67. item_2[6] = [2,1, "Obat Penyembuh", 1, 30, "HP", 10, 7];
68. item_2[7] = [3,1, "Buku Pengetahuan", 1, 50, "EXP", 25, 0];
69. item_2[8] = [0,3, "Tidak jadi beli", 0, 0, "close", 0];
```

8. Buka file **tutorial-14.as** dan lanjutkan prosesnya.
9. Import ke **library sprites** NPC 6, dan tambahkan *linkage* (**export for actionscript**) pada masing-masing *sprites*. (Lihat langkah 7 pada tutorial 13).
10. Kemudian klik **frame 1** kemudian buka panel **Action** dan edit sedikit untuk menyesuaikan peta baru, menjadi sebagai berikut:

```
6. include "map-3c.as";
7. include "map-4c.as";
8. include "map-5c.as";
```

9. Jalankan aplikasi dengan menekan tombol **Ctrl+Enter**. Cobalah untuk masuk bangunan *toko item* serta mencoba *NPC services trading*, yang akan menampilkan barang-barang sesuai variabel **item_2** pada file **dialog.as**.



Gambar 9.22 Hasil tutorial NPC *trading*

Tutorial 15

9.7 Menambahkan data item

Setelah memahami sistem *trading* dengan menambahkan *item* ke dalam inventory pemain, maka kita juga dapat mengaktifasi item-item yang tersebar di arena permainan. Pada saat kita membuat peta pada tutorial-tutorial sebelumnya, juga terdapat sebuah opsi **item_id** di masing-masing peta. Untuk itu kita akan menambahkan fitur “menemukan item”. Untuk melakukannya, perhatikan langkah berikut :

1. Buatlah sebuah file actionscript baru dengan memilih menu **File > New**, pilih opsi **Actionscript File**. Simpan dengan nama **item_db.as**. Pada file ini kita akan menambahkan data item-item yang tersebar di arena permainan.
2. Klik panel **Action** dan ketikan kode berikut :

```
1. var item_db:Array = new Array();
2. item_db[0] = [0, "tidak ada apa-apa disini", "close"];
3. item_db[1] = [0, "ke kanan: Hutan Desa, ke bawah: patung athens", "close"];
4. item_db[2] = [0, "Aku menemukan uang 25G", "item", 0, 25];
5. item_db[3] = [0, "Aku menemukan uang 5G", "item", 0, 5];
6. item_db[4] = [0, "Aku menemukan uang 2G", "item", 0, 2];
7. item_db[5] = [0, "Membaca buku bisa meningkatkan pengetahuan (EXP+2)", "exp", 2];
8. item_db[6] = [0, "Aku menemukan uang 10G", "item", 0, 10];
9. item_db[7] = [0, "Aku menemukan Obat Penyembuh", "item", 7, 1];
```

```

10.item_db[8] = [0, "Aku menemukan uang 10G", "item", 0,
    10];
11.item_db[9] = [0, "Patung ini sepertinya menyimpan
    rahasia....mmmm....", "close"];
12.item_db[10] = [0, "Bentuknya seperti patung yang ada di
    desa, apa ya maksudnya", "close"];

```

Flash Tips

Penulisan struktur **item_db** pada dasarnya sama dengan *database* untuk dialog. Selanjutnya anda juga perlu merencanakan item-item saja yang diperlukan oleh pemain pada awal permainan. Sebagai contoh di tutorial ini, pemain yang mengeksplorasi penuh arena permainan akan mendapatkan uang senilai 52G, dan cukup untuk membeli perlengkapan pedang kayu.

Perencanaan yang baik akan menjadikan permainan semakin menarik untuk diikuti.

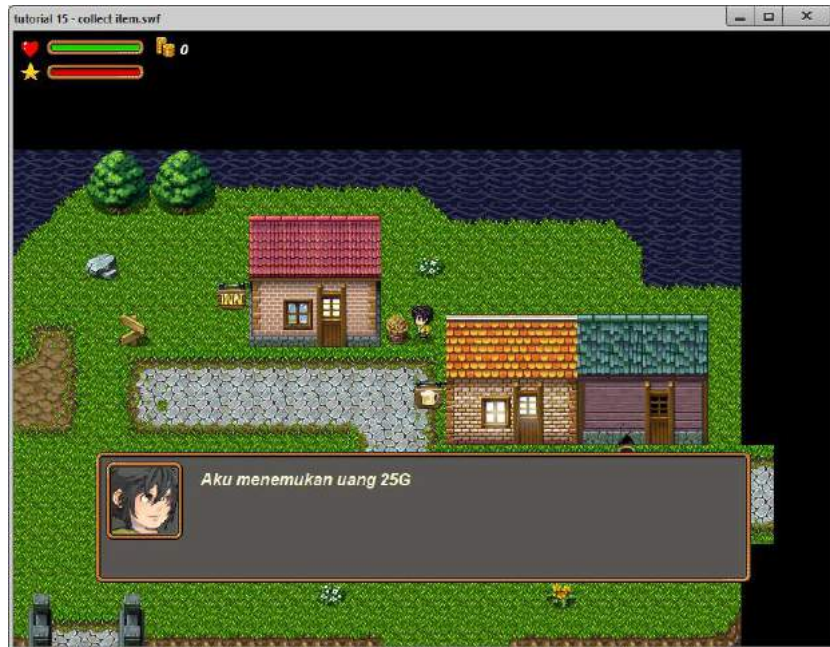
3. Selanjutnya, buka file **tutorial-14.fla** dan simpan dengan nama lain (**tutorial-15.fla**).
4. Klik **frame 1** dan buka **panel action**, selanjutnya tambahkan baris berikut untuk membuka file **item_db.as** yang telah dibuat sebelumnya.

```

1. include "map-2c.as";
2. include "map-3c.as";
3. include "map-4c.as";
4. include "map-5c.as";
5. include "map-6.as";
6. include "map-7.as";
7.
8. include "flashRPG.as";
9. include "dialog.as";
10. include "item_db.as";
11.
12. buat_level(map_2, "tileset_bmp");
13. char = set_karakter("karakter_1", 18, 7);
14. setup_GUI();
15. jalankan_game();

```

16. Jalankan aplikasi dengan menekan tombol **Ctrl+Enter**. Cobalah untuk mengakses lokasi-lokasi item dengan menekan tombol spasi. Hasil tutorial seperti pada gambar berikut :



Gambar 9.23 Hasil tutorial peletakan item.

BAB X

MUSUH DAN *BATTLE SYSTEM*

Salah satu faktor yang membuat permainan menantang adalah elemen *challenge* yang berupa musuh. Ketika pemain bertemu dengan musuh, maka permainan akan berubah pada mode *battle*.

MUSUH DAN *BATTLE SYSTEM*

10.1 Konsep *Battle System* dalam game RPG

Sebuah game memiliki elemen *challenge* yaitu sebuah tantangan yang harus diselesaikan dalam permainan. Tantangan dalam game memiliki bentuk yang beragam, dari yang paling sederhana sampai yang kompleks. Dalam game bertipe RPG, tantangan sering kali ditampilkan dalam bentuk pertarungan dengan musuh, atau penyelesaian teka-teki.

Salah satu aspek permainan peran yang paling menonjol dan paling menarik dalam game RPG pada umumnya adalah *Battle System* (sistem pertempuran). Pada umumnya sistem pertempuran menjadi inti dari permainan game RPG setelah cerita, sehingga perencanaan sistem pertempuran yang baik sangat diperlukan untuk meningkatkan daya tarik permainan. Beberapa konsep musuh dan sistem permainan yang dapat diadaptasi dari game RPG populer, antara lain sebagai berikut :

1. Bagaimana sebuah pertempuran dimulai?

Dalam sistem pertempuran, sangat penting untuk menentukan kapan pertempuran dengan musuh akan dimulai. Setidaknya ada beberapa jenis awal pertempuran dalam permainan, yaitu :

- a. Terencana (*scripted*), dimana permainan selalu menampilkan musuh yang sudah terencana, dengan posisi dan variabel yang selalu tetap. Ketika karakter pemain memasuki area tertentu dimana musuh sudah diletakkan, maka pertempuran akan terjadi. Contoh game RPG yang menggunakan awal pertempuran terencana antara lain : Final Fantasi Tactics, Shining Force dan sejenisnya.
- b. Acak (*random*), dimana musuh tidak tampak di layar dan muncul secara acak ketika pemain melalui daerah yang diset sebagai daerah musuh. Dalam hal ini, keluarnya musuh disesuaikan dengan atribut dari karakter pemain. Dalam game Pokemon misalnya, yang menggunakan sistem acak, musuh akan keluar sesuai dengan level pemain. Pemain dengan level rendah, sangat jarang bertemu dengan musuh dengan level tinggi. Peluang tersebut tetap ada, namun sangat kecil. Contoh lain game RPG dengan awal pertempuran acak antara lain: Final Fantasy 1-10, Pokemon series, Golden Sun, Star Ocean 1 & 2 dan sejenisnya
- c. Aktif menyerang secara langsung (*active*) dimana pertempuran secara langsung terjadi pada arena permainan (*hack and slash*). Pada arena pertempuran musuh secara aktif menggunakan kecerdasan buatan (*AI*) akan menyerang pemain secara langsung, dan pertempuran terjadi secara aktif.

Contoh dari game ini antara lain : Diablo, The Witcher, Mass Effect, Dark Cloud 2, Zelda.

- d. Musuh di arena permainan (*enemy on the map*). Seperti halnya jenis aktif, pada jenis ini musuh terdapat di arena permainan, namun tidak secara langsung menyerang pemain. Pertempuran baru akan terjadi jika pemain sudah sangat dekat dengan musuh, dan pada umumnya permainan akan berpindah dari mode eksplorasi ke mode pertempuran, berbeda dengan tipe sebelumnya yang mana pertempuran terjadi secara langsung pada arena eksplorasi. Contoh dari game ini antara lain : Grandia, Persona, Star Ocean 3 & 4.



Gambar 10.1 Final Fantasy dengan kemunculan musuh secara acak
(Copyright Square Enix, 1997)

2. Bagaimana sistem pertempuran dijalankan?

Setelah sistem pertempuran dimulai, maka selanjutnya adalah permainan berfokus pada sistem untuk menjalankan pertempuran. Setidaknya terdapat 2 kategori utama dalam mode menjalankan sistem permainan, yaitu :

- a. *Realtime*. Pada mode ini pertempuran terjadi secara langsung, karakter pemain dapat dikendalikan oleh pemain untuk melakukan gerakan menyerang. Musuh secara aktif juga menyerang dan aktif secara individu tanpa menunggu giliran menyerang. Pada umumnya game akan bertipe *action RPG /hackn slash RPG*. Contoh game bertipe ini adalah : Diablo, Tales of Phantasia, Kingdom Hearts dan sebagainya.



Gambar 10.2 Sistem pertempuran dalam game Kingdom Hearts
(Copyright Square Enix, 2017)

- b. *Turn based*. Pada mode pertempuran ini masing-masing karakter akan mendapatkan satu giliran setiap putaran. Giliran ditentukan oleh variabel kemampuan dari karakter (biasanya variabel *speed* atau *agility*), yang selanjutnya secara berurutan karakter akan melakukan satu dari beberapa pilihan gerakan, bisa jadi menyerang, bertahan, menggunakan item, atau hanya diam dan melewatkan giliran. Contoh dari game dengan sistem pertempuran *turn based* antara lain : Pokemon, Final Fantasy Tactic, Golden Sun, Final Fantasy dan sebagainya.



Gambar 10.3 Sistem *turnbased* pada game Pokemon
(Copyright Game Freak, Nintendo, 2003)

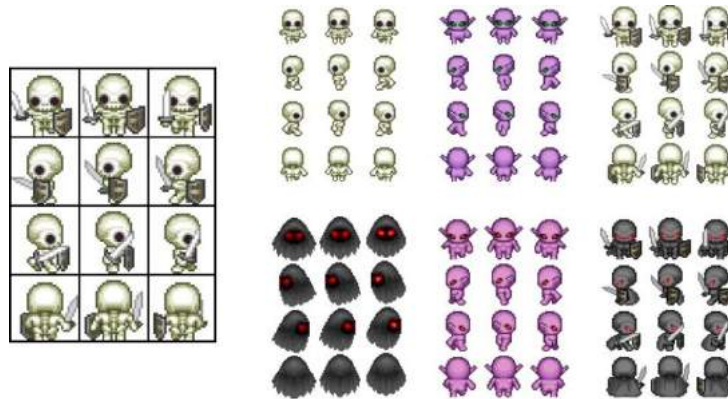
10.2 Mendesain karakter musuh (*enemy sprites*) berikut kemampuannya

Salah satu jenis *Non Playable Character* (NPC) dalam RPG adalah karakter musuh (*villain NPC*). Musuh merupakan karakter antagonis yang bertugas mengalahkan karakter atau menghambat laju karakter, tapi disisi lain musuh juga merupakan media yang digunakan untuk menaikkan skill pemain, mendapatkan bonus item, score maupun *experience* untuk menaikkan level pemain.

Mendesain musuh yang baik dalam game RPG, harus memahami tingkat perkembangan karakter pemain. Kemampuan yang dimiliki musuh, harus disesuaikan sedemikian rupa, agar permainan tidak terlalu sulit untuk dikuasai namun juga tidak terlalu mudah untuk dikalahkan. Hal ini yang disebut sebagai *balancing* dalam game. Proses *balancing* ini bisa dilakukan setelah fitur utama sistem pertempuran selesai, dan dengan menguji coba permainan berkali-kali sampai menemukan titik yang paling ideal.

Seperti halnya *sprites* untuk karakter utama dan NPC, grafik karakter musuh juga menggunakan teknik 3×4 grid dalam bentuk *sprites*. Karakter digambar sedemikian rupa dengan menyesuaikan atribut yang dimiliki. Sebagai contoh, dalam game yang dikembangkan dalam tutorial ini digunakan 6 jenis *sprites* musuh, yaitu tengkorak

kecil, tuyul kecil, tengkorak besar, setan ungu, setan hitam dan satria tengkorak. Masing-masing *sprites* digambar dengan teknis dan peng gayaan yang sama dengan *sprites* karakter lainnya dan disimpan dengan nama **enemy_sprites_1**, **enemy_sprites_2** dan seterusnya. Perhatikan gambar :



Gambar 10.4 *Sprites* untuk NPC musuh

Selanjutnya untuk menambahkan kemampuan musuh, digunakan variabel bertipe *Array* dengan struktur sebagai berikut :

```
enemy[0] = [12, 1, "Tengkorak kecil", 10, 3, 2, 1, 1];
```

Data *array* pertama berisi angka **12** hal ini mengacu pada *avatar* yang mewakili musuh (perhatikan file **face_set**. Pada file tersebut, *avatar* tengkorak berada pada kotak ke 12). Selanjutnya `enemy[0][1]` bernilai **1**, hal ini mengacu pada nomor *sprites* yang akan digunakan untuk menampilkan karakter (**enemy_sprites_1**). `Enemy[0][2]` berisi nama musuh. `Enemy[0][3]` berisi energi (**HP**) dari musuh. `Enemy[0][4]` berisi daya serang (**ATK**) dari musuh. `Enemy[0][5]` berisi daya tahan (**DEF**) dari musuh. `Enemy[0][6]` berisi level minimal dari karakter pemain yang dibutuhkan agar musuh bisa muncul, dengan mengisi angka yang rendah di awal permainan, kode akan menjamin musuh terkuat tidak akan muncul terlebih dahulu. `Enemy[0][7]` berisi kecepatan (**SPD**) dari musuh yang akan menentukan seberapa cepat waktu serangan dari musuh.



Gambar 10.5 Menyesuaikan atribut karakter musuh 1

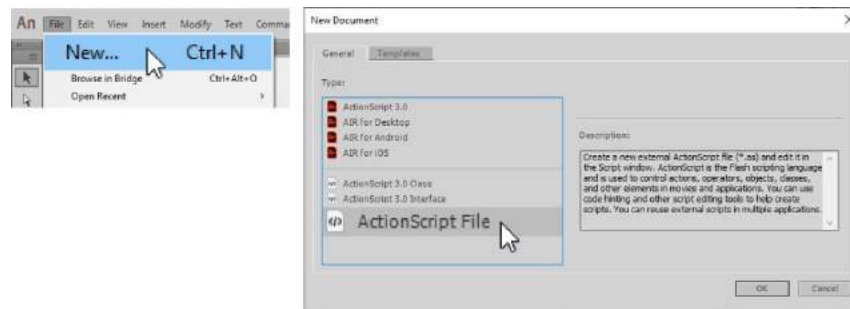
Tutorial 16

Flash Tips

Pengaturan atribut dari musuh membutuhkan proses ujicoba berulang (teknik *balancing*). Angka-angka dalam variabel tersebut dapat disesuaikan dengan melihat perkembangan karakter pemain. Sebagai contoh, apabila karakter musuh terlalu kuat di awal permainan, maka karakter pemain dengan mudah dikalahkan dan permainan akan segera berakhir.

Setelah menentukan data atribut untuk musuh, serta membuat tampilan visual musuh dalam bentuk grafis *sprites*, maka tahapan selanjutnya adalah menuliskan seluruh atribut tersebut ke dalam sebuah file terpisah. Untuk membuat *database* musuh yang dimaksud, ikuti langkah berikut :

1. Buatlah sebuah file actionscript baru dengan memilih menu **File>New** dan memilih opsi **Actionscript File**.

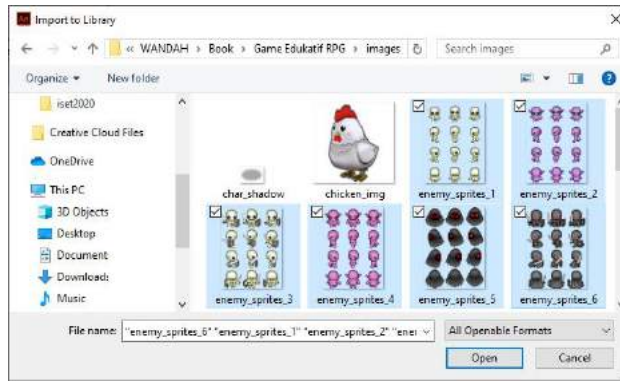


Gambar 10.6 Membuat file *actionscript*

2. Klik panel **action** dan ketikkan kode yang berisi data musuh sebagai berikut :

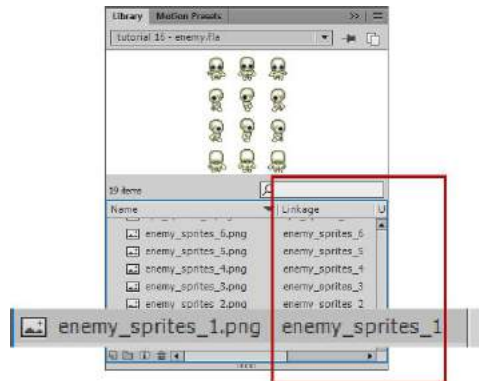
```
1. var enemy:Array = new Array();
2. enemy[0] = [12, 1, "Tengkorak kecil", 10, 3, 2, 1, 1];
3. enemy[1] = [13, 2, "Tuyul kecil", 12, 6, 4, 1, 1];
4. enemy[2] = [12, 3, "Satria Tengkorak", 20, 9, 6, 2, 2];
5. enemy[3] = [13, 4, "Monster Ungu", 25, 11, 7, 3, 2];
6. enemy[4] = [15, 5, "Hantu hitam", 25, 15, 10, 4, 2];
7. enemy[5] = [14, 6, "Tengkorak hitam", 30, 20, 12, 5, 2];
```

3. Simpan dengan nama **enemy_db.as**. Pastikan anda menyimpannya pada folder yang sama dengan folder file proyek game yang sedang dibuat.
4. Buka kembali file **tutorial-15.as**, simpan dengan nama lain (**tutorial-16 fla**).
5. Import seluruh gambar *sprites* musuh ke *library*, dengan memilih opsi **File > Import > Import to Library**.



Gambar 10.7 Mengimport semua *sprites* musuh ke library

6. Buka panel Library (**Ctrl+L**). Klik kanan gambar **enemy_sprites_1** dan pilih opsi **properties**. Selanjutnya centang opsi **Export for actionscript** untuk menambahkan *linkage*, sehingga gambar dapat dikenali oleh kode. Lakukan pada semua gambar *sprites* musuh.



Gambar 10.8 *Linkage* pada *sprites* musuh

7. Klik **frame 1** kemudian buka panel **Action**, kemudian tambahkan kode menjadi sebagai berikut :

```

1. include "map-2c.as";
2. include "map-3c.as";
3. include "map-4c.as";
4. include "map-5c.as";
5. include "map-6.as";
6. include "map-7.as";
7.
8. include "flashRPG.as";
9. include "dialog.as";
10. include "item_db.as";
11. include "enemy_db.as";
12.
13. battle_arena = map_7;
14. buat_level(map_2, "tileset_bmp");
15. char = set_karakter("karakter_1", 18, 7);
16. setup_GUI();
17. jalankan_game();

```


8. Jalankan file dengan menekan **Ctrl+Enter**, dan gerakkan karakter menuju ke arena pertempuran, maka akan didapatkan karakter musuh yang bergerak. Perhatikan pada baris 11, kita membuka file *database* musuh, dan pada baris no 13 mendefinisikan arena yang dijadikan sebagai tempat munculnya musuh, yaitu **map-7**. Pada tahapan ini, permainan akan mengalami *error* apabila anda mendekati musuh, hal ini wajar karena sistem pertempuran atau *battle system* belum diatur.



Gambar 10.9 Hasil tutorial penambahan musuh

10.3 Mengimplementasikan Sistem Kuis pada *Battle System*

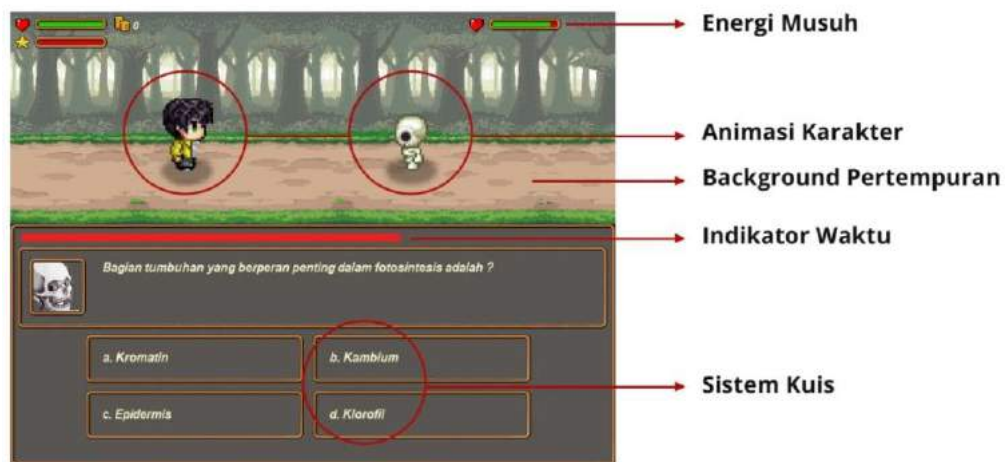
Game yang akan kita buat menerapkan sistem pertempuran *turnbased*, dimana pemain dan musuh akan bergantian menyerang sesuai dengan urutan waktu yang ditentukan. Namun, untuk menekankan elemen pendidikan kita akan merubah urutan waktu dengan sistem kuis, dimana pemain akan menyerang jika menjawab benar dan pemain akan diserang jika menjawab salah.

Sistem kuis menggunakan soal dan jawaban yang diacak, sehingga pemain akan mendapatkan variasi soal yang berbeda setiap kali bertempur dengan musuh. Cara kerja sistem kuis dapat dilihat pada buku “Desain dan Pemrograman Multimedia Pembelajaran Interaktif” halaman 147 atau melalui tutorial game kuis pada website <http://www.wandah.org/membuat-game-kuis-dengan-flash>.



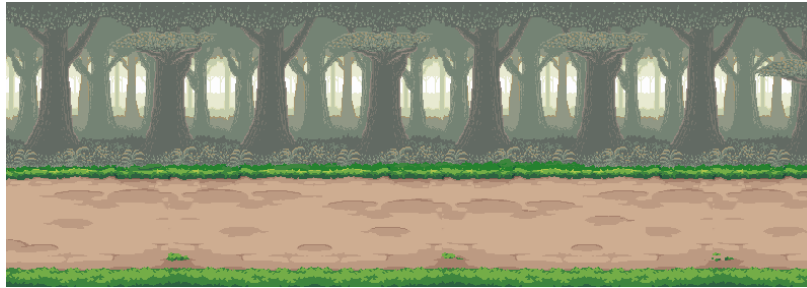
Gambar 10.10 Referensi sistem kuis

Selanjutnya diperlukan sebuah desain (*mockup*) visualisasi dari sistem pertempuran yang akan ditampilkan dalam game. Pada desain ini, sistem kuis akan diletakkan di bagian bawah, dan di bagian atas akan terdapat animasi yang menampilkan karakter pemain dan musuh. Pada layar juga akan terdapat *avatar* musuh, yang seolah-olah memberikan pertanyaan kepada pemain. Selanjutnya terdapat juga indikator batas waktu untuk menjawab dalam bentuk garis merah. Secara visual, *mockup* yang dibuat adalah sebagai berikut :



Gambar 10.11 *Mockup* sistem pertempuran

Pada gambar *mockup* di atas, hampir seluruh elemen grafis sudah tersedia. Game UI dan panel untuk menampilkan soal dapat menggunakan file **UI_set**, sedang animasi karakter secara otomatis oleh kode akan disusun dari grafik **karakter** dan grafik **enemy_sprites**. Satu-satunya grafik yang belum tersedia adalah *background* atau latar belakang permainan. Oleh karena itu, sediakan grafis untuk *background* dengan nama **bg_battle.png** (dengan ukuran 800 x 280 *pixel* menyesuaikan *mockup* yang dibuat).



Gambar 10.12 *Background* untuk sistem pertempuran.

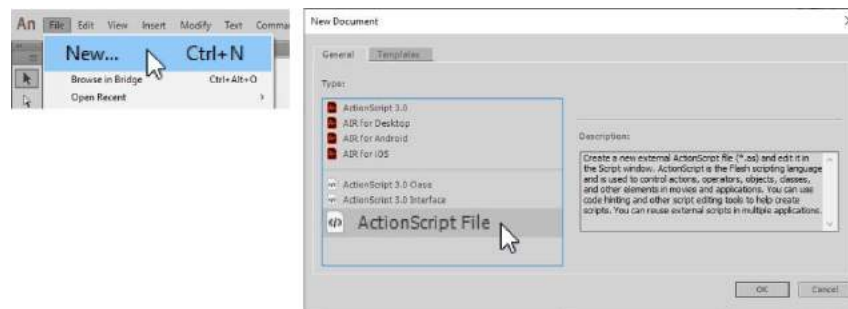
Pada tahap pengembangan selanjutnya, anda dapat menambahkan variasi *background* tempat pertempuran, menambahkan variabel lain seperti elemen yang mempengaruhi atribut. Misal ketika bertempur di arena dengan elemen air, maka musuh-musuh yang berelemen air akan semakin kuat, sedang musuh dengan elemen api akan melemah. Pengembangan seperti ini dapat dilakukan dengan memodifikasi file *flashRPG.as*.

Tutorial 17

10.4 Menyusun *database* Soal untuk Sistem Pertempuran

Untuk mewujudkan *mockup* sistem pertempuran di atas, diperlukan *database* soal sebanyak mungkin agar pemain tidak “mengulang hafalan” jawaban dari soal-soal. Oleh karena itu diperlukan sebuah file terpisah untuk menuliskan *database* soal tersebut, untuk lebih jelasnya ikuti langkah berikut :

1. Buatlah sebuah file *actionscript* baru dengan memilih menu **File>New** dan memilih opsi **Actionscript File**.



Gambar 10.6 Membuat file *actionscript*

2. Klik panel **action** dan ketikkan kode yang berisi data soal sebagai berikut :

```
1. var soal:Array = new Array();
2. soal[0] = ["Siapakah penemu mesin uap ?", "James Watt",
  "Davinci", "Issac Newton", "James bond"];
3. soal[1] = ["Akar dari 676 adalah ?", "26", "24", "16", "34"];
4. soal[2] = ["Bagian tumbuhan yang berperan penting dalam
  fotosintesis adalah ?", "Klorofil", "Kambium", "Epidermis",
  "Kromatin"];
5. ....
```

```

96. soal[94] = ["Jumlah seluruh sudut segitiga siku-siku adalah ?", "180", "270", "90", "360"];
97. soal[95] = ["Provinsi termuda di Indonesia adalah ? ", "Banten", "Bangka belitung", "Batam", "Gorontalo"];
98. soal[96] = ["Hukum gravitasi ditemukan oleh ? ", "Issac Newton", "Einstein", "T. A. Edison", "Kopernicus"];
99. soal[97] = ["(sin 30) x 8 + (sin 90) = ? ", "5", "8", "0", "1"];

```

Pada struktur *database* soal di atas, **soal[0][0]** berisi tentang pertanyaan. Selanjutnya **soal[0][1]** sampai **soal[0][4]** merupakan alternatif jawaban, dimana jawaban yang benar diletakkan pada **soal[0][1]**.



Sebisa mungkin buatlah soal sebanyak-banyaknya untuk menghindari perulangan soal yang terlalu sering muncul.

3. Simpan dengan nama **soal_db.as**. Pastikan anda menyimpannya pada folder yang sama dengan *folder* file proyek game yang sedang dibuat.
4. Buka kembali file **tutorial-16.as**, simpan dengan nama lain (**tutorial-17 fla**).
5. Import gambar *background* **bg_battle.png** ke *library*, dengan memilih opsi **File > Import > Import to Library**.
6. Klik kanan **bg_battle** pada panel *Library*, pilih **Properties**. Kemudian tambahkan *linkage* dengan mencentang opsi **Export for Actionscript**.
7. Klik **Frame 1** kemudian buka panel **Action**, kemudian tambahkan kode menjadi sebagai berikut :

```

1. include "map-2c.as";
2. include "map-3c.as";
3. include "map-4c.as";
4. include "map-5c.as";
5. include "map-6.as";
6. include "map-7.as";
7.
8. include "flashRPG.as";
9. include "dialog.as";
10. include "item_db.as";
11. include "enemy_db.as";
12. include "soal_db.as";
13.
14. soal_quiz = soal;
15. battle_arena = map_7;
16. buat_level(map_2, "tileset_bmp");
17. char = set_karakter("karakter_1", 18, 7);
18. setup_GUI();
19. jalankan_game();

```

20. Jalankan aplikasi dan saat ini anda dapat melawan musuh melalui sebuah sistem pertempuran berbasis kuis.



Gambar 10.13 Hasil penambahan sistem pertempuran

BAB XI

HALAMAN PEMBUKA GAME

Sebuah permainan tidak serta merta dimulai, saat pertamakali aplikasi berjalan, namun pada umumnya terdapat halaman pembuka yang berisi tentang judul game, ilustrasi dan tombol untuk memulai permainan.

HALAMAN PEMBUKA GAME (*TITTLE SCREEN*)

11.1 Mendesain Halaman Pembuka Permainan

Saat game atau aplikasi pertama kali dijalankan, maka permainan tidak sertamerta dimulai, namun terdapat halaman pembuka (*tittle screen*). Halaman pembuka secara umum tersusun atas *background* ilustrasi, judul atau nama game, dan tombol untuk memulai permainan. Selain itu, halaman pembuka sering kali menampilkan tombol untuk pengaturan permainan dan tombol untuk membuka permainan yang tersimpan (*load game*).

Halaman pembuka pada permainan memiliki fungsi sebagai *first impression* sebuah game. Sebagai halaman yang pertama kali dilihat oleh pemain, maka halaman ini harus memiliki daya tarik, yang bisa diwujudkan melalui ilustrasi gambar atau ilustrasi musik. Namun tidak jarang pula, sebuah halaman pembuka hanya menampilkan ilustrasi yang sangat sederhana, atau hanya menampilkan judul dari game saja.



Gambar 11.1 Halaman pembuka game Pokemon Emerald (*Copyright* GameFreak, 2005) dan The Legend of Zelda, the Minish Cap (*Copyright* Capcom, 2004) menampilkan ilustrasi dan judul



Gambar 11.2 Halaman pembuka game Dark Souls, hanya menampilkan judul permainan (*Copyright* FromSoftware, 2009)

Untuk game dalam tutorial di buku ini, halaman pembuka didesain secara sederhana dengan ilustrasi, judul game dan sebuah tombol untuk memulai.



Gambar 11.3 Desain halaman pembuka permainan

Untuk mendesain halaman ilustrasi (*background*) halaman pembuka, anda dapat menggunakan *software* grafis seperti Photoshop. Buatlah dengan ukuran yang sesuai dengan ukuran layar game yang dibuat, dalam hal ini adalah 800 x 600 *pixel* dan simpan dalam format JPG atau PNG.

Tutorial 18

11.2 Menambahkan Halaman Pembuka dan halaman Game Over

Untuk menambahkan halaman pembuka ikuti langkah-langkah berikut:

1. Siapkan sebuah file untuk *background* halaman pembuka. Dalam hal ini tombol tidak perlu ditambahkan, karena kode secara otomatis akan membuat tombol untuk memulai permainan. Simpan dengan nama **cover_bg.jpg**



Gambar 11.4 Ilustrasi halaman pembuka

2. Siapkan juga sebuah file *background* untuk halaman *game over*. Dimana halaman ini nantinya akan muncul jika pemain kalah dalam bertempur. Seperti halnya ilustrasi halaman pembuka, pada ilustrasi halaman *game over* juga tidak diperlukan tombol, karena akan dibuat secara otomatis oleh kode. Simpan dengan nama **gameover_bg.jpg**.



Gambar 11.5 Ilustrasi *game over*

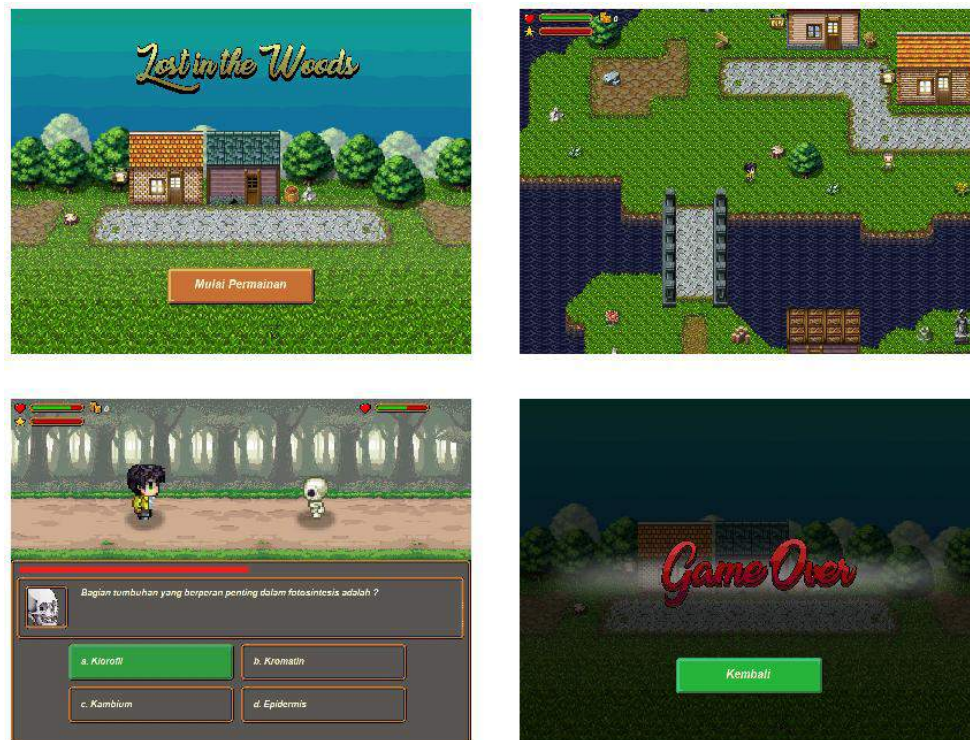
3. Buka kembali file **tutorial-17.fla** dan simpan dengan nama lain (**tutorial-18.fla**).
4. Import ke library ke dua gambar tersebut dengan memilih menu **File > Import > Import to Library**.
5. Pada panel **Library** klik kanan gambar dan pilih opsi **Properties**, kemudian centang opsi **Export for Actionscript** untuk menambahkan *linkage*. Lakukan pada kedua gambar tersebut agar dapat diidentifikasi oleh kode.
6. Klik **frame 1** dan buka panel **Action**, kemudian ubah kode menjadi sebagai berikut :

```

1. include "map-2c.as";
2. include "map-3c.as";
3. include "map-4c.as";
4. include "map-5c.as";
5. include "map-6.as";
6. include "map-7.as";
7.
8. include "flashRPG.as";
9. include "dialog.as";
10. include "item_db.as";
11. include "enemy_db.as";
12. include "soal_db.as";
13.
14. soal_quiz = soal;
15. battle_arena = map_7;
16. char_sprites_name = "karakter_1";
17. karakter_x = 18;
18. karakter_y = 7;
19. map_start = map_2;
20. halaman_awal("cover_bg");

```

7. Jalankan game dengan menekan tombol **Ctrl+Enter** dan akan didapati game yang hampir utuh, dimana dimulai dari halaman pembuka dan dapat diakhiri ketika pemain kalah dari pertempuran



Gambar 11.6 Hasil penambahan halaman pembuka dan halaman *game over*

BAB XII

SUARA DALAM GAME RPG

Suara dalam permainan berperan penting untuk membuat suasana permainan menjadi lebih realis. Penambahan suara yang tepat ke dalam sebuah permainan dapat membentuk *mood* sekaligus imersi.

SUARA DALAM GAME RPG

12.1 Konsep Suara dalam Game

Suara dalam konsep media audio visual memegang peranan yang setara dengan aspek visual. Game sebagai cabang dari media audio visual tentunya membutuhkan suara sebagai elemen pendukung untuk memperkuat pengaruhnya terhadap penggunanya yang dalam hal ini adalah pemain. Tanpa suara, game masih dapat dijalankan, namun tidak semenarik ketika suara hadir dan mendukung tampilan visual yang ada.

Suara dalam game memiliki jenis yang sangat beragam, namun setidaknya dalam buku ini suara dalam game dikategorikan menjadi 5 kategori utama, yaitu :

1. *Background Sound (BG)*, yang merupakan suara latar belakang atau ambians dari lingkungan yang ditampilkan di layar. Seperti dalam game *Zelda the Minish Cap* ketika permainan menampilkan lingkungan hutan, maka suara yang muncul adalah suara hutan dengan suara khas serangga, cicitan burung dan gemerisik angin meniup pohon. Contoh lain dalam game *Ace Gangster* karena menggunakan latar belakang kota, maka muncul suara keriuhan kendaraan yang sedang melintas, sesekali terdengar suara klakson dan suara kerumunan.
2. *Game's Music (BGM/MX)*, merupakan musik yang mendukung suasana permainan. BGM biasanya muncul di awal permainan, saat-sat kondisi tertentu seperti ketika memasuki mode pertempuran, saat pemain menang atau kalah dan sebagainya. Sebagai contoh Game *Super Mario* (meskipun bukan game bertipe RPG), memiliki musik yang sangat khas yang menjadi ciri khas pendukung game, musik akan berubah menjadi lebih intens ketika waktu yang dimiliki pemain kurang dari 100 detik.
3. *Sound Effect (SFX)*, merupakan suara efek yang dihasilkan oleh sesuatu, yang diadaptasi dari kondisi riil di alam nyata. Sebagai contoh ketika dalam kondisi perang dan pemain menembakkan senjata, maka suara senjata akan muncul. Demikian pula jika terdapat ledakan, maka suara ledakan akan mengikuti aspek visual yang ada.

Dalam SFX dikenal pula istilah *Foley (FOL)* yaitu suara efek yang dihasilkan oleh pemain, suara ini bersifat minor namun keberadaannya meningkatkan efek realis. Sebagai contoh efek suara langkah kaki yang berbeda ketika melangkah di atas lantai kayu, lantai beton atau di atas tanah, suara gemerincing uang ketika pemain mendapatkan koin, suara nafas pemain yang mengikuti kejadian yang ada di dalam game, dan sebagainya.

4. *Dialog (DX)* merupakan suara karakter yang secara langsung dilafalkan. Pada game-game modern, dialog karakter direkam secara khusus dari suara aktor/aktris pengisi suara. Suara tersebut selanjutnya dimunculkan dalam game berikut teks dialog dari percakapan tersebut. Pada game RPG klasik, dialog hanya berupa tulisan disertai sedikit *SFX*, hal ini dikarenakan ukuran file suara yang relatif besar belum didukung oleh teknologi di masa tersebut.

Keberadaan suara dalam game memiliki tujuan tertentu, oleh karena itu seorang game developer harus mampu memilih suara, serta mengatur volume dari masing-masing suara agar tujuan dari suara tersebut tercapai dengan optimal. Adapun tujuan utama dari penambahan suara dalam game adalah :

1. Menunjukkan keadaan kegiatan karakter pemain. Sebagai contoh *SFX* akan dimunculkan ketika pemain naik level, mendapatkan poin/item tertentu, membeli atau menjual item, menyerang musuh dan sebagainya.
2. Menunjukkan lingkungan tempat karakter berada. Apakah lingkungan sedang hujan, panas, tenang, menakutkan, dan sebagainya.
3. Memberikan suasana permainan (*mood*). Suara mampu menghasilkan imersi dari kejadian yang dimunculkan di layar, menguatkan suasana hati pemain, dan membangkitkan perasaan.

Tidak ada aturan baku dalam memilih suara yang akan digunakan dalam game, namun seseorang yang sering memainkan game atau menonton film akan menjadi lebih peka dalam memilih suara yang akan dimunculkan. Anda dapat mengelola beberapa jenis suara sekaligus bahkan menampilkannya dalam format 3 dimensi (volume suara muncul secara dinamis berdasarkan jarak sumber suara dengan pemain), atau menampilkannya secara datar (2 dimensi) suara muncul dengan volume yang realtif sama. Seluruh pilihan ini tergantung pada masing-masing developer, kemampuan aplikasi dan kemampuan *hardware* audio.

12.2 Mengatur File Suara untuk Game RPG

Dalam game RPG yang dikembangkan di buku ini, seluruh kategori suara dapat digunakan, namun dalam tutorial ini tidak digunakan suara kategori dialog (*DX*). Game akan menggunakan beberapa musik (*BGM*) di antaranya musik saat halaman pembuka, musik saat di desa, musik saat bertarung dan musik saat *game over*. Selanjutnya digunakan suara efek (*SFX*) ketika membeli item, dan ketika menyerang. Musik atau *BGM* yang digunakan menggunakan musik bergenre klasik medieval, untuk mendukung tema game yang dibuat. Untuk musik ketika memasuki mode pertempuran, digunakan musik bertensi tinggi untuk meningkatkan kesan ketegangan.

File audio yang didukung oleh Adobe Animate/Flash adalah file bertipe **.wav** dan **.mp3**. File audio tersebut dapat anda buat sendiri menggunakan aplikasi editor audio seperti Audacity (*freeware*), Reaper, Fruityloop, Cubase, Sony Acid, Pro Tools, Adobe Audition, dan sebagainya. Anda juga dapat memanfaatkan file suara yang tersedia secara online baik gratis maupun berbayar

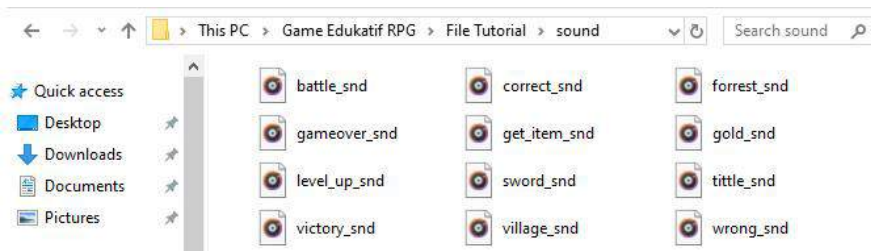
Flash Tips

File audio banyak terdapat pada situs-situs penyedia seperti soundcloud, opensound.org, freesound.org dan sebagainya. Pada situs-situs tersebut terdapat file suara yang berbayar maupun gratis. Hal penting yang harus diperhatikan, jika anda ingin mengkomersialkan game pada tahap selanjutnya, maka anda harus memperhatikan lisensi yang ada. Untuk game komersial sebisa mungkin gunakan audio yang eksklusif agar berbeda dengan game-game lainnya

12.3 Menambahkan Suara ke dalam Game

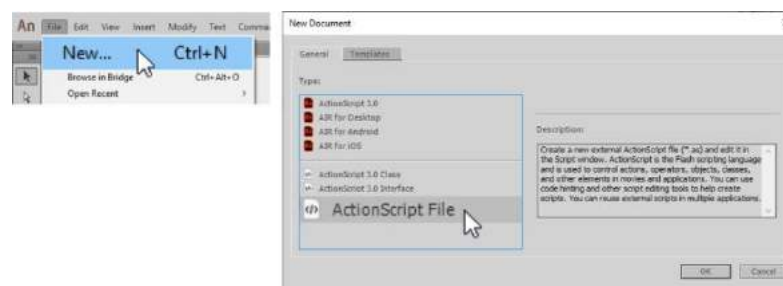
Untuk menambahkan suara ke dalam game, ikuti langkah-langkah berikut:

1. Siapkan file suara yang akan diimport ke dalam game, berikan nama yang spesifik dengan akhiran **_snd** untuk mempermudah identifikasi, misal untuk suara koin ketika pemain membeli item digunakan nama file **gold_snd.wav**, untuk musik di halaman pembuka digunakan nama file **tittle_snd.mp3**, dan seterusnya.



Gambar 12.1 File suara untuk game

2. Buatlah sebuah file *actionscript* baru dengan memilih menu **File>New** dan memilih opsi **ActionScript File**.

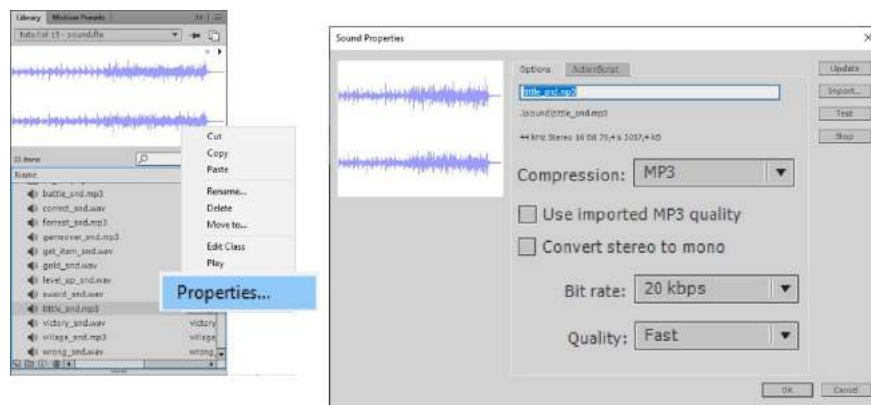


Gambar 12.2 Membuat file *actionscript*

3. Klik panel **action** dan ketikan kode untuk mengidentifikasi file suara sebagai berikut :

```
1. suara_aktif = true;  
2. //masing-masing disesuaikan dengan nama file suara  
3. suara_halaman_pembuka = "tittle_snd";  
4. suara_peta = "village_snd";  
5. suara_battle_arena = "forrest_snd";  
6. suara_mode_pertempuran = "battle_snd";  
7. suara_game_over = "gameover_snd";  
8. suara_level_up = "level_up_snd";  
9. suara_koin = "gold_snd";  
10. suara_dapat_item = "get_item_snd";  
11. suara_menang = "victory_snd";  
12. suara_serang = "sword_snd";  
13. suara_benar = "correct_snd";  
14. suara_salah = "wrong_snd";
```

4. Pada kode di atas, nama masing-masing file dimasukkan ke dalam variabel-variabel suara yang telah diatur sedemikian rupa dalam kode **flashRPG.as**. Simpan file tersebut dengan nama **suara.as**.
5. Buka file **tutorial-18.fla** kembali dan simpan dengan nama lain **tutorial-19.as**.
6. Import seluruh suara ke *library* dengan memilih menu **File > Import > Import Library**.
7. Pada panel **library**, klik kanan objek suara dan pilih opsi **Properties**. Pada panel **option**, pilih tipe **compression MP3**, dan matikan opsi **use imported MP3 quality**. Matikan juga opsi **Convert stereo to mono**. Lalu pilih **Bitrate 20 Kbps** dan **Quality Fast**. Hal ini dimaksudkan untuk mereduksi ukuran file akhir dengan menurunkan kualitas audio. Apabila anda ingin audio dengan kualitas tinggi, anda tidak perlu melakukan langkah ini, dengan konsekuensi ukuran file akhir akan bertambah sesuai dengan ukuran file suara yang diimport.



Gambar 12.3 Pengaturan kompresi suara

8. Dalam pengembangan game, kita perlu memperhatikan ukuran akhir dari file yang akan dihasilkan. Dengan mengompresi suara dengan pengaturan di atas, kita dapat mereduksi ukuran file suara menjadi 3-6% dari ukuran aslinya, dengan sedikit penurunan kualitas suara. Namun jika ukuran file tidak menjadi masalah, maka proses reduksi tidak perlu dilakukan untuk mempertahankan kualitas suara.



Gambar 12.4 Komparasi ukuran file

9. Selanjutnya, klik **frame 1** dan buka panel **action**, tambahkan kode untuk membuka file database suara yang dibuat pada langkah 3, menjadi sebagai berikut :

```
1. include "map-2c.as";
2. include "map-3c.as";
3. include "map-4c.as";
4. include "map-5c.as";
5. include "map-6.as";
6. include "map-7.as";
7.
8. include "flashRPG.as";
9. include "dialog.as";
10. include "item_db.as";
11. include "enemy_db.as";
12. include "soal_db.as";
13. include "suara.as";
14.
15. soal_quiz = soal;
16. battle_arena = map_7;
17. char_sprites_name = "karakter_1";
18. karakter_x = 18;
19. karakter_y = 7;
20. map_start = map_2;
21. halaman_awal("cover_bg");
```

22. Jalankan permainan dengan menekan **Ctrl+Enter**, maka akan didapati sebuah permainan utuh lengkap dengan suara.

Flash Tips

Jika anda ingin menambahkan suara lain, mengkostumisasi suara yang ada anda dapat mengedit file flashRPG.as terdapat fungsi **play_sound(nama link, loops);** Fungsi **play_sound** berfungsi untuk menambahkan suara dari *linkage* library sebanyak *loops* kali. Jadi **play_sound("tittle_snd", 99);** akan memutar suara "tittle_snd" sebanyak 99 kali.

BAB XIII

PUBLISHING

Setelah permainan melalui tahapan pengembangan baik itu pengembangan aset visual, pemrograman dan audio, maka tahapan terakhir adalah melakukan finalisasi atau *publishing* menjadi produk siap pakai.

13.1 Tahap Terakhir dalam Pengembangan Game

Pada tutorial 8 – 20 telah dilakukan proses pengembangan game RPG yang meliputi pengembangan aset visual, pemrograman dan penambahan audio. Hasil akhir dari tutorial 20 merupakan sebuah aplikasi permainan yang secara komplit bisa dimainkan. Namun file *output* dari tutorial tersebut berupa file SWF, dan tidak semua komputer atau perangkat elektronik mendukung format SWF tersebut. Untuk itu diperlukan tahapan selanjutnya yaitu proses *publishing*.

Publishing merupakan tahapan untuk menghasilkan produk siap pakai, dalam hal ini game harus mudah *diinstall* pada perangkat tertentu. Dalam buku ini *publishing* akan menghasilkan dua luaran yaitu game untuk PC atau desktop dengan tipe file EXE, dan game untuk perangkat *mobile phone* berbasis *Android* dengan tipe file APK. File EXE dapat dijalankan secara langsung pada PC, seperti *software* pada umumnya, sedangkan APK akan ditransfer ke *mobilephone* dan *diinstall* secara manual, atau *diupload* ke Google Play Store.

Sebelum melakukan tahapan terakhir yaitu *publishing*, diperlukan beberapa persiapan-persiapan sebagai berikut :

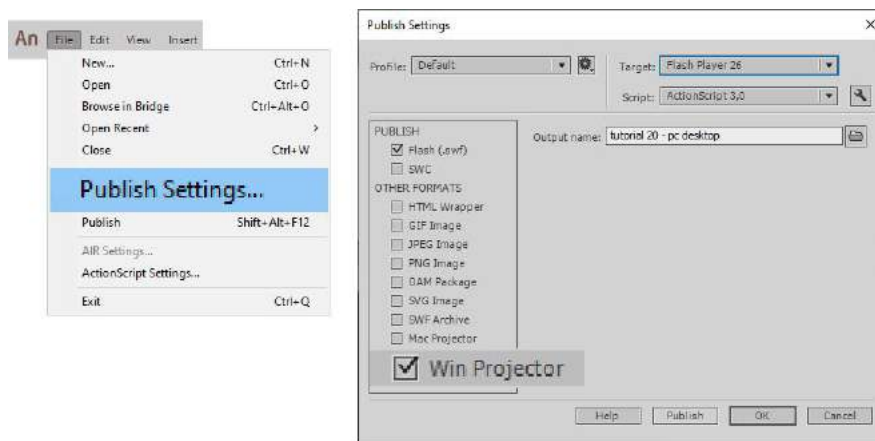
1. *Testing dan Quality Control*. Pada tahapan ini, permainan diuji coba berkali-kali untuk mengetahui jika ada sebuah kesalahan (*bug*). Pengujian perlu dilakukan oleh beberapa orang dengan latar belakang yang berbeda-beda, untuk mengetahui bagaimana perilaku pemain terhadap game. Fitur-fitur yang ada di dalam game seperti tombol, fitur dialog, animasi, mode pertempuran, keterbacaan huruf dan audio diuji sedemikian rupa untuk mengetahui kesalahan-kesalahan yang ada.
2. *Balancing*. Pada tahapan ujicoba, dilakukan pencatatan terhadap perilaku pemain. Apakah secara umum pemain kesulitan pada awal permainan?, jika iya level dan kemampuan musuh harus diturunkan. Apakah pemain sulit untuk mendapatkan koin atau pengalaman? Berarti beberapa atribut harus disesuaikan ulang. Proses ini dilakukan beberapa kali, hingga mendapatkan game yang nyaman untuk dimainkan.
3. *Platform Testing* pada beberapa tipe gawai. Khusus untuk luaran berupa *mobile phone* diperlukan testing berulang pada beberapa tipe gawai. Beberapa gawai memiliki resolusi dan aspek rasio yang berbeda-beda, hal ini yang harus dipahami oleh pengembang. Kode flashRPG.as memiliki kemampuan untuk

mengadaptasi ukuran layar, namun demikian tetap diperlukan tes dan penyesuaian khusus, agar aplikasi tampil prima di semua tipe gawai.

13.2 Publishing untuk PC/Desktop

Proses *publishing* yang paling mudah untuk dilakukan adalah *publishing* untuk PC/Desktop. Perhatikan langkah berikut :

1. Buka file **tutorial-20.fla**
2. Pastikan anda sudah melakukan proses *testing*, *quality control* dan *balancing* secara berulang untuk mendapatkan game yang nyaman dimainkan.
3. Pilih menu **File > Publish Setting**. Centang opsi **Win Projector** pada panel **Publish settings**.



Gambar 13.1 Mengatur *publish settings* untuk PC/Desktop

4. Tekan tombol **Publish** dan file bertipe EXE akan terbentuk setelah proses *publishing* selesai. Pada tahapan ini, sebuah file bertipe EXE tersebut sudah dapat dimainkan di komputer manapun.



Gambar 13.2 Hasil *publishing* berupa file EXE (*Application*)

13.3 Publishing untuk Android

Android merupakan sebuah sistem operasi untuk *mobilephone* yang sangat populer dengan pengguna lebih dari 200 juta di Indonesia saja. Hal ini tentu saja merupakan sebuah keunggulan tersendiri apabila game kita bisa dimainkan pada perangkat *Android*.

Sebelum kita mempublish game menjadi file aplikasi Android, maka kita membutuhkan beberapa penyesuaian, mengingat atribut dari gawai / *mobilephone* sangat berbeda dengan PC/Desktop. Penyesuaian tersebut diantaranya adalah :

1. *Virtual Keyboard.*

Game yang kita buat pada dasarnya menggunakan input *keyboard* dengan tombol panah dan tombol spasi sebagai pengendali utama. Permasalahan yang muncul adalah sebagian besar menggunakan layar *touchscreen*, sehingga diperlukan *virtual keyboard*. Kode dalam **flashRPG.as** mendukung penambahan *virtual keyboard* melalui fungsi **draw_vk()**; Untuk mengaktifkannya cukup dengan menambahkan baris :

```
1. game_output = "android";
```



Gambar 13.3 *Virtual Keyboard*

2. Resolusi layar dan aspek rasio

Resolusi layar game yang kita buat adalah 800x600 pixel, sedangkan pada sebagian besar *mobilephone* memiliki resolusi layar yang bervariasi. Meskipun variasi ukuran layar sangat banyak, kita dapat mengambil rata-rata aspek rasio dari layar *mobilephone* yaitu 2:1, sehingga sebelum publishing nantinya kita perlu merubah ukuran layar dengan aspek rasio 2:1 menjadi 1200x600 *pixel*.

3. GUI (*User Interface*)

Layar *mobilephone* dengan aspek rasio rata-rata 2:1 dan ukuran yang relatif kecil dibanding layar komputer, membutuhkan penyesuaian tampilan antar muka permainan. Tanpa penyesuaian, grafik akan terlihat sangat kecil dan menjadi tidak nyaman untuk dimainkan. Kode **flashRPG.as** secara otomatis menyesuaikan ukuran *interface* menjadi lebih besar ketika berada di layar *Android*. Jika anda ingin melakukan penyesuaian ulang, anda dapat mengedit baris yang memiliki kondisi **if** sebagai berikut :

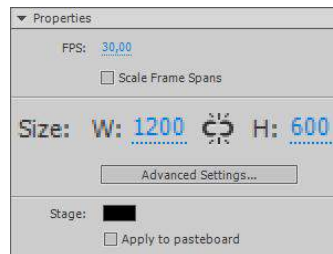
```

1. if (game_output == "android") {
2.     ....
3. }

```

Setelah memahami penyesuaian yang harus dilakukan, maka kita dapat mulai melakukan proses *publishing*. Perhatikan langkah berikut :

1. Buka file **tutorial-20.fla** kemudian simpan dengan nama lain (**tutorial-20 Android.fla**)
2. Pada panel **properties** ubahlah ukuran *stage* dari 800 x 600 menjadi **1200 x 600 pixel**.



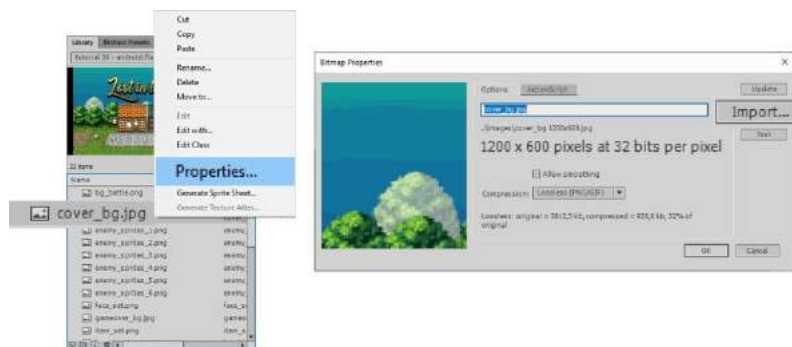
Gambar 13.4 Mengubah ukuran stage

3. Siapkan file grafik untuk **halaman pembuka** dan halaman *game over* dengan ukuran 1200 x 600 *pixel*.



Gambar 13.5 Halaman pembuka dengan resolusi 1200 x 600 *pixel*

4. Pada panel **Library**, klik kanan gambar halaman pembuka **cover_bg** dan pilih opsi **properties**. Kemudian klik tombol **import** dan pilih file baru yang memiliki resolusi 1200 x 600 *pixel*. Langkah ini digunakan untuk mengupdate gambar, tanpa merubah *linkage* dari gambar. Lakukan hal yang sama pada gambar untuk halaman *game over* (**gameover_bg**).

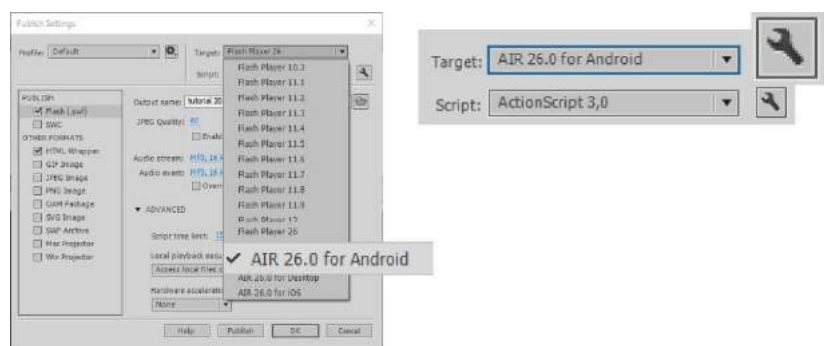


Gambar 13.6 Mengupdate gambar cover_bg

5. Klik **frame 1**, buka panel **Action** kemudian ubah kodenya menjadi sebagai berikut :

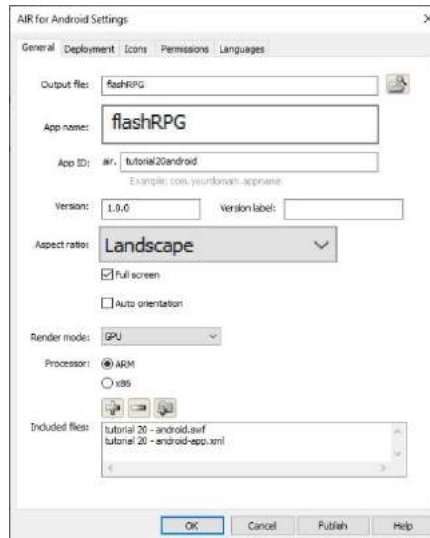
```
1. include "map-2c.as";
2. include "map-3c.as";
3. include "map-4c.as";
4. include "map-5c.as";
5. include "map-6.as";
6. include "map-7.as";
7.
8. include "flashRPG.as";
9. include "dialog.as";
10. include "item_db.as";
11. include "enemy_db.as";
12. include "soal_db.as";
13. include "suara.as";
14.
15. screen_w = 1200;
16. screen_h = 600;
17. game_output = "android";
18.
19. soal_quiz = soal;
20. battle_arena = map_7;
21. char_sprites name = "karakter_1";
22. karakter_x = 18;
23. karakter_y = 7;
24. map_start = map_2;
25. halaman_awal("cover_bg");
```

6. Klik menu **File > Publish Settings**. Kemudian pada **target** pilih opsi **AIR for Android**. Selanjutnya, tekan icon kunci pas, untuk membuka pengaturan **Android**



Gambar 13.7 Mengatur target untuk *AIR for Android*

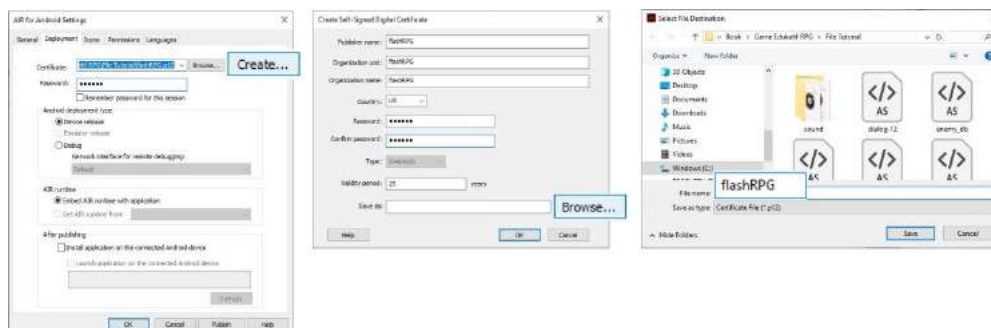
7. Pada panel **Android Settings** tab **General**, ketikkan nama aplikasi, misal **flashRPG**. Pilih **Aspect Ratio Landscape** dan pilih **render mode GPU**. Lalu bergeserlah ke tab selanjutnya.



Gambar 13.8 Pengaturan tab *general Android Settings*

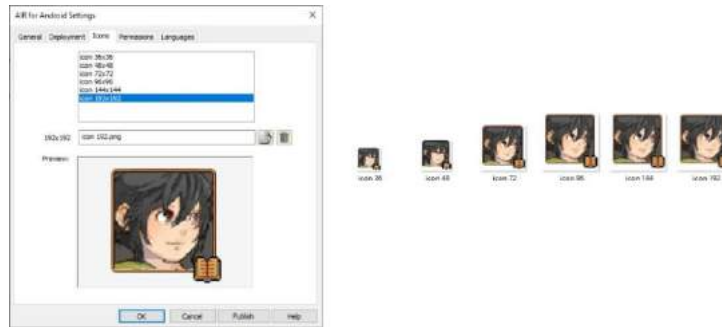
8. Pada tab selanjutnya, yaitu tab **Deployment**, anda membutuhkan sebuah file sertifikat P12, yang secara umum digunakan untuk mengembangkan aplikasi berbasis *Android*. Pada awal pengembangan anda dapat membuat file sertifikat dengan menekan tombol **create**. Ketikkan nama sertifikat, misal **flashRPG**, ketikkan password misal **123456**, kemudian klik tombol **Browse...** Pilih folder tempat anda bekerja, kemudian ketikkan nama file **flashRPG.p12** dan tekan tombol **save**. Setelah beberapa saat file sertifikat akan terbentuk.

Kembalike tab **Deployment** dan pilih sertifikat yang barusaja dibuat, dan ketikkan pula password dari sertifikat tersebut. Kemudian pindah ke tab selanjutnya.



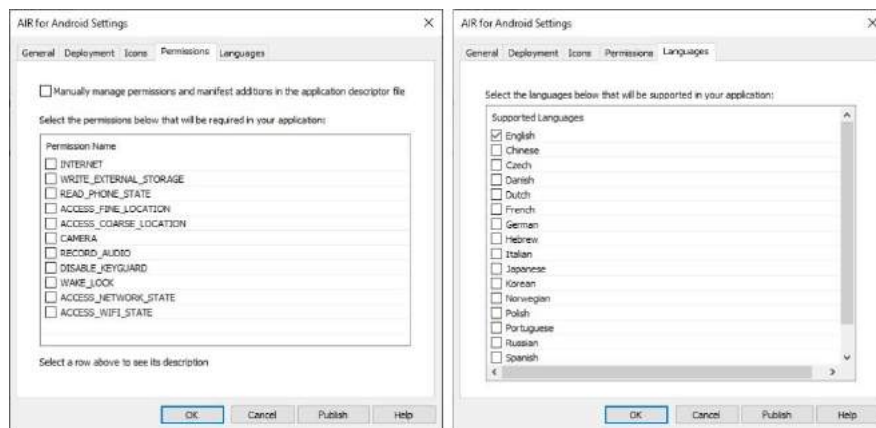
Gambar 13.9 Proses pembuatan sertifikat

9. Pada tab **Icon** siapkan gambar bertipe PNG, yang akan digunakan sebagai icon aplikasi. Ukuran ikon bervariasi mulai dari 36 x 36 *pixel* sampai 192x192 *pixel*. Icon ini nantinya akan muncul pada layar gawai *Android* dan menyesuaikan ukuran icon dengan tema yang sedang aktif. Pilih tombol **Browse** dan pilih masing-masing icon sesuai dengan ukurannya. Kemudian geser ke tab selanjutnya.



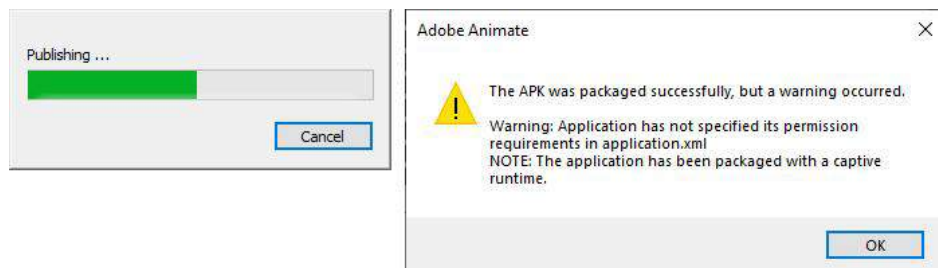
Gambar 13.10 Pengaturan icon aplikasi

10. Pada tab selanjutnya yaitu **Permission** kita lewati, karena game yang kita buat tidak membutuhkan izin untuk akses khusus. Sedangkan pada tab **Language** anda bisa memilih bahasa Inggris.



Gambar 13.11 Pengaturan tab *permissions* dan *Language*.

11. Setelah seluruh pengaturan selesai, tekan tombol **Publish** dan setelah proses selesai akan dihasilkan file **APK** yang siap *install* di gawai berbasis *Android*.



Gambar 13.12 Proses *Publishing*.

Flash Tips

Setelah proses *publish* selesai anda dapat menguji dengan memindah file APK dari komputer ke *mobilephone* melalui kabel transfer USB. Setelah itu anda bisa melakukan penginstalan secara manual, dan melewati proses cek keamanan *Google Play Protect*.



Gambar 13.13 Hasil game pada gawai *Android*

13.4 Tahapan Pengembangan Selanjutnya

Game yang kita buat pada dasarnya sudah dapat dimainkan secara penuh, namun terdapat pengembangan yang belum dijelaskan sampai selesai pada buku ini. Adapun beberapa pengembangan yang dapat anda lakukan untuk memperkaya game edukasi RPG ini antara lain :

1. Pengaturan nama pemain pada halaman awal
2. Sistem cerita yang lebih kompleks
3. Sistem pengaturan suara permainan (on/off)
4. Sistem penyimpanan progres permainan
5. Penambahan *database* musuh yang lebih banyak
6. Penambahan sistem peta yang lebih kompleks
7. Sistem pengaturan perlengkapan pemain
8. Mode pertempuran yang lebih variatif dengan sistem kuis yang berbeda

Dan seterusnya, proses selanjutnya dapat anda ikuti pada blog saya www.wandah.org

BAB XIV

PENUTUP

Setelah memahami proses pembuatan game edukasi RPG, tahapan selanjutnya adalah mengembangkan kreatifitas, menambah fitur dan memperkaya game edukasi di Indonesia.

PENUTUP

Dalam 20 tahun lebih di bidang pengembangan game berbasis Flash, saya telah membuat berbagai jenis game. Pada 10 tahun pertama, tujuan game yang saya buat cenderung ke arah permainan yang menyenangkan, *fun* menjadi elemen yang paling dominan. Sepuluh tahun berikutnya lebih cenderung ke arah permainan yang memiliki konten edukatif, dan saya menemukan sesuatu yang berbeda. Menyampaikan materi edukasi melalui game jauh lebih bermanfaat daripada sekadar memberikan kesenangan melalui game. Meskipun belum memberikan dampak positif yang signifikan, setidaknya kita mulai mengurangi dampak negatif dari game, dan mulai menggantikannya dengan dampak positif.

Proses pengembangan game edukatif bertipe RPG dalam buku ini telah selesai dijelaskan dari awal sampai menghasilkan luaran game PC dan *Android*. Namun, tugas kita tidak berhenti sampai disini. Penambahan fitur dan pengelolaan konten edukatif yang lebih kreatif merupakan tugas lanjutan yang harus diselesaikan. Secara teknis pembaca juga saya ajak untuk berbagi hasil pengembangan dari tutorial yang ada di buku ini melalui situs www.wandah.org/game-edukasi-RPG atau diskusi melalui email wandah@wandah.com

Buku ini dan tutorial yang ada di dalamnya masih jauh dari standar industri game yang ada. Buku ini hanyalah sebuah pengantar atau referensi pendamping bagi seseorang yang ingin memulai belajar mengembangkan game. Oleh karena itu saya menyadari berbagai macam kekurangan yang ada dalam buku ini. Akhirnya saya sangat mengharapkan kritik dan masukan dari para pembaca, *game developer*, akademisi dan seluruh *stakeholder* terkait. Semoga buku sederhana ini menjadi pemicu bagi developer game di Indonesia dan sebagai pemicu bagi saya untuk aktif menulis dan berbagi. Insya Allah bermanfaat.....

Tentang Penulis



Wandah Wibawanto S.Sn., M.Ds. lahir di Malang, 28 Januari 1983. Menyelesaikan program S1 Desain Komunikasi Visual di Universitas Negeri Malang pada tahun 2006 dan S2 Magister Desain (*Game tech*) ITB pada tahun 2012 dan saat buku ini ditulis sedang menempuh program S3 Pendidikan Seni Universitas Negeri Semarang. Mendalami pengembangan game Flash sejak tahun 2001 dan mengkhususkan pada pengembangan game *browser*.

Pada tahun 2006 bekerja di sebuah perusahaan game asing yaitu FreeOnline Games.com dan menghasilkan beberapa karya game yang cukup populer, di antaranya adalah Sim Taxi, Gangster Life, The Empire, dan beberapa game lainnya. Tahun 2008 memutuskan untuk mengembangkan game flash secara independen dan menghasilkan beberapa game flash yang ditujukan untuk beberapa game portal seperti Armorgames, Gameninja, Gamebooks, DailyFreeGames, dan Froyogames.com. Mendapatkan beberapa penghargaan di bidang aplikasi games seperti pemenang INAICTA 2009 kategori digital media, *Honorable mention Dictionary Games award*, *FOG daily game programing challenge* dan beberapa lainnya. Tahun 2014 bergabung menjadi dosen DKV Universitas Negeri Semarang, menjadi peneliti di bidang ekonomi kreatif, dan pengembangan media edukasi.

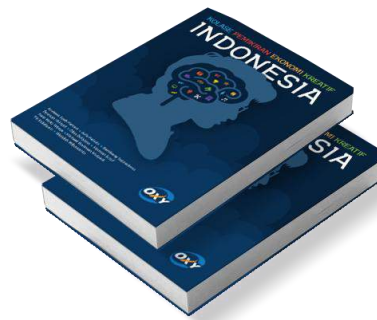
Beberapa buku yang telah ditulis terkait dengan pengembangan Aplikasi antara lain :

- Membuat Game dengan Flash (2003) Penerbit Andi, Yogya
- Dasar-dasar Pemrograman Flash Game (2006) e-book
- Membuat Flash Game 3D (2013) Penerbit Andi, Yogya
- Desain dan Pemrograman Multimedia Interaktif (2017) Penerbit Cerdas Ulet Kreatif, Jember
- Membuat bermacam-macam Game *Android* dengan Adobe Animate (2019) Penerbit Andi, Yogya

Beberapa buku terkait ekonomi kreatif antara lain :

- *Book chapter "Kolase Pemikiran Ekonomi Kreatif"*
- Pendampingan OPD dalam Pengembangan Ekonomi Kreatif Kab/Kota
- Integrasi Rencana Induk Pengembangan Ekonomi Kreatif (Rindekraf) dalam RPJMD Kab/Kota

Beberapa karya aplikasi, tutorial dan materi perkuliahan dapat ditemukan di situs wandah.com, wandah.org dan froyogames.com. Penulis dapat dihubungi via email di wandah@wandah.com atau wandah@mail.unnes.ac.id



Game Edukasi RPG (*Role Playing Game*)

Membuat game bertipe RPG (*Role Play*) merupakan impian bagi para pengembang game, baik pemula maupun tingkat menengah. Di sisi lain, kekompleksan game RPG sering kali menjadi tantangan tersendiri. Buku akan menjelaskan konsep pengembangan game RPG dengan teknik yang sederhana, sehingga diharapkan dapat menjadi salah satu solusi atau referensi bagi pengembangan game RPG. Pada tahapan selanjutnya, muatan edukasi ditambahkan ke dalam game, dengan pendekatan yang lebih kreatif untuk menghasilkan sebuah game edukasi yang menarik untuk dimainkan. Game yang dihasilkan dapat dimainkan pada *platform* PC (*Dekstop*), maupun *mobilephone Android*.

Buku ini sesuai bagi pemula maupun pengembang game tingkat menengah. Tutorial di dalamnya disertai dengan penjelasan konseptual agar mudah dipelajari dan sangat dimungkinkan untuk diaplikasikan dengan software pengembang game lainnya.

Material dan Tutorial dapat diunduh melalui situs
www.wandah.org/game-edukasi-RPG



Penerbit LPPM UNNES

Gedung Prof. Retno Sriningsih Satmoko Lantai 1
Kampus Sekaran Gunungpati Semarang 50229

Telp/Fax : (024) 8508089

Email : lppm@mail.unnes.ac.id

Komputer

12+

ISBN 978-623-7618-61-4

